

LY Workday Pre-MVP Product, System, and Code Audit

Canonical findings register, accepted product boundaries, research traceability, and implementation gates before external beta.

Document ID	LYW-PREMVP-AUDIT-2026-01
Audit period	03 July – 20 August 2026
Calendar span	49 days inclusive
Closed	20 August 2026
Scope	Product · Architecture · Code · Data · Privacy · Security · Operations · Research traceability
Decision authority	Founder, Love Yourself
Analysis	OpenAI Codex under founder direction
Baseline	ac2711da00c3af6bac1690da3458a629d98260f4

Contents

LY Workday — Pre-MVP Product, System, and Code Audit

5

Final audit status

5

Document control

5

Executive audit statement

5

Audit inventory

6

Scope reviewed

6

Technology stack in scope

7

Methodology

7

Limitations and exclusions

8

Deliverable and closure rule

8

Source of truth

9

Audit rules

9

Severity levels

9

New Founder Decisions

10

FD-01 — Automatic same-format continuation after completion

10

FD-02 — Remove the two-hour completion trigger

11

FD-03 — MORNING / DAY / EVENING are internal tags only

11

FD-04 — Onboarding is a deterministic mechanism-sale script

12

FD-05 — No behavioral contingency in MVP; variety by randomness, feedback by explicit ask

14

FD-06 — Exercise Delivery Contract adopted

15

FD-07 — Store three explicit feedback signals in one database table

16

FD-08 — Plan-centric lifecycle: one authoritative source, one derived mode

17

FD-09 — Privacy-gated company analytics and independent aggregation

19

FD-10 — Content Library overhaul: a versioned beta protocol, not an instant-effect catalogue

20

FD-11 — Telemetry measures product interaction, not inferred wellbeing

21

FD-12 — Production data is covered by Railway backups

22

FD-13 — Love Yourself is the company; LY Workday is the product

23

FD-14 — The notification preview is neutral; the exercise lives inside the chat

23

FD-15 — The MVP cycle summary is a deterministic Telegram image

23

FD-16 — Instructional GIFs are required for three technique-sensitive exercises	24
FD-17 — Coach has no user-visible quota, with invisible abuse controls	24
FD-18 — Production access is roster-gated; individual beta runs on an isolated testnet	25
FD-19 — Exercise on demand is an independent user-initiated delivery channel	28
Scheduler Findings	31
SCH-01 — Re-engagement job active	31
SCH-02 — SCHEDULEADJUSTMENT job active	31
SCH-03 — Fast completion after last task action missing	32
SCH-04 — dailypulse disabled	33
SCH-05 — pulse snapshots frozen	33
SCH-06 — adaptation call path must be checked	33
Lifecycle Completion Findings	35
Lifecycle Completion Area — Audit Round 2026-07-03, revalidated 2026-07-16	35
Onboarding Findings	41
Onboarding Area — Audit Round 2026-07-04	41
Coach / Orchestrator Integration Findings	44
Coach / Orchestrator Integration Area — Audit Round 2026-07-16	44
Plan Generation Findings	48
Plan Generation Area — Audit Round 2026-07-18	48
Delivery Renderer Findings	52
Delivery Renderer Area — Audit Round 2026-07-18	52
Runtime Tools Findings	57
Runtime Tools Area — Audit Round 2026-07-22, full-matrix revalidation 2026-07-23	57
Lifecycle State & Persistence Findings	66
Lifecycle State & Persistence Area — Audit Round 2026-07-22, consistency pass 2026-07-23	66
Privacy & Data Boundaries Findings	74
Privacy & Data Boundaries Area — Audit Round 2026-07-30	74
Content Library Findings	80
Content Library Area — Audit Round 2026-08-06	80
Telemetry Area — Audit Round 2026-08-08	84

Database & Redis Integrity Findings	92
Database & Redis Area — Audit Round 2026-08-12	92
Delivery UX & User-Facing Product Surface Findings	96
Delivery UX Area — Audit Round 2026-08-13	96
Security & Configuration Findings	101
Security & Configuration Area — Audit Round 2026-08-15	101
Legacy Reachability Cleanup Findings	107
Legacy Reachability Cleanup Area — Audit Round 2026-08-15	107
Company Deployment Findings	111
Company / B2B Onboarding Area — Audit Round 2026-08-15	111
Release & Operations Findings	119
Release & Operations Area — Audit Round 2026-08-15	119
Exercise on Demand Findings	125
Exercise on Demand Area — Audit Round 2026-08-19	125
Research-to-Product Traceability Findings	133
Research-to-Product Alignment — Audit Round 2026-08-20	133
Miscellaneous Findings	142
MISC-01 — Company-level timezone model for B2B onboarding	142
MISC-02 — Time Picker UX (Phase 3, not MVP)	142
MISC-03 — Remove stale product and architecture documents before production	142
MISC-04 — Finalize the Product Contract after core implementation stabilizes	142
MISC-05 — Synchronize the public website before production	142
MISC-06 — Retire or rebuild development tools against the target model	142

LY Workday — Pre-MVP Product, System, and Code Audit

Final audit status

Status: FINAL / AUDIT DISCOVERY CLOSED · **Version:** 1.0 · **Close date:** 2026-08-20

Document ID: LYW-PREMVP-AUDIT-2026-01 · **Classification:** Confidential — founder and authorized implementation team

This document is the authentic, canonical findings register for the Love Yourself / LY Workday pre-MVP audit. It records the accepted product decisions, factual repository findings, required corrections, frozen or rejected legacy behavior, implementation dependencies, research-to-product traceability, and release gates identified before external beta.

`Audit discovery closed` means the agreed repository and product surfaces have been reviewed and the findings baseline is frozen for implementation planning. It does **not** mean the findings are fixed, the test suite is green, the system is production-ready, or the product hypotheses are validated. Those are separate implementation, verification, beta, and release outcomes.

Document control

Field	Recorded value
Client / product organization	Love Yourself
Employee-facing product	LY Workday
Audit type	Founder-directed internal pre-MVP product, architecture, privacy, operations, research-traceability, and static code audit
Audit period	2026-07-03 through 2026-08-20
Calendar span	49 calendar days inclusive; active labor hours were not metered and are not claimed
Product and decision authority	Founder, Love Yourself
Analysis and report preparation	OpenAI Codex, operating under founder direction and explicit decision approval
Audited application repository	<code>/Users/Baracuda/Desktop/Love Yourself/Tech/ly_wellness_bot_mvp</code>
Committed application baseline	<code>branch docs/plan-generation-audit, commit ac2711da00c3af6bac1690da3458a629d98260f4</code>
Worktree boundary	Static review included the committed baseline and explicitly identified local uncommitted drafts/changes; the dirty worktree is not represented as a releasable commit
Canonical source	<code>docs/audit/pre_mvp_code_audit_findings.md</code>
Companion deliverable	<code>docs/audit/output/LY_Workday_Pre-MVP_Product_System_Code_Audit_2026-08-20.pdf</code>
Evidence corpus	Application repository, Product Contract and maps, 19 primary end-user interview documents, buyer-side discovery, R&D syntheses, evidence reviews, Founder Decisions, and read-only Railway metadata recorded in the relevant rounds
Independence statement	Internal founder-directed, AI-assisted audit; not an independent third-party certification or assurance engagement

Executive audit statement

The existing repository does not implement one coherent beta-ready product. It contains valuable current components alongside multiple historical product models: old onboarding and free-form plan creation, duplicated lifecycle truth, adaptation and engagement remnants, mixed telemetry identity, legacy completion reports, incomplete privacy/deletion boundaries, and non-reproducible release operations. Adding features on top of those contradictions would make beta behavior and resulting metrics unreliable.

The audit therefore defines a smaller target system:

- a privacy-respecting B2B2C product delivered through Telegram;
- short, bounded, versioned workday actions from one canonical Content Library;
- a scheduled channel and an independent user-initiated `Вправа зараз` channel;
- deterministic onboarding, exercise presentation, lifecycle controls, and

cycle summary;

- a reactive Coach whose tools remain bounded by backend authorization;
- PostgreSQL-authoritative lifecycle and event identity;
- telemetry that measures delivery and registered interaction without inferring

wellbeing, burnout, execution, or productivity;

- no hidden behavior-based adaptation, individual employer view, streak

pressure, or speculative expansion before beta evidence.

The repository requires substantial correction before external beta. The dominant work is removal, consolidation, migration, and verification rather than feature accumulation. The research-to-product round additionally concludes that the model is aligned at the problem and trust-boundary level but remains a set of testable hypotheses at the exact channel, cadence, content, Coach, and continuation level.

Audit inventory

The final register contains **19 accepted Founder Decisions** and **216 unique numbered findings** across **19 numbered finding areas**, plus one separate research-to-product traceability round.

Area	IDs	Count
Scheduler	SCH-01...06	6
Lifecycle completion	LIF-01...19	19
Onboarding	ONB-01...08	8
Coach / Orchestrator	COACH-01...12	12
Plan generation	PLAN-01...09	9
Delivery renderer	DEL-01...08	8
Runtime tools	RT-01...15	15
Lifecycle state / FSM	FSM-01...13	13
Privacy and personal data	PRIV-01...10	10
Content Library	CONTENT-01...09	9
Telemetry	TEL-01...15	15
PostgreSQL / Redis integrity	DB-01...20	20
Delivery UX / exercise presentation	UX-01...18	18
Security and configuration	SEC-01...10	10
Legacy reachability cleanup	LEG-01...04	4
Company deployment	COMP-01...10	10
Release and operations	OPS-01...12	12
Exercise on demand	EOD-01...12	12
Miscellaneous / closure items	MISC-01...06	6

Long severity labels in individual findings retain their contextual release gate, but the normalized reporting classes remain **BLOCKER**, **P1**, **P2**, **FROZEN**, **OK**, and **UNCLEAR** as defined below. A finding marked confirmed or accepted may still describe unimplemented target behavior.

Scope reviewed

The audit reviewed the complete application surface relevant to the accepted MVP model:

- all 57 Python modules under `app/`, their import/reachability relationships,

Telegram entrypoints, callbacks, command and Coach routing;

- scheduled delivery, exercise rendering, media fallback, action windows,

completion, pause/resume/cancel/time change, automatic continuation, and on-demand lifecycle;

- onboarding, plan generation/finalization, Content Library schema and assets,

Product Contract, product maps, and user-facing language;

- Coach prompt/tool boundaries, model request handling, conversation memory,

abuse/cost controls, safety and failure fallbacks;

- event contract, activation/retention definitions, feedback, channel

separation, personal telemetry, independent aggregation, and company-facing reporting limits;

- SQLAlchemy models, PostgreSQL constraints and migrations, Redis ownership,

idempotency, concurrency, restart/reconciliation, retention and deletion;

- secrets and configuration, FastAPI endpoints, bearer links, enrollment and

entitlement design, Docker/Railway release topology, backups, observability, health/readiness, rollback, and incident handling;

- all repository tests, development tools, stress-test scripts, active and stale

documentation, templates, static website assets, and release declarations;

- primary end-user interviews separately from secondary R&D interpretation,

buyer discovery, evidence reviews, and the final product constructor.

Technology stack in scope

The reviewed implementation and deployment surface uses:

- Python 3.11 container runtime;
- aiogram 3 / Telegram Bot API polling;
- FastAPI and Uvicorn;
- SQLAlchemy with PostgreSQL / psycopg2;
- Redis and aiogram Redis FSM storage;
- APScheduler;
- OpenAI Python SDK / Responses API for Coach and historical plan components;
- Jinja2 HTML templates and application-managed SVG/media assets;
- Docker and Railway deployment infrastructure;
- SQL migrations, Pytest-based tests, and manual development/stress tooling;
- structured JSON/YAML-style content and event contracts defined by the audit.

This list describes the audited stack, not an endorsement that every pinned version, dependency, process declaration, or tool should survive implementation.

Methodology

The work used a contract-first audit method:

1. establish source authority among Product Contract, later Founder Decisions,

product maps, current code, and historical documents;

2. inspect reachable producers and consumers, not filenames or registered

handlers alone;

3. trace user journeys, durable state, external side effects, event identity,

and failure/restart paths across modules;

4. reproduce selected renderer, lifecycle, and test behavior where the local

environment allowed it;

5. separate current behavior, expected behavior, minimal fix, and intentionally deferred work;

6. apply privacy, concurrency, inversion, failure-mode, and margin-of-safety reasoning only where they changed an implementation or trust boundary;

7. classify primary interview signals separately from secondary synthesis and mark exact product choices as hypotheses where evidence does not validate them;

8. preserve founder product authority: analysis could expose trade-offs and contradictions, while accepted product decisions required explicit founder approval.

Limitations and exclusions

This report must not be represented as more than the work performed:

- it is a static repository and product-contract audit, not a penetration test, formal security certification, SOC/ISO audit, financial audit, or legal opinion;

- stopped production services were not started merely for audit completion;

the physical production PostgreSQL schema and row-level state were not introspected, and must be verified from a restored backup before migrations;

- live Railway ingress, deployed image contents, environment-variable values,

provider budgets, database roles, alert delivery, and current network visibility remain release-gate verification items;

- the audit did not establish one clean, complete, passing test-suite baseline;

collection/environment failures and obsolete tests are recorded in OPS-05;

- no live iOS/Android Telegram acceptance test, load test, failover drill,

backup-restore drill, full incident exercise, or production migration was performed as part of closing the findings register;

- dependency-vulnerability findings were not inferred without a resolved SCA

run; provider compromise is outside the repository threat model;

- legal roles, notices, contracts, cross-border processing, and any DPIA need

require qualified legal review before the first company deployment;

- exercise evidence review does not constitute medical approval. The cool-water

action remains gated, and exact efficacy of all short exercises remains a beta hypothesis;

- interview and R&D traceability can show support, absence, or tension. It

cannot manufacture product-market fit, causality, or representative market demand from the supplied sample;

- the local worktree contains pre-existing uncommitted application and document

changes. They were preserved and reviewed where relevant, but this report does not certify that dirty state as a release artifact.

Deliverable and closure rule

This Markdown file remains the editable source of truth. The companion PDF is a typeset, read-only representation of the same version and must be regenerated after any approved source change. Implementation planning may prioritize and group findings, but may not silently rewrite their product boundary or mark them resolved without code, migration, test, and operational evidence proportionate to the finding.

The next controlled stages are:

```
closed audit baseline
-> prioritized implementation plan
```

```
-> reversible implementation rounds
-> migration and release verification
-> testnet beta
-> evidence review
-> company production gate
```

The goal of the findings register remains to identify what must be fixed, frozen, removed, implemented, or verified before each applicable gate. It is not a backlog mandate to build every deferred idea.

Source of truth

1. `docs/audit/product_contract.md` — baseline product contract.
2. Founder decisions in this file — override baseline when newer.
3. `docs/audit/delivery_contract.md` — accepted product contract for the daily exercise touchpoint (adopted by FD-06).
4. `resource/assets/product/conceptual_map.md` and `conceptual_map_en.md` — synchronized user-facing product facts; lifecycle changes must stay aligned with the founder decisions and contract.
5. Area audit findings — factual code observations, not product decisions.
6. Audit discussions — context only, already distilled into findings below.

Audit rules

- Do not treat this file as a product brainstorm.
- Do not add features unless they are accepted as founder decisions.
- Findings must map to the Product Contract or accepted audit decisions.
- Each finding must have severity, status, current behavior, expected behavior, and minimal fix.
- Coach prompt has been rewritten (all 7 sections, integration pass done, PR #246 open) and is now in scope — see "Coach / Orchestrator Integration Findings" below.
- Old product documents are not source of truth unless explicitly referenced by `docs/audit/product_contract.md`.
- If a finding is unclear, mark it as `UNCLEAR` and specify what file/function must be inspected next.

Severity levels

BLOCKER

Will break beta, trust, privacy, core delivery, or core lifecycle.

P1

Must be fixed before first beta, but not immediately catastrophic.

P2

Useful after first users; should not block beta.

FROZEN

Existing or legacy component that should not be developed now. It may stay if it does not affect core MVP behavior.

OK

Matches current product contract.

UNCLEAR

Insufficient evidence. Needs inspection of another file or code path.

New Founder Decisions

FD-01 — Automatic same-format continuation after completion

Status: accepted Priority: P1 Area: Completion / Retention / Lifecycle

Overrides previous baseline

Old flow:

completion report → user chooses the next 7- or 14-day format

New flow:

completed 7 days → next 7 days are already prepared
completed 14 days → next 14 days are already prepared

Decision

After a user completes a 7- or 14-working-day plan, the system automatically creates the next period in the same format with the same delivery time or times and the same `work_days`.

The completion report is not the end of the relationship. It is a bridge between two plans.

The default is same-format continuation: 7→7 and 14→14.

Default behavior

- create the next plan automatically from the completed plan's canonical format: `SHORT→SHORT`, `MEDIUM→MEDIUM`;
- reuse the same DAY delivery time and, for 14 days, the configured EVENING time;
- reuse the same `work_days`;
- start on the next selected working day;
- do not ask the user to reconfirm the same format;
- do not ask for evening time again when it is already configured;
- do not collect new onboarding data;
- do not push the user toward a different format by default.

User control

User can still:

- stop / cancel;
- pause;
- change time;
- explicitly switch 7↔14 through the dedicated format-switch flow;
- write Coach.

User-facing framing

Example:

Наступні 7 або 14 робочих днів уже готові в тому самому ритмі.
Перший таск прийде [date] о [HH:MM].

Якщо хочеш змінити формат, час або зупинитись — просто скажи.

Rationale

The gap after completion is a high-risk churn point.

Same-format continuation reduces friction while preserving user agency.

The user does not need to recommit to the same behavior they already accepted.

This is not coercion because opt-out, pause, cancel, change time, and switch-to-14 remain available.

Code implications

- completion handler derives the canonical format from the completed plan and creates the same format automatically;
- next plan starts on the next valid workday;
- next plan uses the existing DAY time, existing work_days, and the existing EVENING time for MEDIUM;
- no new onboarding;
- no duration choice;
- no repeated evening-time collection for an existing 14-day format;
- report copy changes from “choose next plan” to “the next period is ready”;
- switching 7↔14 remains an optional explicit action, not the default CTA;
- no mini-onboarding is needed for same-format continuation.

FD-02 — Remove the two-hour completion trigger

Status: accepted Priority: P1 Area: Completion / Scheduler / Lifecycle

Decision

The two-hour completion trigger has no valid MVP role and must be removed.

There are only two completion paths:

```

user completes/skips the last task before expiry
→ trigger completion report quickly, ideally within 1-2 minutes

user does not press completed/skipped
→ task expires at the end of its window
→ completion buttons are removed
→ normal path finalizes and reports on the next selected work day at the saved DAY time
→ fixed UTC cron remains a recovery safety net only

```

Do not use a two-hour delay as lifecycle logic. At that point the task window is still open, the user may still act, and the report metrics may change afterward.

Code implications

- remove `_maybe_schedule_plan_completion` as a two-hour report trigger;
- `task_complete / task_skip` handle the fast active-action path;
- expiry closes unanswered task buttons;
- the normal no-action path resolves the user's timezone, next selected work day, and saved DAY time;
- `check_plan_completions` remains a fixed-UTC recovery safety net;
- the current plan must be finalized before its completion report is sent.

FD-03 — MORNING / DAY / EVENING are internal tags only

Status: accepted Priority: P1 Area: Onboarding / Scheduling / User-facing terminology

Decision

MORNING, DAY, and EVENING are internal technical tags attached to a concrete HH:MM time chosen by the user.

They are not user-facing product concepts and must not appear in user-facing questions, messages, buttons, reports, or plan descriptions.

The user sees:

- the actual time, for example 14:30;
- natural language when needed;

- the logical plan day number only when useful.

The user must not see or choose an internal slot name. In particular, output such as `День 3 · День` must be removed: the first value is a plan day number, while the second is a leaked internal `DAY` tag.

`MORNING` remains frozen legacy metadata and must not be used by the MVP onboarding or first-plan flow.

FD-04 — Onboarding is a deterministic mechanism-sale script

Status: accepted Priority: P1 Area: Onboarding / Activation

Decision

Core onboarding is a **deterministic scripted funnel, not an AI conversation**. It sells the **mechanism** (timely interception at the user's own low-energy moment), not exercises or content. No demo exercise in the base version.

Why a script beats a human/AI here: the funnel must be consistent, measurable, free of extra questions, and guarantee the privacy promise. Human craft goes into writing the script once — not improvising each run. A different pitch per run makes the primary activation metric unmeasurable and puts trust surfaces at risk.

Structure

```
recognition
→ mechanism
→ setup
→ concrete promise + control + privacy
```

1. **recognition** — mirror the user's own chosen moment back with specificity;
no invented psychological conclusions.
2. **mechanism** — one causal line: right moment → ready action → fewer decisions.
3. **setup** — collect one concrete HH:MM time + work_days (trigger setup, not a questionnaire).
4. **concrete promise + control + privacy** — exact first-task date/time, skip/change/stop, and a direct privacy statement.

Accepted rules

- Script, not AI, in the core funnel.
- Privacy answer is **approved fixed text only**; AI must never improvise the answer to "what does my employer see?".
- Competence is created by **specific mirroring** of the user's chosen moment — not by demo, and not by unproven borrowed proof (e.g. do NOT claim "HR already vetted the product").
- Safety stated directly: user is in control; individual answers/completion are not shared with the company/HR.
- Known recurring questions → a short **approved FAQ (scripted)** that grows from real onboarding logs. Do not hardcode a fixed "objection count".
- AI (Coach) may pick up non-standard questions **only after onboarding completes** — never inside the critical funnel, never for privacy. Safety/crisis handling is a **separate guard**, not Coach improvisation.
- One selected moment per day is an existing product invariant (contract 2.4 / t23_t24) — restated, **not a new decision**. Evening only in the 14-day format.
- First real task arrives today **only when the chosen time is still ahead**; otherwise the next work day. No artificial "task now".

Operating hypothesis — value in the first 2 minutes (NOT a proven decision)

The first 2 minutes deliver value not as an "exercise effect" but as three other values: **recognition** ("they understood my specific moment"), **relief** ("I don't have to remember or choose"), **predictability** ("I know exactly when and what will happen"). This is the activation bet to validate in beta.

Explicitly NOT decisions (A/B or rejected — keep out of contract)

- preview of tomorrow's real task message during onboarding — future A/B hypothesis;
- delivering a task "now" when the chosen moment has already passed — risks teaching a pull model; hypothesis only;
- borrowed "HR vetted this" proof — unproven, potential trust hit; rejected.

Note on framing

The proof → promise → plan shape is used as a working structure only. No external framework attribution is recorded (no verified primary source for a named "PPP" framework; only a related "Proof > Promise" principle exists).

Code implications

- onboarding writes DAY time + work_days and transitions to first-plan creation

deterministically;

- confirmation copy uses the real first-step schedule (see ONB-08);
- onboarding must call the canonical `create_plan()` service directly and must

not depend on the dead `IDLE_ONBOARDED` wrapper (see ONB-07).

Non-linear onboarding risks and acceptance constraints

These are product-system risks, not requests for additional MVP features. They must be checked when the script and onboarding implementation are reviewed:

- **Funnel-length risk:** more mechanism explanation can increase understanding while

reducing completion. The script must remain short enough that the user reaches time selection before the sales explanation becomes a new source of friction.

- **Mirroring risk:** specific recognition can create trust, but an invented cause,

diagnosis, or emotional claim can feel manipulative or intrusive. Mirror only what the user actually selected or stated.

- **Promise/reliability coupling:** onboarding copy cannot create durable activation

if the first task misses the promised moment. Successful plan creation and first-task delivery at the confirmed schedule are part of the onboarding acceptance path, not a separate downstream concern.

- **Privacy-truth coupling:** the fixed privacy statement is valid only if actual data

access, reporting, logs, and HR/company surfaces match it. The statement must be verified against the implemented data flow before beta; copy must not promise more than the system guarantees.

- **Off-script input risk:** because AI is excluded from the critical funnel, arbitrary

text during onboarding must not strand the user or silently reset progress. It needs deterministic handling: approved FAQ where matched, separate crisis guard where applicable, and a safe return to the current onboarding step.

- **Control-path risk:** back, edit, restart, stop, and time/work-day correction paths

must preserve a comprehensible state and must not create duplicate plans.

- **Measurement risk:** onboarding completion alone can hide a broken activation loop.

Review the observable chain: onboarding started -> time/work_days saved -> plan created -> first task delivered at the confirmed moment -> user responded.

The script is therefore necessary but not sufficient: its promise, persisted setup, state transition, and first real delivery form one activation system.

FD-05 — No behavioral contingency in MVP; variety by randomness, feedback by explicit ask

Status: accepted (2026-07-22) Priority: P1 Area: Plan Generation / Delivery / Retention Resolves: PLAN-05 (from "P1 defect" to "OK by design")

Decision

Two separate levers, set independently:

- **Content variety (stimulus unpredictability): MAX.** The exercise

sequence is randomized so the user cannot predict tomorrow's exercise. This is a deliberate design property, not a bug. Its randomness derives only from **shown-history** (which exercises were already delivered), never from completion-history.

- **Behavioral contingency: ZERO.** The generated sequence does not

change based on `completed`, `skipped`, or `ignored`. The system never infers *why* a user skipped and never rewards or punishes behavior with future content. Delivery follows the user-selected schedule.

Terminology (corrected 2026-07-22). Do *not* call this "variable reward." In operant terms a variable reward is a variable *consequence of behavior*; a random exercise sequence is not a consequence of anything the user did, so the label is wrong and, worse, it imports the exact TikTok/slot-machine connotation this product avoids. The accurate framing: **content variety is intentionally non-contingent — each cycle receives a different valid sequence, independent of completion, skip, or ignore.** The unpredictability may still drive anticipation/engagement (real psychology), but through novelty of a non-contingent stimulus, not through reinforcement of behavior. These two levers are compatible precisely because the variety is never earned or lost through the user's actions.

Three levels of "learning" — only level 3 is excluded

"Personalization" and "learning" are not one thing. There are three distinct levels, and the ZERO-contingency decision applies **only to level 3**. Conflating them would wrongly forbid work that is actually allowed.

1. **Global product learning (for everyone, equally).** Aggregate metrics

and feedback rewrite weak exercises, retune timing, adjust the library or recipe. The builder gets smarter, but produces the same output for all users. Intended. This is the primary improvement path.

2. **Context personalization by explicit ask.** The product asks the user

about their anchor/context (where they work, when they take breaks — Fogg Model 5, Anchoring) and the builder fits the exercise and time to that stated context. Personal, but collected by **asking, not inferring** — same rule as the feedback button. Ethically clean. Also the real answer to the fixed-14:00 prompt problem: one onboarding question about the user's routine lets both time and exercise fit their actual day, with no tracking. Deferred **only** for onboarding simplicity (funnel-length risk, FD-04), not for any ethical reason. A clean v2, not a forbidden zone.

3. **Behavioral-inference personalization.** The system guesses from

`completed/skipped/ignored` what suits a user and diverges their content. This is the excluded lever. Deferred until real usage data, a direct effect signal, and critical mass exist — "not before critical mass," not "not before v2." This is where the surveillance and adverse- selection risks live.

The dividing line for exclusion is **inference vs asking**, not "personal vs not." Level 2 is personal and allowed because the user states it; level 3 is forbidden because the system infers it.

Why contingency is excluded (not deferred by accident)

- **Map ≠ territory.** `task_completed` does not prove the exercise was

done or helped; `skip` does not prove the exercise is bad (the user may have been on a call). Personalizing on clicks is pseudo-precision.

- **Incentive corruption.** If `completed` unlocked "better" exercises or

`skip` degraded the future, the system would train button-pressing and punish honest answers. In a B2B context this risks the product being perceived as employee monitoring — a fat-tail trust failure that can break the corporate sale even without any individual data reaching HR.

- **Adverse selection.** If future delivery depended on compliance, the

most exhausted users would receive least support exactly when they need it most. Fixed-time delivery requires nothing to be "earned."

- **No statistical base.** At 10–15 users there is no basis for

per-user personalization — only the risk of building complexity around noise.

- **History.** Behavioral adaptation was built in v3 (65 exercises) and

deliberately cut as speculative. v1 ships with none by decision. The `ADAPTATIONS_ENABLED` mentions in code are deletion residue, not a pending feature.

What MVP does keep

- **Different but neutral, non-evaluative response handling.** `completed`,

`skipped`, and `ignored` record distinct states and telemetry (counters, time-to-response), and completion summaries are factual. None of this gates or shapes future content.

- **Optional explicit feedback ("did it help?") — after `completed` ONLY.**

This is the *direct effect signal* that click data alone cannot provide. Critical constraint: the question is asked **only after** `completed`. After `skipped` the user did not do the exercise and cannot rate its effect — bolting an "оцініть вправу" prompt onto a skip is nonsensical and embarrassing. A skip needs a *different* question (why skipped), and that is a separate feature, deferred. For `completed` only: one optional tap; store `exercise_id`, answer, timestamp; change nothing in the current or next cycle; never send an individual answer to the company; use only for aggregated exercise review.

This is a deliberate, scoped **exception to the "no new features" audit rule** — justified because it is architecturally trivial (no plan/state changes) and high-EV for product-level learning. Effort is realistically a few hours, not 15–20 minutes: `UserEvent.context` is already JSONB so likely no migration, but callback, idempotency, keyboard state, storage, and tests remain. It belongs to the **Delivery / Telemetry** area as a small standalone experiment, not to Plan Generation.

Boundary for future work

User-level adaptation may be reconsidered only after real usage data and the accumulated effect signal exist. Completion clicks alone are insufficient. Product-level learning (rewriting weak exercises, re-tuning timing globally) is Bayesian updating of the product, not per-user profiling, and is the intended path.

Code implications

- remove `ADAPTATIONS_ENABLED` mentions and any "pending adaptation" copy;
- disable legacy adaptation/engagement CTAs (see DEL-03);
- selection reads shown-history for variety, never completion-history;
- build the optional feedback capture; wire it to storage only;
- drop the `weight` field for now (PLAN-06) — difficulty grading is a

post-data hypothesis, not an MVP dial.

FD-06 — Exercise Delivery Contract adopted

Status: accepted (2026-07-22) Priority: P1 Area: Delivery / product touchpoint

The daily exercise notification is the single primary product touchpoint. Its product contract is `docs/audit/delivery_contract.md`, adopted here as an authoritative source of truth (see Source of truth #3). The delivery findings (DEL-01...DEL-08) check the code against it.

Key accepted points (full text in the contract):

- channel-neutral shape `title · duration · ordered steps · Done/Skip`;

first line self-sufficient for the push preview;

- no rationale in the message — **not before and not after** (removed

100%); no counters, mechanic/slot/internal IDs, persona copy, or a Coach message right after delivery;

- on tap, **edit the same message** (not a new one); buttons change with

state; after `Виконано` show the optional completed-only feedback tap; after `Пропустити` show no feedback;

- Mini App / rich cards deferred (post-beta, selective, only where

interaction makes the action *easier*);

- delivery timing / anchor is a reasoned MVP stance + open beta learning

goal — the onboarding time question is purpose-framed and scoped to the workday, the time is a likely-good moment not a deadline, company anchor is recommended-only, and no delivery time/content adapts automatically in MVP.

FD-07 — Store three explicit feedback signals in one database table

Status: accepted (2026-07-23) Priority: P1 for beta learning Area: Feedback / Coach / Delivery / Telemetry

Decision

The beta collects three different feedback signals. They answer different product questions but share one storage model:

1. `exercise_efficacy` — optional one-tap feedback after an exercise was

marked completed (already accepted under FD-05/FD-06);

2. `coach_quality` — thumbs up/down attached to free-form Coach responses,

with a short reason after thumbs down;

3. `product_feedback` — feedback the user explicitly asks to submit about

Love Yourself, captured through natural conversation and a dedicated `capture_product_feedback` runtime tool.

All three are stored in one `feedback_events` table with a `source` field. The database is the source of truth. Beta scope is capture only: no email notification, support inbox, admin Telegram channel, dashboard, automatic triage workflow, or response-time promise is required.

Explicit-intent boundary

`capture_product_feedback` is available only when the user clearly asks to submit or pass feedback to the product/development team. A direct request that already contains the feedback is sufficient consent.

Examples that allow capture:

```
Хочу залишити відгук про продукт: ...
Передай розробникам, що ...
Залиши це як фідбек: ...
```

A complaint, question, frustration, or negative phrase alone is not consent to store product feedback. For example, "я не розумію, коли приходять вправи" is first a product-support question: Coach answers it from the Product Map. It is captured only if the user also clearly asks for it to be submitted.

Do not require the feedback topic to be absent from the Product Map. The map determines whether Coach can answer a factual question; it does not restrict what the user may criticize or submit as feedback.

If the user expresses intent to leave feedback but has not said what should be submitted, ask one brief question. Do not infer feedback text or submit ambient conversation automatically.

Classification and fidelity

Classification happens only after explicit submission intent is established. It may sort feedback into a small enum such as `bug`, `confusion`, `feature_request`, `content`, `coach`, or `other`, but classification never acts as the trigger.

Store the exact source message and any model-extracted submitted text separately. The original user wording is authoritative; a category or summary must not replace it. Do not attach the full conversation by default.

`product_feedback` is not COACH-10 escalation:

- feedback records an opinion and does not promise a human response;
- escalation asks a human to resolve an unanswered product question and may

require a response path;

- they may share infrastructure later, but their user-facing contracts remain

distinct.

Storage shape

Minimum shared fields:

```
id
user_id
source                # exercise_efficacy / coach_quality / product_feedback
source_entity_id      # exercise, Coach message, or source Telegram message
value                 # source-specific rating/answer
reason                 # optional structured reason
feedback_text          # optional submitted text
category               # optional post-intent classification
context                # bounded JSON: plan/exercise/prompt/model/state identifiers
created_at
```

Do not make feedback company-facing. Feedback may contribute only to product-internal learning and privacy-safe aggregates under FD-09; no individual answer, free text, or small-group slice is available to a company. Feedback does not change the current or future exercise sequence for the individual user.

Source-specific interaction rules

- `exercise_efficacy`: show only after completed, never after skipped;

one optional tap, no plan adaptation.

- `coach_quality`: attach only to free-form Coach responses, not scheduled

exercises or deterministic tool confirmations. A negative rating may expose a compact reason row (`incorrect`, `irrelevant`, `tone`, `too_long`, `other`). One user/message pair must be idempotent and may update its latest rating rather than create duplicates.

- `product_feedback`: no slash command is required. Natural-language explicit

intent invokes the tool. Confirm storage only after the database write succeeds; do not promise that a person will reply.

Deferred deliberately

- outbound email to `hello@loveyourselfua.com`;
- admin/support Telegram channel;
- feedback dashboard or issue-tracker integration;
- automatic alerts and triage;
- automatic prompt, exercise, timing, or per-user plan changes based on

feedback.

Feedback is reviewed in batches during product iteration. If volume later makes manual database review a bottleneck, notifications or a review surface can be added without changing the capture contract.

FD-08 — Plan-centric lifecycle: one authoritative source, one derived mode

Status: accepted (2026-07-23) Priority: P1 before beta (not a "fix today" — see sequencing) Area: FSM / State / Lifecycle
Resolves the open architectural decision in the FSM round (FSM-04 / FSM-08).

Decision

The user lifecycle has **one authoritative representation, and one derived one** — not three parallel sources that can diverge. Today `users.current_state`, `UserProfile.is_paused`, and `AIPlan.status` each independently describe the same lifecycle; the FSM round proved they drift (FSM-06/08/09). This is deliberately collapsed:

- **Authoritative (owns lifecycle truth):**
- onboarding progress — owns setup state before a plan exists;
- `AIPlan.status` (`active` / `paused` / `completed` / `abandoned`) — owns

plan lifecycle. `paused` **is wired** (the enum already has it; pause sets `plan.status = paused`, resume sets `active`);

- step statuses — own exercise execution;
- one plan completion date **derived from the actual last step**, not a

separate stored authority.

- **Derived (never stored as a second source of truth):**
- `current_mode` (`ONBOARDING / ACTIVE / ACTIVE_PAUSED /`

`NO_ACTIVE_PLAN`) — computed at runtime by **one** function and handed to Coach / UI / tool-filtering. It is a view, not a column.

Removed as duplicate lifecycle authorities

- `users.current_state` — removed as a stored lifecycle source (derive `current_mode` instead);
- `UserProfile.is_paused` — removed; pause lives in `plan.status`;
- `User.plan_end_date` — removed as a separate authority; derive from steps.

`pause_count` survives as **pure telemetry**, not a lifecycle signal.

Why (the founder rationale)

Keeping three sources means maintaining **two systems that can disagree** — a "plan signal" system and a "user state" system — and every new lifecycle feature must keep both in sync. The correct move is to name the authoritative one (the plan) and make the rest derived. Not doing this compounds: each new consumer multiplies invalid combinations, developers pick the convenient field (architectural Gresham's law), tests green-light a broken whole, races produce duplicate plans / wrong reports, telemetry becomes untrustworthy (risking a false "the exercises don't work" conclusion when the real fault is lifecycle data), and the migration only gets more expensive once real user data exists. At 0 users this is the cheapest possible moment to collapse it.

Hard dependencies (this is a refactor + migration, not a column drop)

- **One derivation authority:** `current_mode` must be computed by a single

shared function used everywhere, or drift is merely relocated.

- **One current plan per user:** a partial unique constraint allowing at most

one `AIPlan` in `active/paused` per user is **mandatory** (plan-centric is ambiguous with two active plans), plus a dedup of any existing multiples — see FSM-12.

- **Reader refactor:** scheduler, Coach tool-filtering, task callbacks, and

runtime tools currently read `current_state`; they move to the current plan / derived mode. Incremental, behind one migration.

- **Recovery is outbox, not a state:** unfinished automatic continuation is an

idempotent outbox/reconciliation record, never a lingering `IDLE_FINISHED` used as an improvised recovery queue.

Sequencing (do not big-bang; do not patch the hybrid further)

1. finish the audit;
2. this decision (done);
3. one refactor moving lifecycle onto `AIPlan.status` + derived `current_mode`;
4. add the one-current-plan constraint (FSM-12) and dedup;
5. one forward migration + operation-level integration tests;
6. remove `current_state / is_paused / plan_end_date` last.

Until then the hybrid may stand for the remaining audit days, but **do not** build format switch (RT-07), wire automatic continuation (LIF-04), extend pause/resume, or start beta on top of it — each new patch grows the future operation.

Note

FSM survives as a **concept and a derived view**, not as a duplicated stored column. `app/fsm/states.py` (the value/mode vocabulary) stays as the canonical mode enum; `app/fsm/guards.py` (the transition matrix) is removed per FSM-04.

FD-09 — Privacy-gated company analytics and independent aggregation

Status: accepted (2026-07-30), company-reporting boundary amended 2026-08-15 Priority: P1 data architecture before beta; reporting gate before the first company pilot closes Area: Privacy / Telemetry / B2B Analytics

Decision

Love Yourself may retain user-linked behavioral telemetry for product operation and product learning while the account exists. It must also build independent statistical aggregates from accepted events at ingestion time. Account deletion removes the user-linked history but does not subtract a contribution that has already become genuinely aggregate.

This is **parallel aggregation from day one**, not a transfer performed during account deletion:

```
accepted behavioral event
-> user-linked event
-> independent aggregate contribution
```

The operation must be idempotent. Retries cannot increment the aggregate twice, and a partial failure cannot silently leave personal and aggregate records inconsistent. The implementation may use one database transaction or an event plus a durable outbox, but it must have one stable event identity and an observable reconciliation path.

Independent aggregates contain no user identifier, stable user hash, raw conversation or feedback text, exact user-level timeline, or field combination that can reconstruct a person's history. High-cardinality metrics use coarse buckets and bounded contribution. Product-internal aggregate metrics may include completion / skip / ignore counts, response-latency buckets, streak buckets, plan completion, activation, and D3/D7 retention. Response latency is delivery-to-response time; it is not exercise duration. These aggregates may be retained indefinitely once they meet this independence standard.

An idempotency/outbox ledger may temporarily retain an event identity to make retries safe, but that ledger belongs to the personal/operational retention boundary. The indefinite aggregate counters do not retain the event identity or another join key back to the personal event.

Company-facing boundary

During a pilot the company receives no live behavioral view, employee-level data, manual spreadsheet, or improvised "anonymous" report. The current code has no company dashboard or HR endpoint; preserve that safe default.

After the pilot, the company may receive **one aggregate product-usage summary**, and only from the independent aggregate layer after both design gates are satisfied:

- at least **100 eligible users** in the covered cohort;
- at least **50 distinct actual contributors** in the reporting period.

Both conditions are required. Invited or licensed users who did not contribute data do not increase the effective cohort. The threshold is centrally controlled and cannot be lowered by HR, the buyer, or a company administrator. Reports use broad periods, coarse buckets, rounding, small-cell suppression, and no person-level comparison. The 100/50 rule is a conservative starting default, **not a mathematical anonymity guarantee**; differential privacy and formal contribution limits remain future hardening if stronger guarantees are needed.

The accepted post-pilot summary contains only:

```
Employees eligible at launch:      200
Deployment enrollments created:    150
Active in the stated 7- or 14-day window: 90
```

Every label has an event-contract definition. `Deployment enrollment` means a valid one-time invitation created an enrollment; it is not a verified-employment claim. `Active` means the enrolled user performed at least one accepted exercise response **or** sent at least one user-authored Coach turn after onboarding during the exact printed 7- or 14-day window. Automated delivery, onboarding itself, and backend continuation do not count.

The company receives no names, Telegram identities, invitation-level status, teams/offices, completion or skip breakdown, Coach/exercise split, Coach text, weekly or finer dynamics, real-time dashboard, or drill-down. The buyer cannot lower the privacy threshold. This is aggregate product-usage reporting, not a wellbeing, stress, health, or productivity outcome.

User-facing and sales language must say **aggregate data without individual or small-group views**, not promise absolute anonymity.

Product rationale

The privacy gate is also a forcing function for serious adoption: useful company reporting becomes possible only after broad rollout and real employee use, rather than a five-person pilot selected from visible high performers. This aligns incentives without weakening the employee boundary. Privacy is the reason for the gate, not an artificial seat-sales tactic.

Sequencing

1. implement the user-linked event plus idempotent aggregate write before the first beta so trustworthy aggregate history begins on day one;
2. implement retention and account deletion against the personal layer;
3. do not build a live company dashboard or user-level export during MVP;
4. before the first post-pilot summary ships, implement the centralized gate, compute the three values from the accepted event contract, and re-review re-identification risk against the real cohort.

FD-10 — Content Library overhaul: a versioned beta protocol, not an instant-effect catalogue

Status: accepted (2026-08-06) Priority: P1 before beta content migration Area: Content Library / Plan Generation / Delivery / Telemetry

Decision

Replace the current eight-parent/variation-based library with nine independent, stable exercise records: six `switch` exercises and three `unload` exercises. Every exercise has its own stable ID, content version, exact user-facing copy, duration, modality, requirements, and review state. Runtime variations are removed; changing instructions creates a new `content_version`, not an untracked mutation or nested variation.

Love Yourself does not promise an instant physiological hit or a game-like energy refill. The intended product value is the repeated rhythm of interrupting workday autopilot with one short, bounded action. Individual exercises are selected for low execution cost, a clear endpoint, credible `switch` or `unload` intent, and enough variety to reduce habituation. Their exact effect remains a beta hypothesis and must not be presented as diagnosis, treatment, or guaranteed cognitive recovery.

The accepted active catalogue is:

```
switch: breathing, fist PMR, surface touch, distant gaze, one sound,
        cool-water facial immersion
unload: one-item brain dump, one thing that went well, first step tomorrow
```

The cool-water exercise belongs to the catalogue but is release-gated pending medical review. All exercises begin with `review_status=unreviewed`; only the cool-water exercise has `review_required=true`. Eligibility is:

```
is_active
AND (review_required = false OR review_status = approved)
```

Therefore the cool-water exercise cannot be selected or delivered until its required review is approved. The other exercises may enter a limited beta while their broader evidence/behavioral review remains visibly uncompleted.

Boundaries

- Internal metadata supports selection, accessibility reasoning, telemetry, and later learning. It is not user-facing and must not be turned into hidden psychological labels.
- No user trait, diagnosis, disability, or preference is inferred from a skip.
- Telegram response latency is not exercise duration, and a completed tap does not prove execution or benefit.
- Text instructions remain complete without media. GIFs/visuals are optional versioned delivery assets, not hidden instructions or evidence of effect.

- Exact notification composition, preview, media placement, buttons, and

channel fallbacks belong to a separate Delivery UX / Exercise Presentation audit. This decision defines the content payload supplied to that renderer.

Sequencing

1. record this decision and the `CONTENT-*` findings without modifying runtime code or the production JSON during the audit;
2. complete the separate Delivery UX / Exercise Presentation audit;
3. obtain the required medical review for cool-water immersion and seek an external behavioral/evidence review of the remaining catalogue before broad market claims;
4. implement the JSON/schema/loader/builder/renderer/telemetry migration as one reviewed change, with backward-data handling and tests;
5. calibrate content through versioned beta evidence, not silent edits or behavior-based personalization.

FD-11 — Telemetry measures product interaction, not inferred wellbeing

Status: accepted (2026-08-08) Priority: experiment-validity blocker before beta Area: Telemetry / Activation / Retention / Privacy

Decision

Telemetry exists to reduce uncertainty about two MVP questions:

1. can a company deployment bring an eligible employee through onboarding to a successfully delivered first exercise;
2. does the employee continue registering interaction with scheduled short actions across working days and into the next automatic cycle.

The scheduled prompt is part of the product mechanism. Continued response to scheduled delivery is valid evidence of product use; the user does not need to start exercises without a prompt for the product thesis to survive. Reactive or self-initiated use is a useful secondary signal only when the product actually offers a distinct self-start action and records it explicitly.

Telemetry may describe delivery, registered actions, timing, and continued use. It may not infer that an exercise was physically performed, that the user felt better, that stress or burnout decreased, or that company productivity improved. Such outcome claims require a separate validated evidence design.

Metric hierarchy

- **Deployment:** eligible population and attributed bot starts by deployment.
- **Activation:** onboarding completion, plan creation, successful first delivery, first explicit response, and first registered completion.
- **Rhythm:** registered completion by plan working day, completed-day coverage, explicit skip, silent expiry, and response in the next automatic cycle.
- **Content diagnostics:** exposure and response by stable exercise ID and content version; optional explicit feedback under FD-07.
- **Operations:** delivery, scheduler, tool, and continuation failures. These explain whether the mechanism worked and are not user-value metrics.

User-facing streaks, streak-triggered praise, and streak-based adaptation are not part of the MVP value model. Raw dated actions remain available so run lengths can be derived later for analysis without storing or optimizing a `streak score`. `hidden_compensation_score` and similar inferred-state scores are rejected, not merely disabled.

Interpretation rules

- A `completed` tap means **registered completion**, not verified execution or benefit.
- `skipped` means an explicit user choice; `expired` means no registered action before the real action deadline. Neither is a diagnosis or reason code.
- A Coach message is not automatically reactive exercise use. Product support, plan control, free feedback, and workday support must not be collapsed into one behavioral metric.
- Automatic creation or delivery of the next cycle is an operational event, not retention. Retention requires a later user response in that cycle.
- Percentages must always include numerator, denominator, raw `n`, cohort, and observation window. With a 10-15 person beta, thresholds are directional priors rather than statistically decisive kill/continue rules.
- Raw conversation text, brain-dump content, and free-text feedback are never analytics dimensions.

Sequencing

1. lock the event and metric contract in documentation;
2. implement stable event identity, linkage, and idempotency;
3. instrument the deployment and activation funnel together with deterministic onboarding;
4. repair task expiry and runtime-action telemetry;
5. implement the independent aggregate contribution required by FD-09;
6. validate events against operational state before interpreting beta results;
7. build reports only after the underlying events pass reconciliation tests.

FD-12 — Production data is covered by Railway backups

Status: accepted (2026-08-12) Priority: operational requirement Area: Data durability / Railway operations

Decision

Railway volume backups are now a required part of operating Love Yourself, not an optional safeguard added only before a risky migration.

The production PostgreSQL volume must have scheduled backups enabled. The MVP default is daily and weekly backups. A fresh manual backup must additionally be created before any migration, backfill, cleanup, or schema drop.

A backup is not treated as proven recovery until it has been restored to a separate scratch/staging instance and the expected schema and data can be read. PITR is not required for the MVP.

Operational implications

- purchase/enable Railway volume backups for the existing production volume;
- keep the backup schedule enabled while production data exists;
- periodically verify that a backup can be restored;
- never use the live production database as the first place to test a migration;

- include backup storage in the normal infrastructure cost of the product.

Database implementation order

When implementation reaches the database round:

```
scheduled backup + fresh manual backup
→ restore to scratch/staging
→ start and verify the restored database
→ read-only physical schema and row-count inspection
→ Alembic baseline
→ forward migrations
→ post-migration verification
```

The stopped production services do not need to be started merely to complete the audit. The restored copy is started when database implementation begins.

FD-13 — Love Yourself is the company; LY Workday is the product

Status: accepted (2026-08-13) Priority: P1 before beta-facing copy is finalized Area: Product identity / Delivery UX / User-facing documentation

Decision

The employee-facing product is named **LY Workday**. **Love Yourself** remains the company and umbrella brand.

The product name should be used consistently in the Telegram bot profile, entry flow, onboarding, exercise delivery, cycle summary, support surfaces, privacy copy, and future channel adapters. Internal Python package names and database identifiers do not need a cosmetic rename before beta.

This is a naming and product-clarity decision. It is not a claim that Telegram notifications can conceal the sender or that the product should disguise its purpose.

FD-14 — The notification preview is neutral; the exercise lives inside the chat

Status: accepted (2026-08-13) Priority: P1 before beta Area: Delivery UX / Notification preview / Telegram

Decision

The first visible line of a scheduled exercise notification uses one short, neutral workday-break label such as `Пайса` or `Перерва`. It does not expose the exercise title, an emotional label, a wellness claim, or internal scheduling metadata on the lock screen.

The actual exercise is revealed after the user opens the Telegram chat. The in-chat message contains the exercise title, duration, exact instructions, and the available action buttons. Curiosity may help the user cross the first tap, but the product is optimized for registered exercise action, not notification opens as an engagement metric.

Telegram cannot place the bot's custom inline action buttons directly in the operating-system notification. The accepted MVP path is therefore:

```
neutral notification preview
→ open Telegram chat
→ read/do the exercise
→ press the in-chat action
```

Exercise duration must not be inflated to keep the phone screen awake. Buttons and exercise state must remain recoverable after screen lock and chat reopen. The exact word and truncation behavior must be verified on real iOS and Android devices before beta.

FD-15 — The MVP cycle summary is a deterministic Telegram image

Status: accepted (2026-08-13) Priority: P1 before beta lifecycle completion Area: Completion / Delivery UX / Privacy

Decision

Every completed cycle produces a small, tangible result artifact. For MVP this artifact is a deterministic image rendered by application code and delivered in Telegram, not prose or artwork generated by an LLM and not a long-lived bearer-link HTML report.

The summary contains only controlled factual data:

- a visual completed/not-completed day grid;
- registered completion count such as 5 is 7 or 10 is 14;
- confirmation that the next same-format cycle is prepared;
- the date and local time of the next scheduled exercise.

It contains no score, tier, streak reward, psychological interpretation, diagnosis, productivity claim, adaptation count, dominant-slot inference, or choice CTA that contradicts FD-01 automatic continuation.

The image renderer must be deterministic and testable from structured data. A concise text caption/fallback must preserve the same facts when image delivery or image access fails.

FD-16 — Instructional GIFs are required for three technique-sensitive exercises

Status: accepted (2026-08-13) Priority: P1 before the affected exercise enters beta delivery Area: Content Library / Delivery UX / Media

Decision

Instructional GIFs are required for:

```
breathing
fist PMR
cool-water facial immersion
```

For these exercises the visual is part of execution-quality support, not a generic engagement decoration. It demonstrates the key movement or sequence that compact text can be read too quickly or performed incorrectly.

Each GIF is a versioned content asset linked to the exercise ID and `content_version`. Text instructions remain complete and authoritative; a media failure falls back to the same text exercise without losing the action. The cool-water GIF and exercise remain blocked by the medical review gate in FD-10. Other exercises do not require GIFs for the first beta.

FD-17 — Coach has no user-visible quota, with invisible abuse controls

Status: accepted (2026-08-15) Priority: beta operating decision Area: Coach / Telemetry / Operations

Decision

The beta does not impose a product allowance, daily message allocation, or user-facing Coach quota. The product does not display remaining messages or ask a normal user to ration Coach usage.

This product decision does not authorize unbounded concurrency or automated abuse. The beta uses the following invisible operational controls:

- at most one Coach turn runs at a time per user. When no turn is active, the

first accepted free-text message starts immediately with no debounce or quiet window;

- while that turn runs, a process-local FIFO accepts at most nine additional

free-text messages for the same user. They remain separate ordered turns and are processed one at a time after the active turn. A tenth pending message is refused rather than creating another waiter. Refusal does not cancel, pause, or reorder the active turn or the nine accepted pending turns;

- after Telegram-update deduplication, the abuse guard counts every attempted

user-authored free-text message, including messages rejected because the FIFO is full. It allows at most 30 attempts in a rolling minute and 300 in a rolling hour per user. Deterministic commands and callbacks do not count and must bypass the model. These deliberately generous thresholds target obvious automation rather than ordinary fragmented conversation, are deployment configuration rather than a product allowance, and must be reviewed against beta evidence;

- Coach usage and cost are separate counters: an accepted free-text message is

one Coach turn; a bounded tool-result continuation belongs to that initiating turn; every actual model request and token counts toward operational cost;

- a blocked request receives one neutral availability response such as

`Coach is temporarily unavailable. Please try again shortly. It must not be framed as the user exhausting a quota;`

- model calls have an explicit timeout, bounded retries, and a global

application circuit breaker for abnormal request, token, or cost volume;

- Coach runs in a dedicated OpenAI project with budget alerts. Configure a

provider-enforced hard spend limit only if that behavior is available and verified for the active account; otherwise the application circuit breaker remains the hard stop;

- the future Bounded Tool-Result Loop in COACH-09 is capped at one initial

model response, at most one runtime tool execution, and one final model response with tools disabled. It must never recurse.

The process-local lock is sufficient only while one bot process owns Coach traffic. A second replica requires distributed serialization or a single ingress owner before scaling; silent multi-process lock divergence is not an acceptable upgrade path.

The per-user lock covers the complete ordered turn: persist the accepted user message, read the authoritative conversation history, call the Coach, execute the bounded tool-result continuation when required, and persist the final result. Locking only the OpenAI request does not preserve conversation order. The lock/FIFO registry entry is removed through race-safe cleanup once that user has no active or pending turn, so a long-running process does not accumulate one permanent synchronization object per user.

The active turn and nine-slot FIFO are deliberately process-local for the one-process MVP. A Railway restart can discard the active result and messages accepted into the local queue but not yet processed. This is an accepted beta limitation, must be named in incident diagnosis, and must not be misclassified as user abandonment. Add durable Redis/PostgreSQL intake only if restart loss is observed, multiple replicas are introduced, or delivery guarantees become a product requirement.

Queue overflow uses a deterministic Telegram response, not the Coach model: `Я ще опрацьовую попередні повідомлення. Це повідомлення не додалося — надішли його трохи пізніше.` The notice does not enter the FIFO, does not consume the Coach rate limit, and does not interrupt accepted work. Emit it once per continuous full-queue episode rather than once for every rejected message, so an automated sender cannot make the bot generate reply spam.

Rate limiting is defense-in-depth, not the financial source of truth. Per-user thresholds protect conversation ordering and obvious automation; the global application circuit breaker is derived from measured cost per complete Coach turn and the founder's explicit loss budget. Provider project budgets and alerts are monitored, but the application must not assume that a configured provider budget is an immediate hard stop unless that behavior is verified for the active account.

The existing `DEFAULT_DAILY_LIMIT` setting does not represent accepted product behavior and must be removed rather than left as misleading dead configuration.

Unmetered product access does not mean unobserved use. Each model request must record privacy-safe operational usage facts such as model, input/output/total tokens, request outcome, and an estimated cost where available. These facts are for founder operations and beta learning, not user scoring or company-facing analytics. Rate-limit events, circuit-breaker state, and spend alerts are also founder-only operational facts and must not become company-facing behavior scores.

The decision may be revisited only after real beta usage and cost data exists.

FD-18 — Production access is roster-gated; individual beta runs on an isolated testnet

Status: accepted (2026-08-15) Priority: BLOCKER for testnet isolation and token enrollment before individual beta; BLOCKER for roster/SSO/reconciliation before the first company production deployment Area: Company Deployment / Privacy / Enrollment

Decision

Love Yourself has two isolated runtime environments built from the same release artifact and product behavior:

1. **Production** — first enrollment is possible only through a personal,

scoped, expiring, single-use Telegram handoff token backed by an active company entitlement. The token is issued only after the employee passes the company's roster and corporate-identity check.

2. **Testnet** — the individual developer beta uses separate infrastructure,

credentials, bot identity, data, telemetry, and token namespace. The founder issues test entitlements and the same one-time handoff tokens directly. Once a token is redeemed, onboarding, plans, exercises, Coach, lifecycle, and telemetry behave exactly as in production.

Testnet is not a reduced or divergent product branch. Environment isolation and the source that grants the initial entitlement are the only intended differences. Testnet users and events never enter production company counts, sealed aggregates, reports, or operational data.

Testnet participation does not create a right to migrate the account, Coach history, plans, or telemetry into production. The testnet bot may be retired after the beta under its disclosed retention/deletion terms. The intended exit for a successful beta participant is to become a champion and bring a company deployment; that production enrollment starts in the production bot without history migration. Record founder-known `testnet participant` -> `company introduction/deployment` conversion as a qualitative/commercial beta signal, not as behavioral company analytics.

Production has no public or organic first-enrollment path. A bare `/start`, an unknown token, or an expired/revoked entitlement cannot create a production account or plan. This does not block a returning enrolled user from opening the bot normally while their entitlement remains active.

Production enrollment

Before launch, the company supplies a current roster containing only the email required for access control. An employee opens the deployment-specific enrollment page and proves control of that roster-listed address. Google Workspace or Microsoft 365 OIDC is preferred; a one-time email verification flow supports roster-listed contractors, personal addresses, and companies on other mail providers. The backend verifies the authentication/challenge, normalized email, deployment, and active roster membership. A successful check creates or retrieves one revocable `access_entitlement`.

The enrollment page then mints one short-lived Telegram handoff token bound to that entitlement. The raw value is shown only for the immediate Telegram open; the backend stores only its hash and bounded issuance/redeem state. Redemption atomically consumes the token and creates or resumes one `deployment_enrollment` linked to the entitlement and Telegram user. The token is transport, not the license: it is never reusable and is irrelevant after redemption.

Returning users continue through their Telegram account and do not repeat SSO or enter another token while the entitlement remains valid. Renewing a company deployment keeps its active entitlements without user action. SSO is an initial identity proof, not a background membership-sync mechanism.

Roster reconciliation and revocation

Roster synchronization is batch-based; SCIM is not required. The cadence is a deployment contract field: annual may be accepted for a small client, while a large client normally supplies a quarterly roster or another explicitly agreed interval. Love Yourself owns requesting, validating, and applying the update; the user never reauthenticates because a reconciliation is due. New hires may be added between full reconciliations.

Each import is versioned, declares an explicit `import_mode`, and runs as `validate` -> `preview diff` -> `explicit confirm` -> `atomic apply`:

- `full_snapshot` represents the complete current roster. Only in this mode may

omission from the accepted file revoke an existing entitlement.

- `delta` contains explicit add/revoke operations only. Omission has no meaning

and every stable entitlement remains untouched.

The system never infers mode from file size, cadence, filename, or prior imports. A missing/unknown mode, empty or malformed input, an unexpectedly small full snapshot, or wrong-domain/wrong-deployment data fails closed and cannot mass-revoke access. Preview shows mode and exact add/keep/revoke counts before confirmation. The previous accepted roster version remains available for operational rollback.

Roster cadence does **not** define entitlement expiry. If no roster arrives, or an import fails validation, the current roster and every existing entitlement remain unchanged. A missed reconciliation raises founder/champion reminders and an overdue operational alert; it never triggers mass revocation.

When a valid full snapshot is explicitly accepted, an employee present remains active and an employee absent is revoked at the import's agreed effective time. An accepted delta changes only its explicit rows and cannot revoke an omitted employee. People who leave between reconciliations may therefore retain access until the next successful annual, quarterly, or otherwise contracted sync. The company may also request an earlier access-only revocation. Active company entitlements otherwise live through the deployment's commercial period; renewal extends the deployment centrally without SSO, another token, or user action. If the company does not renew, the deployment commercial end stops all company-sponsored access. Revocation stops delivery and Coach access but does not delete the personal account, historical plan, messages, or telemetry; retention and deletion remain governed by the Privacy round.

Individual paid production access is a known future path, not an almost-built pricing choice. It requires payment, verified email, a personal entitlement, and the same one-time Telegram handoff token without a company roster. It is not part of MVP and no current production user can enter through it.

Explicit MVP edge cases and market boundary

- A lost or replaced Telegram account is handled manually by support. After the

person re-verifies the roster-listed identity, an audited operator procedure may close the old enrollment and rebind the active entitlement. No self-serve account recovery is built for MVP.

- One Telegram account may have only one active deployment enrollment. Genuine

simultaneous entitlement through two client companies is unsupported until a real case exists; changing employers is handled as an explicit enrollment replacement rather than rewriting history.

- Contractors and employees using personal or non-Google/Microsoft addresses

are eligible when the company includes that address in the roster and the user completes email verification.

- Workforces whose members have no reachable individual email identity are

outside the current ICP. Production, retail, field, and similar access models are not silently promised by this decision.

Privacy and access boundary

Love Yourself necessarily retains a restricted mapping from corporate identity to entitlement and enrollment so access can be reconciled and revoked. It does not promise that this mapping is physically unknowable. The enforceable promise is narrower and operationally useful: corporate identity is used only for authentication, license administration, revocation, support, fraud prevention, and security; it is not used for employee scoring, behavioral profiling, Coach analysis, or company-facing individual reporting.

Identity/access records are separated from exercise, Coach, feedback, and behavioral telemetry in schema, service permissions, logs, and administrative views. HR may submit the roster and receive its validation result and total accepted count, but has no SSO/enrollment status or identity-to-activity view. Company reporting remains limited by [FD-09](#). This purpose limitation must appear consistently in the company agreement, data-protection terms, deployment-bound privacy notice, and employee-facing SSO screen.

Required controls:

- request only the minimum OIDC scopes and validate signature, issuer,

audience, expiry, nonce/state, tenant/domain, stable subject, and verified roster-listed email server-side;

- for non-OIDC addresses, use a short-lived, single-use email challenge with

rate limits, neutral responses, hashed challenge storage, and no indication whether an arbitrary address exists in a roster;

- store normalized corporate identity with restricted access; raw roster files

have bounded retention and are not copied into product telemetry;

- cryptographically random, entitlement-bound, expiring, single-use handoff

tokens; raw values are never logged, persisted, or exposed after issuance;

- atomic token redemption and idempotent enrollment creation;

- one neutral response for unknown, expired, revoked, already-spent, or

unauthorized tokens;

- `Cache-Control: no-store`, `Referrer-Policy: no-referrer`, CSRF protection,

and privacy-safe request logging on the enrollment page;

- deployment-wide pause, entitlement-level revocation, roster-import audit

history, and emergency token-key revocation without exposing behavior;

- fully separate production and testnet secrets, databases, Redis namespaces,

Telegram bots, OpenAI projects, token signing keys, URLs, backups, alerts, and aggregate sinks.

Do not add a speculative provisioning-provider interface, SCIM adapter, HR dashboard, or per-seat billing system before a real requirement exists. The durable data boundary is required now: `access_entitlement` remains separate from `deployment_enrollment`, and both remain separate from behavioral data.

FD-19 — Exercise on demand is an independent user-initiated delivery channel

Status: accepted (2026-08-19) Priority: P1 before the feature enters beta Area: Exercise on demand / Delivery / Coach / Telemetry

Decision

LY Workday adds one small reactive channel named `Вправа зараз`. It lets an authorized, onboarded user request one short `switch` exercise when the user already recognizes a need to interrupt the current work state. It is not a second plan, does not create another proactive schedule, and does not assume that scheduled delivery is the preferred or ultimately correct product model.

The beta must preserve scheduled and on-demand activity as separate observable channels so the product can learn whether users prefer scheduled delivery, reactive delivery, both, or neither. Movement from scheduled use toward on-demand use is a product-learning outcome, not a failure or a risk to be prevented. No combined adherence percentage may hide that channel choice.

Entry surfaces

The MVP has exactly two entry surfaces:

1. a deterministic Telegram command-menu item with the user-facing description

`Вправа зараз; /exercise` is the internal Telegram command identifier;

2. a Coach runtime tool invoked only when the user explicitly asks to receive

an exercise or to switch state now.

Both entry surfaces call the same authoritative on-demand application service. The Coach does not generate, rewrite, choose, or deliver its own exercise. A clear direct request is sufficient consent and does not require another confirmation question. Mentioning tiredness, stress, a difficult day, or a work problem without asking for an exercise must not trigger the tool.

There is no persistent reply-keyboard button and no Mini App or Web App in the MVP. Product copy refers to the action as `Вправа зараз`; it does not instruct the user to type the internal `/exercise` command.

Availability and isolation

The channel is available after onboarding while the user's access entitlement is valid:

- with an active scheduled sequence;
- while that sequence is paused;
- with no active sequence;
- on weekends, non-working days, and outside configured delivery times.

It is unavailable during incomplete onboarding and after entitlement revocation or account deletion. It creates no new user/FSM mode and never changes plan status, day progression, scheduled steps, scheduled jobs, continuation, pause/resume/cancel behavior, or scheduled completion metrics. The two channels share the Content Library and presentation infrastructure but have independent execution records and source identities.

Selection

The target source pool is the six `switch` records accepted by FD-10. Runtime selection includes only records that are active, pass their review gate, and have all beta-required assets configured. The expected launch pool is five exercises after the required GIF work; `cold_water_face` becomes the sixth only after both its medical-review gate and media prerequisite are satisfied.

For each request, selection is uniform random over the currently eligible pool after excluding the most recently successfully delivered **on-demand** exercise when it would otherwise repeat immediately. Every remaining record has equal probability. This is the only variety rule:

```
A -> A                forbidden
A -> B -> A            allowed
A -> B -> A -> B      allowed
```

There is no shuffle bag, anti-ping-pong rule, full-cycle uniqueness, behavior-based weighting, completion/skip contingency, inferred preference, or AI choice. FD-10's provisional `cooldown_days` does not impose a separate on-demand cooldown. The feature must not be enabled with fewer than two release-eligible exercises; an empty or undersized pool fails closed rather than delivering gated content or violating the immediate-repeat rule.

The immediate-repeat rule is scoped to consecutive on-demand selections. Scheduled history does not change the on-demand pool, and on-demand history does not reorder or replace a prepared scheduled sequence. The channels share the library, renderer, and telemetry infrastructure, not selection state.

Occurrence lifecycle and frequency

Each accepted request creates one independent PostgreSQL occurrence:

```
pending_delivery
-> delivered
-> completed | skipped | expired

pending_delivery -> delivery_failed
open occurrence -> canceled only for access/account closure
```

Only one `pending_delivery` or `delivered` on-demand occurrence may exist per user. A duplicate command, duplicate Coach tool execution, Telegram retry, or double tap returns/reuses the existing occurrence and cannot select or send a second exercise. After completion, skip, expiry, or terminal delivery failure, the user may request another exercise immediately. There is no product daily quota, frequency cap, or user-visible allowance in the MVP; broad invisible abuse protection remains an operational control, not a product rule.

The response window is 30 minutes from confirmed Telegram delivery. At expiry the system atomically records `expired`, removes `Виконано` / `Пропустити`, and edits the message with the exact neutral state:

```
Час виконання минув.
```

The callback handler checks authoritative database time as well as visible message state, so an in-flight, duplicated, or stale-client callback cannot complete an expired occurrence. A late callback receives the same factual copy; it does not silently fail and does not automatically create a new exercise.

Presentation and feedback

Delivery uses the shared canonical `ExercisePresentation` and renderer. The in-chat exercise contains only the exercise title, exact duration, exact steps, required versioned media when applicable, and the actions `Виконано` and `Пропустити`. It contains no sequence day, internal slot, scheduled-plan progress, plan rationale, inferred state, or adaptation message.

Text remains authoritative. A transient media-send failure falls back to the same complete text presentation and records the actual `delivery_variant`; a missing required asset at release configuration time makes the content ineligible instead. Optional FD-07 `better` / `same` / `worse` feedback appears only after registered completion and never changes selection or either delivery channel.

Telemetry and summary

On-demand uses the canonical telemetry infrastructure with an explicit `exercise_source=on_demand`, an `on_demand_request_id`, stable `exercise_id`, exact `content_version`, delivery variant, source-operation identity, and delivery/response timestamps. Scheduled references are null. Request, delivery, delivery failure, completion, skip, expiry, and optional feedback are separate validated events. Response latency means time from confirmed delivery to accepted callback, not exercise duration or effect.

Scheduled and on-demand denominators remain separate. In the deterministic cycle summary, the on-demand section appears only when at least one on-demand exercise was completed inside the summary window. The accepted language is:

За розкладом: 4 із 5 виконано
За власним запитом: 2 виконано
Усього виконано: 6

If no on-demand exercise was completed, the second line is absent and the summary does not display a zero or imply another obligation. Skipped, expired, and failed occurrences remain available to founder analytics but are not a user-facing voluntary-channel score. The combined line is a count only; the system never divides scheduled and on-demand activity by one shared denominator.

On-demand exercise and Coach activity remain personal product data under FD-09. A company receives no channel split, exercise history, completion, skip, expiry, feedback, or individual activity. A valid on-demand completed or skipped response may contribute once to FD-09's sealed aggregate definition of an active user; request, delivery, delivery failure, and expiry alone do not.

Explicitly outside MVP

- content expansion beyond the FD-10 catalogue;
- AI/contextual selection and behavior-inference personalization;
- persistent reply keyboard, Mini App, Web App, or separate exercise browser;
- user-facing quotas, therapeutic cooldowns, and gamified rewards;
- replacing scheduled delivery or declaring either channel the winner before

beta evidence;

- experiment-assignment infrastructure, pilot cohort machinery, or feature-

flag administration solely for this audit round;

- a standalone weekly pulse/report outside the accepted FD-15 cycle summary.

Scheduler Findings

SCH-01 — Re-engagement job active

Severity: BLOCKER Status: confirmed Area: Scheduler / Background jobs

Problem

Silent re-engagement job is active.

The scheduler still runs logic for:

```
silent_2_days  
silent_5_days
```

This sends messages outside the agreed scheduled plan flow.

Contract

Re-engagement is OFF for MVP.

Only two message sources are allowed:

1. scheduled delivery at agreed time
2. reactive response when user writes first

Why it matters

Unexpected proactive messages are a trust break, especially in B2B2C context where the user may already be cautious because HR invited them.

It can feel like surveillance or pressure.

Fix

Disable `silent_check` job.

Keep the function frozen if needed, but do not register it in scheduler for MVP.

Expected behavior:

```
silent re-engagement code may exist  
but no cron / interval job should trigger it
```

Founder decision needed

No.

C12 / re-engagement OFF is already accepted.

SCH-02 — SCHEDULE_ADJUSTMENT job active

Severity: P1 Status: confirmed Area: Scheduler / FSM legacy

Problem

Scheduler still checks for stuck `SCHEDULE_ADJUSTMENT` state.

This state is no longer part of the target MVP FSM.

Time changes should be handled through deterministic runtime tools, not a separate state tunnel.

Contract

`SCHEDULE_ADJUSTMENT` is removed / replaced by tools.

Expected flow:

```
user asks to change time  
→ tool collects/validates time  
→ tool updates future steps  
→ confirmation only after success
```

Why it matters

A dead state can create:

- unreachable transitions;
- stuck users;
- duplicate logic;
- mismatch between Coach, tools, and lifecycle state;
- confusing recovery behavior.

Fix

Disable `stuck_schedule_adj_check` job.

Keep the function temporarily if needed for migration, but it should not run in MVP.

Founder decision needed

No.

C_state is already accepted.

SCH-03 — Fast completion after last task action missing

Severity: P1 Status: confirmed Area: Lifecycle Completion, discovered during Scheduler audit

Problem

After the user presses `completed` or `skipped` on the last task of the plan, the completion report should arrive quickly.

Expected delay:

1-2 minutes after last `completed/skipped` action

Current suspected behavior:

completion scheduling is triggered mainly after `delivery` / `fallback`,
not necessarily after `task_complete` / `task_skip` callback

Contract

Active user path:

user presses `completed/skipped` on last task
→ system closes plan loop quickly
→ completion report arrives in 1-2 minutes
→ next period in the same 7- or 14-day format is prepared by default

Fallback path:

user does not press anything
→ task window closes / task expires
→ completion buttons are removed
→ next selected work day at the saved DAY time finalizes and sends the report
→ fixed UTC cron is recovery only

Why it matters

A user who actively completed or skipped the last task should not wait like an inactive user.

This is the moment where the behavior loop closes and continuation can happen.

If the report is delayed too long, the user may leave before seeing progress or the next prepared plan.

Fix

In `task_complete` and `task_skip` handler:

1. update step status;
2. check if this is the last step of the current plan;

3. if yes, trigger completion report with short delay;
4. continue into the same 7- or 14-day format according to FD-01.

Remove the 2-hour delivery-based completion trigger. Use expiry + the user's next selected work day and DAY time for the normal no-action path; keep the fixed UTC cron as recovery only.

Founder decision needed

No.

Fast completion after active last action is accepted as P1.

SCH-04 — daily_pulse disabled

Severity: OK Status: confirmed Area: Scheduler / Background jobs

Finding

`daily_pulse` is disabled.

Daily pulse job should remain off for MVP.

Contract

Daily pulse is frozen/off.

Pushes without clear user value increase mute/block risk.

Fix

No fix needed.

Keep disabled.

SCH-05 — pulse snapshots frozen

Severity: OK / FROZEN Status: confirmed Area: Scheduler / Plan snapshots

Finding

SHORT does not receive pulse snapshots.

This matches current MVP logic.

Contract

First 7-day plan does not need pulse snapshot.

Pulse snapshot for longer formats can remain frozen.

Fix

Do not touch before beta unless it breaks SHORT plan.

SCH-06 — adaptation call path must be checked

Severity: P1 check Status: pending Area: Scheduler / Adaptation / Frozen components

Problem

Scheduler does not clearly call adaptation, but comments imply adaptation may mutate step content after scheduling.

Contract

Adaptation layer is fully frozen on MVP.

Important:

```
frozen does not mean "do not improve"  
frozen means "must not trigger"
```

The system must not automatically change:

- plan sequence;
- exercise content;
- delivery time;
- user path.

Why it matters

Unexpected adaptation can look like surveillance or hidden personalization.

It can also break the promise that the sequence is prepared in advance and not based on hidden profiling.

Fix

Verify call path for:

- `adaptation_executor`
- `plan_adaptations`
- adaptation-related jobs
- any mutation of plan steps after generation

Expected result:

```
adaptation code may exist  
but no MVP path should trigger it
```

Founder decision needed

No.

C7 is already accepted: adaptation does not trigger at all on MVP.

Lifecycle Completion Findings

Lifecycle Completion Area — Audit Round 2026-07-03, revalidated 2026-07-16

Status: completed, revalidated against the current Product Map and MVP contract

Files inspected

- `app/telegram.py`
- `app/scheduler.py`
- `app/orchestrator.py`
- `app/plan_runtime/tools.py`
- `app/plan_finalization.py`
- `app/plan_completion/report.py`
- `app/plan_completion/cta.py`
- `app/plan_completion/metrics.py`
- `app/plan_completion/timeline.py`
- `app/plan_completion/tokens.py`
- `app/api.py`
- `app/templates/completion_report.html`
- completion-related tests in `tests/`

Summary

Task status writes and local-day plan scheduling work, but the completion lifecycle is not wired correctly. The two-hour trigger can send a report while the old plan is still active. Fast completion from task callbacks is missing, the no-action path ignores the user's DAY time and selected work days, automatic same-format continuation is not connected, and the report/CTA still implement the old duration/load/focus lifecycle. Report delivery is also not durable enough to guarantee the Product Map promise that every completed period receives a summary.

Accepted findings

LIF-01 — Completed/skipped status writes work

Severity: OK Status: confirmed

`handle_task_completed` and `handle_task_skipped` persist canonical step status, legacy compatibility fields, telemetry, and commit successfully.

LIF-02 — Last-task detection and fast completion trigger are missing

Severity: P1 Status: confirmed

The callbacks do not detect the final task and do not trigger completion within 1–2 minutes. They only update the step and send a task-level acknowledgement.

For a 14-day period, “last task” must mean that no other non-terminal step remains in the plan, not merely that the clicked exercise is last by time or order. Completion must not fire while the second exercise for that working day is still actionable.

LIF-03 — Completion report is sent before plan finalization

Severity: BLOCKER Status: confirmed

Current default path:

```
last task delivered
→ scheduler creates a +2 hour completion job
→ plan_end_date is still 23:59:59
→ _auto_complete_plan_if_needed exits without finalizing
→ _trigger_plan_completion still sends the report
```

The user can receive a completed-plan report while the plan and user are still `ACTIVE`. The existing follow-up helper cannot run correctly from that state because it requires a finished/idle state.

Minimal fix: remove the two-hour trigger and ensure the old plan is actually finalized before its completion report is sent.

LIF-04 — Automatic same-format continuation is not connected

Severity: P1 Status: confirmed

`create_followup_plan` already supports both canonical formats and reuses stored schedule data, but the completion flow never calls it. The target contract is now same-format continuation: `SHORT`→`SHORT` and `MEDIUM`→`MEDIUM`, with the same `work_days`, `DAY` time, and configured `EVENING` time for 14 days.

Minimal fix: after finalizing the explicit completed plan, derive its canonical format and atomically create the same-format successor before confirming that the next period is ready.

LIF-05 — Telegram completion CTA is dead

Severity: BLOCKER Status: confirmed

The Telegram completion report creates `start_plan`: callback data, but no matching Telegram callback handler exists. A user can press a visible button and get no deterministic action. Under FD-01 these legacy next-plan buttons should be removed rather than repaired as the continuation mechanism; format switching is a separate explicit flow.

LIF-06 — Completion report copy and CTA use the old lifecycle

Severity: P1 Status: confirmed

The current report asks the user to choose/repeat/change a plan and exposes legacy 21/30-day, load, focus, adaptation, persona, and internal slot logic. It also contains unsupported interpretations such as *Ранок виявився твоїм часом, Ти тримав ритм навіть коли було складно, and Це більше ніж більшість.*

Minimal fix: replace the legacy recommendation engine with a factual 7/14-day summary and confirmation of automatic same-format continuation. Preserve pause, cancel, time change, and explicit 7↔14 switching as separate controls.

LIF-07 — No-action completion belongs to expiry + cron

Severity: P1 Status: confirmed

If the user does not press completed/skipped, the task must remain actionable until its expiry window closes. Expiry removes the buttons. The normal path then finalizes and reports on the next selected work day at the saved `DAY` time; the fixed UTC cron is recovery only. A two-hour delivery timer must not produce the report while the task is still actionable.

LIF-08 — Completion report is not gated by completion rate

Severity: OK Status: confirmed

Completion rate changes report metrics/copy but does not prevent low-completion users from receiving closure.

LIF-09 — Completion may finalize the wrong plan

Severity: P1 Status: confirmed

`_trigger_plan_completion(user_id, plan_id)` does receive an explicit `plan_id`, but `_auto_complete_plan_if_needed` finalizes the latest active plan by `created_at(active_plans[0])`, not that `plan_id`, while the report is built for the passed `plan_id`. If they differ, it finalizes plan A and reports on plan B. Narrow trigger (multiple active plans) — the same code already logs "Multiple active plans found".

Minimal fix: finalize by explicit `plan_id`, not latest-by-`created_at`.

LIF-10 — Follow-up helper cannot be wired naively

Severity: P1 Status: confirmed

`create_followup_plan` raises unless `current_state` is in `_FOLLOWUP_STATES` (`IDLE_FINISHED`, `IDLE_DROPPED`, `IDLE_PLAN_ABORTED`). Per LIF-03 the state may still be `ACTIVE` at report time, so calling it directly from the current path raises. It also returns no `plan_id` or first-task date, and silently falls back to 14:00 when `DAY` time is missing. The target operation therefore needs explicit ordering and a completion-specific result: finalize old plan → create same-format successor → obtain its first delivery → send factual confirmation.

LIF-11 — No idempotency key on automatic next-plan creation

Severity: P1 Status: confirmed

The report has an idempotency guard (`plan_completion_sent` event). Next-plan creation has no equivalent. Once FD-01 is wired, concurrent action, cron, retry, or message-entry paths could each attempt same-format continuation, producing duplicate plans or `ActivePlanExistsError`.

Minimal fix: guard creation with a plan-scoped key or unique parent-plan relationship, e.g. `next_plan_created_for:{completed_plan_id}`.

LIF-12 — No-action completion does not use the user's DAY time or work_days

Severity: P1 Status: confirmed

`check_plan_completions` runs once at 10:30 UTC and immediately finalizes every expired active plan it finds. It does not schedule the user-facing report for the user's saved DAY time and does not move delivery to the next selected working day. A Friday completion can therefore surface on Saturday, and users in different timezones receive it at unrelated local times.

Minimal fix: treat the fixed UTC cron as detection/recovery only. Resolve the next selected work day and saved DAY time in the user's timezone for the normal no-action path.

LIF-13 — Completion report delivery is not durable or atomic

Severity: BLOCKER Status: confirmed

`_auto_complete_plan_if_needed` may create a fire-and-forget send task before the caller commits completion. If no event loop exists, it skips the send. After the plan is committed as completed, the cron no longer selects it because the cron queries only active plans. Metrics failure returns without a fallback report or retry. The `plan_completion_sent` check is also a non-atomic read-before-send guard, so concurrent paths can duplicate the message; success is recorded only after Telegram delivery. Normal Telegram send failures reschedule themselves every 30 minutes with no attempt cap, while the failure notice runs only when the submitted coroutine raises rather than when the send returns `None`.

Minimal fix: persist a plan-scoped completion workflow/outbox state in the same transaction as finalization, then process it idempotently through explicit `pending` → `sent` states. A report-generation failure must retain a retryable pending record rather than silently return.

LIF-14 — Web report misstates completed days for 14-day plans

Severity: P1 Status: confirmed

The web report calculates `completed_days = round(completion_rate * total_days)`, but a 14-day plan can contain two exercises per day. Exercise completion rate is not the same as completed-day count. Completing one of two exercises on every day displays 7 з 14 днів, even though the user interacted on all 14 days.

Minimal fix: derive day-level counts from the existing timeline or label the metric honestly as completed exercises instead of completed days.

LIF-15 — Completion tests are stale and currently broken

Severity: P1 Status: confirmed

Scheduler tests explicitly assert the deprecated +2-hour behavior. Report and CTA tests still use 21/30-day, load/focus/adaptation contracts and currently fail because their `CompletionMetrics` fixtures omit newer required fields. No test covers the target end-to-end invariants: explicit-plan finalization before report, fast completed/skipped path, local expiry path, same-format continuation, or plan-scoped idempotency.

Minimal fix: replace legacy assertions with 7-day and 14-day lifecycle tests derived from FD-01/FD-02 and the Product Map. Do not merely repair the old fixtures to keep testing obsolete behavior.

LIF-16 — Plan end boundary respects timezone and selected work days

Severity: OK Status: confirmed

`finalize_plan` maps logical plan days onto real dates from the user's selected `work_days`, schedules each step in the user's timezone, sets each `expires_at` to local 23:59:59, and derives `plan_end_date` from the last actually scheduled step rather than by adding calendar days. This calculation is correct. LIF-12 concerns the later report-delivery path, not this boundary.

LIF-17 — Expiry, keyboard removal, and ignored telemetry are not one lifecycle event

Severity: P1 Status: confirmed

`expire_overdue_steps` checks hourly at minute :05, marks steps `expired`, and then removes Telegram keyboards. Depending on timezone offset, a dead keyboard can remain visible for up to roughly one hour after local expiry even though `validate_step_action` rejects the click. Separately, `check_ignored_tasks` logs `task_ignored` at 08:00 UTC using a sliding 24-hour window rather than the step's local expiry. User-facing closure, canonical step state, and ignored telemetry can therefore happen at different times.

Minimal fix: make local expiry the single source event for terminal state and ignored telemetry, and remove the keyboard within the accepted post-midnight UX window. Final telemetry schema details belong to the later telemetry audit.

LIF-18 — Report sending is an ambient side effect of `_auto_complete_plan_if_needed`

Severity: P1 Status: confirmed

`_auto_complete_plan_if_needed` decides whether to send the completion report based on whether a running event loop happens to exist in the caller's context: it calls `asyncio.get_running_loop()` and, on success, fires `asyncio.create_task(send_plan_completion_message(...))`; on `RuntimeError` it logs and skips. Its three callers rely on opposite behavior. `handle_incoming_message` is `async`, so the loop exists and the hidden task sends the report. `_trigger_plan_completion` and `check_plan_completions` assume the function does not send and submit `send_plan_completion_message` themselves via `_submit_coroutine`.

Today this does not double-send for one incidental reason: `scheduler` is a `BackgroundScheduler`, so its jobs run in worker threads with no running loop and the hidden `create_task` is skipped. Completion correctness therefore depends on the `APScheduler` executor type, not on the completion logic. Switching to `AsyncIOScheduler` — an otherwise innocuous cleanup — gives both scheduler paths a running loop and produces two completion reports per completed plan, guarded only by the non-atomic `plan_completion_sent` read-before-send described in LIF-13.

The plan id also differs between the two senders: the hidden task uses `plan.id` from `active_plans[0]`, while the explicit senders pass the caller's `plan_id` (see LIF-09).

Minimal fix: remove sending from `_auto_complete_plan_if_needed` entirely. Finalization must only finalize and record durable outbox state (LIF-13); delivery must be an explicit, separately invoked step for every caller.

LIF-19 — The plan-to-plan seam is undefined and mechanically biased toward same-day collision

Severity: P1 (target-behavior design gap) Status: confirmed; founder-flagged 2026-07-22

Context: this is the seam where one plan ends and the FD-01 same-format successor begins — specifically the timing of the completion report relative to the new plan's first exercise. Automatic continuation is not wired yet (LIF-04: `create_followup_plan` is only reachable as a Coach tool, never from the completion path), so the seam does not exist in code today. But the mechanical pieces already in place bias the *target* seam toward exactly the "дибілізм" to avoid. Recorded now so the continuation is built with a defined seam, not an emergent one.

Mechanical facts (verified):

- `create_plan/finalize_plan` anchor a new plan at ``activation_time_utc`

= `now (service.py)`, and map logical day 1 via `next_active_date(anchor_date)`, which is `**inclusive**` (`>= from_date, active_days.py:72'`). So a successor created on a working day gets **day 1 = the same day**.

- Consequence by path:
- Active completion** (target fast path, LIF-02): user presses

completed on the last task at, say, 14:30 while DAY time is 14:00. Successor day 1 = today, `scheduled_for` = today 14:00 — **already in the past**. The first new exercise either fires immediately (user gets a fresh task seconds after the "план завершено" report) or is silently missed. Either is broken.

- No-action / ignored**: last task expires 23:59 local; the cron

finalizes on a later day; a successor created then still anchors day 1 to that same day, and the report timing is already wrong per LIF-12.

- The report is supposed to state the first task's date/time (FD-01 copy):

"Перший таск прийде [date] о [HH:MM]"), but `create_followup_plan` returns no `plan_id` and no first-step datetime (LIF-10), so the report cannot currently state a true, consistent next-task moment.

Required target seam (но-дубілізм contract):

- successor **day 1 = next active day strictly AFTER** the completion/

report day — never the same day. Offset the anchor by one day before day-mapping, or make the day-1 resolution exclusive of the completion date. This removes past-time day 1 and same-day double-exercise at once.

- exactly **one** seam message on the closing day: the report. It is

delivered **first**, states the exact next-task date and time, and the first new task then arrives at precisely that stated moment — never before the report, never in the same instant.

- the report reads the successor's real first-step datetime (fix LIF-10 so

the helper returns it), so stated and scheduled moments cannot diverge.

- both report and successor creation are guarded by plan-scoped

idempotency (LIF-11) so cron + fast-path + any Coach path cannot double-create the plan or double-send the report.

- no-action path delivers the report at the user's DAY time on the next

active day (LIF-12), consistent with the successor's first task.

Minimal framing: the seam is "close old → report with a concrete future moment → new plan's first task lands exactly at that moment on the next active day." One message now, one task later, never both at once.

Required MVP work

- remove the two-hour completion trigger;
- add last-task detection to completed/skipped callbacks;
- trigger the report quickly after active completion/skip;
- keep unanswered buttons active only until expiry, then remove them;
- use the user's timezone, next selected work day, and DAY time for the normal no-action path; keep the fixed UTC cron as a safety net only;
- finalize the old plan before sending its report;
- create the next same-format plan atomically (`SHORT→SHORT`, `MEDIUM→MEDIUM`);
- rewrite completion report copy and CTA for FD-01;
- remove the dead legacy `start_plan`: continuation buttons;
- finalize the completing plan by explicit `plan_id`, not `latest-by-created_at`;
- add an idempotency key for automatic continuation to prevent duplicate plans;
- make report delivery durable and retryable after plan finalization;
- remove the event-loop-dependent hidden send from `_auto_complete_plan_if_needed` so finalization never delivers;
- fix day-level report metrics for 14-day plans;
- unify local expiry, keyboard removal, and ignored telemetry timing;
- replace stale tests with completed, skipped, expiry, timezone/work_days, report ordering, same-format continuation, and idempotency coverage.

Verification on 2026-07-16

- `pytest -q tests/test_plan_completion_scheduler.py tests/test_plan_completion_metrics.py tests/test_orchestrator.py -k 'not trio' → 27 passed, 2 deselected;`
- those green scheduler tests still assert the deprecated +2-hour path and therefore do not validate the target contract;
- isolated report/CTA run with test environment variables → 18 failed, all from stale `CompletionMetrics` fixtures missing `engagement_rate`, `silent_miss_rate`, and `current_streak`;
- `tests/test_dashboard.py` could not be collected with the system interpreter because `fastapi` is unavailable there; the local `.venv` has `fastapi` but does not have `pytest`.

Scope boundary

This audit records required behavior and confirmed code gaps. Exact transaction ordering, function signatures, and implementation architecture belong to a separate implementation plan. Report-token lifetime and company/individual access boundaries are intentionally deferred to the dedicated privacy audit.

Onboarding Findings

Onboarding Area — Audit Round 2026-07-04

Status: current implementation inspected; onboarding itself remains TODO

Files inspected

- `app/telegram.py`
- `app/orchestrator.py`
- `app/workers/mock_workers.py`
- `app/db.py`
- `app/time_slots.py`
- `app/plan_runtime/tools.py`
- `app/plan_drafts/service.py`
- `app/plan_finalization.py`
- `app/ux/task_notification.py`
- `app/ux/plan_messages.py`
- `app/api.py`
- `app/fsm/states.py`
- `app/fsm/guards.py`

Summary

The real MVP onboarding flow is not implemented. `/start` creates a user in `ONBOARDING:START`, but subsequent messages are routed to a mock onboarding worker that does not collect data, change state, or create the first plan. Backend support for a SHORT first plan exists, but DAY time, `work_days`, time recommendation/range, and the first-task confirmation are not wired.

Findings

ONB-01 — Real onboarding flow is missing

Severity: BLOCKER Status: confirmed

Current code:

```
/start
→ create user in ONBOARDING:START
→ next user message calls mock_onboarding_agent
→ static test response
→ no data write
→ no state transition
→ no first plan
```

The user remains stuck in onboarding.

ONB-02 — User-selected time and `work_days` are not collected

Severity: BLOCKER Status: confirmed

`UserProfile` receives defaults (`DAY=14:00`, Monday–Friday), but no onboarding code asks for or stores the user's actual chosen time or `work_days`. Defaults are not evidence of user consent or preference.

Contract:

```
collect one concrete delivery time
→ collect work_days / weekend work preference
→ save both
```

Collect the one user-confirmed delivery time using the purpose-framed, workday-scoped onboarding question defined in `delivery_contract.md` (FD-06) — not a generic "when are you free" prompt.

ONB-03 — Internal slot tags leak into user-facing surfaces

Severity: P1 Status: confirmed

Examples:

- task delivery maps internal tags to Ранок / День / Вечір and can render День 3 · День;
- completion report logic can produce Ранок виявився твоїм часом;
- web report exposes a dominant slot label;
- legacy time-slot surfaces model all three slots instead of one user-selected HH:MM.

Contract:

Internal slot tags remain available for storage, generation, and scheduling only. Remove all user-facing references according to FD-03.

ONB-04 — Recommended and allowed onboarding time range is not defined consistently

Severity: P2 (consistency cleanup) Status: founder decision made (2026-07-22) — no hard range; reconcile legacy claims

Current code contains conflicting behavior:

- onboarding has no time recommendation or validation because it is a mock;
- generic HH:MM parsing accepts every valid clock time from 00:00 to 23:59;
- `change_day_time` therefore accepts a value such as 03:00;
- legacy schedule-adjustment UI offers DAY options from 12:00 to 17:59;
- schedule-adjustment text claims 06:00–23:59, while its actual slot inference rejects times before 12:00.

The current product contract requires one daytime time but does not define one authoritative numeric recommendation and hard allowed range.

Minimal fix:

Do not implement a range by inference. Use one shared validator for onboarding, change-time tools, API, and scheduling — but the founder decision below sets what that validator enforces.

Founder decision (2026-07-22):

No artificial hard range for MVP. Allowed DAY time = any valid clock time up to 23:59; the end-of-local-day expiry window already exists and handles late choices. Rationale: with 0 users this is a hypothesis, not a constraint to guess — give people freedom and let aggregate data reveal which hours actually work (good times survive, bad timing fades). Steering toward the productive window is done **softly**, by the purpose-framed onboarding question (FD-06 / `delivery_contract.md`), not by a hard numeric gate — the two would be redundant, and the soft version preserves freedom while still keeping most users out of the leisure-time slot. Remaining cleanup is consistency only: remove the contradictory legacy range claims (12:00–17:59 UI vs 06:00–23:59 text vs <12:00 rejection) so one shared validator agrees with "any time, end-of-day window."

ONB-05 — Canonical first-plan service already avoids duration and evening collection

Severity: OK Status: confirmed

`create_plan()` enforces that the first plan is `SHORT`; deterministic onboarding can pass the stored DAY time and `evening_time=None`. The first plan does not need a duration choice or evening time. The dead `create_first_plan()` runtime wrapper is not required for this invariant (ONB-07 / RT-05).

ONB-06 — First-task date/time confirmation is missing

Severity: P1 Status: confirmed

There is no post-onboarding confirmation containing the real first delivery date and HH:MM, such as:

```
Перший таск прийде [сьогодні/завтра] о [HH:MM].
```

The existing generic first-plan reply does not expose the calculated activation anchor.

ONB-07 — deterministic onboarding must bypass the dead first-plan wrapper

Severity: P1 Status: confirmed

`create_first_plan()` raises unless `user.current_state == "IDLE_ONBOARDED"` (`tools.py:77`), but it has no caller and is not a Coach tool. The target lifecycle removes `IDLE_ONBOARDED` and routes deterministic onboarding directly through canonical plan creation.

Minimal fix: delete the wrapper rather than patching its state guard. The real onboarding completion transaction calls `create_plan()` directly, commits the first SHORT plan and onboarding completion idempotently, and exposes the derived active mode only after success.

ONB-08 — First-task confirmation must use the finalized step, not the anchor

Severity: P1 Status: confirmed

The activation anchor computes today/tomorrow, but `finalize_plan` then shifts the first task to the next work day via `next_active_date`. A confirmation built from the anchor can say "tomorrow" while the real first `AIPlanStep.scheduled_for` is a later work day.

Minimal fix: build the confirmation copy from the actual first step's `scheduled_for` after finalization. Complements ONB-06 (confirmation missing) by fixing the data source.

Backlog items to add

- implement onboarding for exactly one concrete HH:MM time and work_days;
- do not ask for duration, MORNING, or evening time;
- remove user-facing MORNING / DAY / EVENING terminology;
- reconcile the contradictory legacy time-range claims (12:00–17:59 UI vs

06:00–23:59 text vs <12:00 rejection) into **one shared validator** that enforces the ONB-04 founder decision: any valid time up to 23:59, no artificial hard range (soft steer via the onboarding question only);

- create the first SHORT plan automatically after onboarding data is saved;
- confirm the actual first-task date and time;
- build the confirmation from the finalized first step's `scheduled_for`, not the activation anchor (ONB-08);
- delete the dead `create_first_plan` wrapper and let deterministic onboarding

call `create_plan()` directly (ONB-07 / RT-05);

- implement onboarding as the FD-04 deterministic mechanism-sale script (recognition → mechanism → setup → promise + control + privacy), with an approved privacy text and approved FAQ;
- keep the core script short and review funnel drop-off before and at time selection;
- verify the approved privacy statement against actual data access, reporting, logs, and HR/company surfaces before beta;
- handle arbitrary onboarding text deterministically without losing the current step, with approved FAQ matching and the separate crisis guard;
- test back/edit/restart/stop paths and ensure retries cannot create duplicate plans;
- instrument the full activation chain: onboarding start, setup saved, plan created, first task delivered at the confirmed moment, and first response;
- add onboarding tests for state transition, persistence, forbidden questions, time validation (any time up to 23:59 accepted; one shared validator), first-plan creation, and confirmation copy.

Coach / Orchestrator Integration Findings

Coach / Orchestrator Integration Area — Audit Round 2026-07-16

Status: Coach prompt rewrite complete, findings below discovered during that review (moved here from Coach-session working notes so the broader MVP audit has one source of truth).

Context

All 7 sections of `COACH_SYSTEM_PROMPT` were rewritten and an integration pass synced `COACH_TOOLS`, the context builder (`_context_message/_compose_messages`), `orchestrator.py`, and the Product Map to a single target contract. PR #246 (`refactor/t5-8c-prompt-refactor` → `main`) is open, not merged. The findings below are runtime/tool-level gaps discovered while writing and verifying that prompt against the actual code — not prompt-text issues.

Files inspected

- `app/workers/coach_agent.py`
- `app/orchestrator.py`
- `app/plan_runtime/tools.py`
- `app/plan_pause.py`
- `app/scheduler.py`

Findings

COACH-01 — `record_evening_time` has no runtime validation

Severity: P1 Status: confirmed

`record_evening_time(hhmm)` does not check `user.current_state`, does not check `evening_slot_collected == False` before writing, and `_validate_hhmm()` only checks the `NN:NN` shape — it accepts `99:99`. The "first-time collection only" constraint currently exists only as prompt text (Section 7), not as a runtime guard.

Minimal fix: add state check + `evening_slot_collected` check + a real HH:MM range validator to `record_evening_time()`.

Founder decision needed: No — enforcement of an already-agreed contract.

COACH-02 — `get_plan_status` paused-state display bug — RESOLVED

Severity: was P1 Status: RESOLVED (commit `83370f8`, "fix(T5.8C): report paused delivery in plan status")

Was: `orchestrator.py::_execute_plan_tool`, in the `get_plan_status` branch, checked only `result.get("plan_active")` (True/False) and always rendered "□ Стан: активный план" when true — never read `result["state"]` to distinguish `ACTIVE` from `ACTIVE_PAUSED`.

Fix shipped: new `_format_plan_status()` helper branches on `result.get("state") == "ACTIVE_PAUSED"` and renders "доставка вправ призупинена" vs "доставка вправ активна". Covered by new tests in `tests/test_orchestrator.py`.

COACH-03 — `create_followup_plan` silently defaults a missing `plan_type`

Severity: P1 Status: confirmed

`_build_tool_registry()`'s lambda for `create_followup_plan` does `args.get("plan_type", "SHORT")` — if the model or a parser returns empty arguments, the system silently creates a `SHORT` plan without explicit user consent for that specific choice, even though `plan_type` is a required field in the `COACH_TOOLS` schema.

Minimal fix: fail closed (raise / reject) instead of defaulting when `plan_type` is absent.

Founder decision needed: No.

COACH-04 — `pause/resume` does not reschedule deliveries missed during pause

Severity: P1 Status: confirmed (code-verified)

`plan_pause.py::pause_plan()` docstring states: "Does NOT rewrite or reschedule any plan steps." `resume_plan()` only flips `is_paused/ current_state.scheduler.py:113` gates delivery on `current_state == "ACTIVE"` — any step whose `scheduled_for` falls inside the pause window is silently skipped when its job fires, and nothing reschedules it after resume. It is lost, not deferred.

Contract: Conceptual Map §10 says resume continues "з того самого місця" (day 3→4) — the current implementation does not fully honor this for steps that were due during the pause window.

Minimal fix: on pause, clear the completion date and stop future jobs without closing the remaining steps. On resume, re-anchor those remaining steps from the next selected working day, reschedule them, and derive the new completion date from the actual last step. An exercise already delivered before pause remains actionable through its existing expiry (DEL-04 / FSM-06).

Founder decision needed: No — already implied by the existing Conceptual Map contract (§10).

COACH-05 — change_evening_time: Coach cannot reliably disambiguate day vs evening

Severity: P2 Status: confirmed

Coach's runtime context does not currently expose whether the user is on a 7-day (SHORT) or 14-day (MEDIUM) sequence. When the user says "change my time" without specifying day/evening, Coach has no reliable signal to pick between `change_day_time` and `change_evening_time`.

Minimal fix: expose current plan format (SHORT/MEDIUM) in Coach's runtime context.

Founder decision needed: No.

COACH-06 — State-filtered tool registration — RESOLVED

Severity: was P1 (architecture improvement) Status: RESOLVED 2026-07-16

Previously: all 8 tools were sent to the model on every Coach call regardless of `current_state`; the FSM×tool table in the prompt was the only enforcement (text, not code).

Resolution: `_coach_tools_for_state()` + `_TOOL_NAMES_BY_STATE` implemented in `coach_agent.py`. `coach_agent()` now includes `tools=` in the API call only when the current state allows at least one tool. Verified with a parametrized test covering all live and dead states.

COACH-07 — dead first-plan wrapper (cross-reference, not a new finding)

Severity: P1 Status: confirmed — same issue as ONB-07 above, tracked there

`create_first_plan()` in `plan_runtime/tools.py` still hard-requires `IDLE_ONBOARDED`, but Coach no longer exposes or calls it and production has no other caller. Delete the wrapper under RT-05. Deterministic first-plan creation belongs to the onboarding transaction tracked under **ONB-07**; no separate Coach action is needed.

COACH-08 — P0: delivered exercise context missing from Coach runtime

Severity: P0 (pre-production) Status: confirmed

Coach's prompt (Exercise Explanation Boundary) is written to explain the current exercise only from instructions available in `current_exercise_context` — but the runtime does not populate or send this field to Coach at all. `short_term_history` doesn't carry it either (scheduled exercise messages aren't appended to Redis session history). With the current runtime, Coach has no way to answer "how do I do this exercise?" without inventing steps, which violates its own anti-hallucination rule.

Minimal fix: add a structured `current_exercise_context` to the Coach payload rather than relying on `short_term_history`, built from the latest relevant delivered `AIPlanStep` and its trusted `ContentLibrary.content_payload.display data`:

```
{
  "title": "Дихання",
  "steps": [
    "Вдих — повільно, на 4 рахунки.",
    "Затримай подих на 7.",
    "Видих — повільно, на 8.",
    "Повтори 4 рази."
  ],
  "duration_label": "30-60 сек"
}
```

`delivered_today` must be evaluated in the user's local timezone. In the 14-day format (two exercises possible on the same working day), use the most recently delivered exercise; if a later workflow needs Coach to distinguish both explicitly, expand to `delivered_exercises_today` — do not silently guess which one the user means. If nothing was delivered today: `{"current_exercise_context": null}`, not invented data. Coach may clarify/repeat only the supplied `title/steps/duration` — no variations, no added steps.

Tests needed: latest delivered exercise included for `ACTIVE`; `display.steps/duration_label` preserved exactly; future/pending/ skipped/canceled/unrelated exercises not exposed as current; missing content produces `null`, not invented data; context reaches `_compose_messages()` before the user message.

Related P0 — v5 delivery rendering (separate bug, same root data): the v5 content library stores instructions under `content_payload.display.title / .steps / .duration_label`, but `plan_finalization._build_step_title()` reads `root content_payload.title`, `plan_finalization._build_step_description()` reads `root description/ text/instructions`, and `format_task_notification()` reads `root instructions` and `root duration` fields. The content loader preserves the nested `display` object without flattening it — the current delivery path may fail to render `display.steps` in the actual Telegram exercise notification the user receives. Fix this first: if the user doesn't receive `display.title/display.steps/display.duration_label`, the primary daily product loop is broken independently of Coach. The user and Coach must end up reading the same trusted exercise data.

Founder decision needed: No — already implied by the existing exercise-explanation contract.

COACH-09 — Architecture decision: Bounded Tool-Result Loop

Severity: P1 (before first external MVP user) Status: accepted as target architecture, not implemented

Current: Coach is a one-shot command dispatcher — it returns a tool call, the orchestrator executes it, and a hardcoded Ukrainian template is sent back. Coach never sees the result and cannot phrase a context-aware, tone-consistent reply. Concrete evidence that this class of drift is real, not hypothetical: COACH-02 above — a hardcoded template rendered "active" for a paused sequence until it was fixed directly in the template. This general risk (a canned string silently diverging from runtime truth) remains even after that specific instance is resolved.

Target: bounded two-step loop. Coach makes at most one tool call; runtime executes it; a structured result (`{"status": "success"/"error", "facts": {...}}`) is returned to Coach; tools are disabled on the second call; Coach writes one natural reply from the facts; deterministic templates remain only as a fallback if the second LLM call fails.

Estimate: 2-4h minimal; ~1 day with full test coverage of all 8 tool results.

Founder decision needed: implicitly yes (architecture investment before first external user) — already agreed as P1, not P0.

COACH-10 — Product question escalation flow (backlog)

Severity: backlog (not blocking) Status: not implemented; current Coach behavior is safe in the meantime

If the Product Map and runtime context don't contain a factual answer, Coach currently says it doesn't have that detail and stops there — there is no actual escalation channel (support contact, DB queue, or an `escalate_product_question` tool) for these questions to go anywhere.

Founder decision needed: Yes — where escalated questions go, who answers them, automatic vs. user-confirmed escalation, response-time expectation shown to the user, whether unresolved questions get stored for future Product Map updates.

Possible implementation options: a deterministic support contact/button; a support queue persisted in the database; an `escalate_product_question` runtime tool available to Coach; an admin notification with a later human reply flow.

Required Coach behavior until implemented (already in the prompt): say it doesn't have the detail; never guess/approximate/invent an answer; never claim a question was sent/escalated/reported unless a real escalation action actually completed and returned success.

Future tool contract, if implemented: minimum input `{"question": "...", "relevant_context": "..."}; result` should distinguish accepted-for-human-review / already-answered-by-existing-source / failed-to-submit.

Status: backlog. Not required for current prompt work, but must be resolved before Coach is instructed to offer product escalation as an available user action.

COACH-11 — `switch_plan_format` tool needed (target architecture, not implemented)

Severity: P1 (confirmed founder decision, target architecture) Status: confirmed as target 2026-07-16, not implemented

Switching between 7-day and 14-day format from an ACTIVE/ACTIVE_PAUSED sequence currently requires two separate steps (`cancel_plan`, then `create_followup_plan` from `IDLE_PLAN_ABORTED`) — real friction for a "let's try the other format" request.

Decision: build a dedicated atomic `switch_plan_format(plan_type)` tool — available from ACTIVE/ACTIVE_PAUSED, single user confirmation, atomically ends the old sequence and creates the new one (collecting evening time first if switching to 14-day), old period isn't cancelled until the new one is guaranteed creatable.

Note: the Product Map (`conceptual_map.md/conceptual_map_en.md`) already describes this as "one user-confirmed action" — this is intentional target-contract documentation, not a shipped capability. Until the tool exists, Coach can only offer the two-step path.

COACH-12 — Lifecycle friction: format switch immediately after FD-01 auto-continuation

Severity: P2 (edge case, not blocking) Status: identified, not resolved

Once FD-01 auto-continuation is wired, if the system already auto-created a new 7-day ACTIVE sequence right after completion and the user then asks to switch to 14 days, the just-created sequence must be cancelled first — same friction as COACH-11, worth resolving together with `switch_plan_format`.

Required work before beta

- fix remaining COACH-01, COACH-03, COACH-04, COACH-05 (runtime validation/scheduling gaps — COACH-02 already resolved);

- implement Bounded Tool-Result Loop (COACH-09) before first external user;

- fix P0 delivered exercise context (COACH-08) before ACTIVE-state

exercise explanation ships to real users;

- decide and implement product question escalation (COACH-10) before

Coach is instructed to offer escalation as an available action;

- implement `switch_plan_format` (COACH-11/COACH-12) as part of closing

FD-01 lifecycle work;

- merge PR #246 after review.

Plan Generation Findings

Plan Generation Area — Audit Round 2026-07-18

Status: completed

Files inspected

- `app/plan_drafts/plan_builder_v5.py`
- `app/plan_drafts/service.py`
- `app/plan_drafts/plan_types.py`
- `resource/assets/plan/plan_context_template.yaml`
- `resource/assets/content_library/tasks/burnout_combined_content_library.json`
- `app/telegram.py` (consequence of completed/skipped, for the operant frame)

Method

Findings below are not read-only inference. The builder was executed directly against the real library and recipe for 500 synthetic users and for repeated regenerations of the same user, and the resulting series were measured. Simulation commands and counts are reproducible from `get_default_builder()`.

Summary

The builder is clean, deterministic, and correctly enforces its stated invariants. The problem is not correctness — it is that the generated series is **frozen per user and blind to behavior**. The same user receives a byte-identical exercise sequence on every plan, forever, and nothing in generation reacts to whether the user did anything. Combined with an 8-exercise library, this makes the product's core artifact repetitive by construction.

Findings

PLAN-01 — Every regenerated plan is byte-identical for the same user

Severity: BLOCKER Status: confirmed by execution; fix blessed as the variety mechanism (FD-05, 2026-07-22)

Decision note (2026-07-22). The fix below is not just bug-removal — it is the variety mechanism of FD-05 (level "variety by randomness"). Unfreezing the seed turns the current single frozen combination into a fresh random draw from ~20,480 valid sequences per plan. Full random is deliberately blessed. Today the system does not have "one chosen combination" — it has one *accidental* frozen draw nobody selected, which is the worst of both worlds.

`_weighted_choice` seeds `random.Random(seed_key)` with `f"{user_id}:{day}:{slot}"`. `day` is the plan-local day number, which restarts at 1 for every new plan, and `last_used` starts empty on every build. The seed therefore contains nothing that distinguishes plan N from plan N+1.

Measured: building SHORT three times for the same `user_id` returns the identical sequence every time.

```
user 101, SHORT, runs 1/2/3 — all three:
somatic_001, somatic_006, somatic_005, somatic_006,
somatic_005, somatic_001, somatic_006
```

Under FD-01 (automatic same-format continuation) this is not a cosmetic issue: the user completes 7 days, receives "наступні 7 днів готові", and gets exactly the same seven exercises in exactly the same order. Every period. Indefinitely.

Minimal fix: include a stable **per-cycle discriminator** in the seed — `plan_instance_id` or a cycle number, **not** a creation timestamp. The contract is: **deterministic within one cycle** (re-generating the same cycle yields the same sequence) but **different across cycles**. A timestamp breaks the first half. Also carry `last_used` across the plan boundary so the first days of a new cycle do not repeat the final days of the previous one.

PLAN-02 — Switching SHORT→MEDIUM replays the first 7 days

Severity: P1 Status: confirmed by execution

Because the seed depends only on `user_id:day:slot`, the DAY slots of a MEDIUM plan reproduce the SHORT plan exactly for days 1–7; only days 8–14 are new. Measured for user 101 and user 202: `MEDIUM.DAY[0:7] == SHORT.DAY` is `True`.

A user who explicitly upgrades to the longer format is shown a full week of repeats before reaching anything unseen — the opposite of what an explicit upgrade should feel like. Same fix as PLAN-01.

PLAN-03 — Small library: repetition within a week is accepted, not fixed by growth

Severity: was BLOCKER → **decision recorded** (2026-07-22): library not expanded for MVP

Decision note (2026-07-22). Original framing ("library too small, grow it") is rejected by the founder. The 5 `switch` exercises are the deliberately approved core — the previous 65 were mostly fillers and were cut. The product is state-change + rest, not exercise count. Adding exercises is itself an untested hypothesis and reintroduces fillers, so the library is **not expanded for MVP**. Consequence, stated honestly so it is not a hidden assumption: with 5 exercises over 7 day-slots, within-week repetition is mathematically forced (pigeonhole — at least two exercises must repeat). This is accepted. Variety comes from **cross-plan** sequence randomness (PLAN-01 fix), not from library size: each new period is a different random draw of order/frequency over the same 5 exercises. Day-to-day unpredictability ("never know tomorrow's exercise") holds; the content *pool* stays at 5. Whether a 5-exercise pool avoids week-3 habituation is a real open question answerable only in beta — recorded, not pre-answered.

The library holds 8 active exercises: 5 `switch`, 3 `unload`.

Measured across 500 users, SHORT (7 days, DAY slot only):

distinct exercises in a 7-day plan	users
5	177
4	268
3	54
2	1

max repeats of a single exercise	users
2×	261
3×	226
4×	13

So the median user sees 4 distinct exercises across a 7-day period, and nearly half see one exercise three or more times. For MEDIUM the EVENING slot draws from 3 `unload` exercises across 14 evenings — measured distinct count is 3, i.e. a forced rotation of the same three items.

Measurement note: with 5 `switch` exercises over 7 day-slots, some within-week repetition is mathematically forced (pigeonhole). The seeding fix (PLAN-01) does not add exercises the library lacks — it only varies order/frequency **across** cycles. Per the decision note above, this is accepted; library size is not increased for MVP. (Earlier reference points about ≥ 7 / ≥ 14 exercises assumed the rejected "grow the library" path and no longer apply.)

PLAN-04 — `cooldown_days: 1` permits every-other-day repetition

Severity: not a bug — tuning knob, accepted as beta hypothesis (2026-07-22) Status: confirmed

`_is_in_cooldown` returns `(current_day - last_used[id]) <= cooldown_days`. With `cooldown_days: 1` — the value on all 8 library items — an exercise used on day N is blocked on day N+1 and available again on day N+2.

Combined with PLAN-03 this is what produces the 3× and 4× repeat counts above. The cooldown mechanism works as written; the value is a **content pacing dial**, not a defect. Given the accepted 5-exercise library (PLAN-03), raising it too far starves the pool (`NoCandidatesError`). Decision: keep 1 as the beta hypothesis; revisit only if usage shows the repetition texture actually hurts.

Note the mechanism is shared across slots: `last_used` is one dict for both DAY and EVENING, which correctly prevents the same exercise appearing twice on the same day.

PLAN-05 — Generation is blind to user behavior (operant frame)

Severity: OK by design — resolved by FD-05 (2026-07-22) Status: confirmed; product decision recorded

Requested analysis: does plan generation react to user behavior? **No, verified structurally.** `plan_builder_v5.py` contains no reference to step status, completion, skip, streak, or any history. Its only inputs are `plan_type`, `user_id`, the two time strings, the recipe, and the library. `ADAPTATIONS_ENABLED` is named in the invariant header but does not exist as a config flag anywhere in `app/` (only in comments) — it is deletion residue, not a disabled feedback path.

Locus note: reinforcement (in the operant sense) lives in the response→consequence path, not in sequence generation, so the operant question is really about the callback layer, not the builder. This finding is filed under Plan Generation only because the question was asked about generation. Two facts about that callback layer, recorded for accuracy: `completed` and `skipped` record **distinct** states and telemetry (they are not identical — they only lead to the same future next exercise); and the completion report **arrives** for everyone regardless of completion rate (LIF-08) but its **content** reflects actual completion (14/14 and 1/14 get different text, not identical text).

Resolution: this is not a defect. FD-05 records behavioral contingency as intentionally excluded from MVP. Residual work is cleanup (remove adaptation residue, disable legacy engagement CTAs), tracked under FD-05 code implications — not a new adaptation system.

PLAN-06 — `weight` is effectively decorative

Severity: P2 Status: confirmed; founder decision — drop for now (2026-07-22, FD-05)

Active weights span 1.2–1.5 (1.5×1 , 1.4×2 , 1.3×3 , 1.2×2). Across a candidate pool of 2–5 items this is close to uniform selection; the field implies editorial control over exercise frequency that it does not meaningfully exert.

Founder decision: **drop the field** for MVP; selection is uniform (within cooldown). Weighting exercise frequency by difficulty or editorial prior is a post-data hypothesis, not an MVP dial — it should be driven by behavioral data later, not guessed now.

PLAN-07 — `source_exercises` records the whole library, not the plan

Severity: P2 Status: confirmed

`source_exercises=[e.id for e in active]` stores every active exercise, not the ones actually scheduled. It is persisted into `draft_data` (`service.py:127`) and typed as "Which content library exercises were used" (`plan_types.py:193`). Any later provenance or content-performance analysis reading this field will be wrong.

Minimal fix: store the distinct `exercise_id` values actually placed in `steps`.

PLAN-08 — `Library` variations is unused

Severity: P2 Status: confirmed; parked as deferred new feature (2026-07-22)

Every inventory item carries a `variations` array. **Correction (2026-07-22)**: it is no longer empty everywhere — 3 of 8 items now carry variations (`somatic_006_v2=3`, `somatic_001_combined=2`, `somatic_003_v2=5`), the other 5 are still empty. Regardless of fill state, `ExerciseV5.from_library_item` does not read the field and no code in `app/` consumes it — it remains **authored but unwired**.

Decision note (2026-07-22). Not dead schema — this is the founder's own **unfinished** variations feature (e.g. a longer second exercise on the same theme for reactively-engaged users). It is a new feature, still at the "not yet thought through" stage, and the audit rule is "no new features in MVP." So it is **parked**: leave the field in place but unwired, no authoring guidance yet. Revisit only when the variations feature is actually designed. It is one candidate mechanism for level-2 content variety later (FD-05), but not MVP.

PLAN-09 — Builder invariants that hold

Severity: OK Status: confirmed

Verified correct and worth protecting with tests:

- first plan is forced to `SHORT` (`service.py:51`), independent of caller;
- `MEDIUM` refuses to fall back to the default `21:00 EVENING` value unless

`evening_slot_collected` is `True` — the silent-default trap is explicitly closed (`service.py:65-78`);

- `mechanic` is snapshotted onto the step at build time and never

recomputed (invariant 6);

- an empty candidate pool raises `NoCandidatesError` rather than silently

producing a short plan;

- no `Focus` / `Load` / `StepType` / `SlotType` / `DifficultyLevel` anywhere in the

builder — the v5 cleanup is genuinely complete on this path.

Required MVP work

Mandatory:

- add a stable per-cycle discriminator to the selection seed and carry

cooldown state across cycle boundaries (PLAN-01, PLAN-02);

- fix `source_exercises` to record what was actually scheduled (PLAN-07);
- drop the `weight` field (PLAN-06);
- remove adaptation residue / `ADAPTATIONS_ENABLED` mentions and disable

legacy engagement CTAs (FD-05, PLAN-05);

- add regression tests: same cycle re-generates identically; the next

cycle differs; SHORT→MEDIUM does not replay days 1–7.

Consciously accepted (no work — recorded decisions, FD-05 / PLAN-03):

- five DAY exercises; within-week repetition; no behavioral adaptation;

current `cooldown_days: 1` kept as a beta hypothesis (PLAN-04 is a tuning knob, not a bug — revisit only if beta shows repetition hurts).

Deferred (not MVP):

- `variations` (PLAN-08); exercise levels/unlocks; personalization;

automatic future-cycle changes from feedback.

The optional post-`completed` feedback tap (FD-05) is a small standalone experiment owned by the Delivery / Telemetry area, not Plan Generation.

Scope boundary

This round covers exercise selection and series construction only. Rendering of `display.*` belongs to the Delivery renderer round (assigned in parallel). Scheduling of logical days onto real dates was already confirmed correct in LIF-16 and is not re-audited here. Whether the library's editorial content is clinically appropriate is out of scope.

Delivery Renderer Findings

Delivery Renderer Area — Audit Round 2026-07-18

Status: completed; findings recorded only, no runtime fixes applied

Files inspected

- `app/content_library.py`
- `app/plan_finalization.py`
- `app/ux/task_notification.py`
- `app/scheduler.py`
- `app/telegram.py`
- `app/plan_guards.py`
- `app/plan_runtime/tools.py`
- `app/plan_pause.py`
- `app/ux/catalog.py`
- `resource/assets/content_library/tasks/burnout_combined_content_library.json`
- `resource/assets/ux/trigger_messages.json`
- `resource/assets/product/conceptual_map.md`
- `resource/assets/product/conceptual_map_en.md`
- `delivery/task-related tests` in `tests/`

Method

The audit traced the complete user-facing path:

```
v5 content JSON
→ ContentLibrary.content_payload
→ plan finalization / AIPlanStep snapshot
→ scheduler job payload
→ Telegram exercise message + buttons
→ complete/skip callback
→ post-click acknowledgement
→ expiry / keyboard removal
```

The current renderer was also executed directly against a real v5 library item to verify the output rather than relying only on static code inspection.

Summary

The core exercise message is deterministically broken for the current v5 library. All 8 active items store their user-facing fields under `display`, while finalization and rendering still read the legacy root schema. The resulting Telegram message can contain an internal exercise ID, no instructions, and an unlabeled numeric duration. Buttons are attached correctly, but their pause/cancel lifecycle is inconsistent with the Product Map. The post-click path also activates legacy randomized engagement/adaptation copy that is outside the accepted MVP contract.

Findings

DEL-01 — V5 `display.*` is not rendered in the exercise message

Severity: BLOCKER / P0 Status: confirmed and reproduced

`load_content_library()` preserves the nested v5 payload; it does not flatten `display`. Every current inventory item has `display.title`, `display.steps`, and `display.duration_label`, and none has root-level `title` or `instructions`.

The live path still reads the legacy shape in two places:

- `plan_finalization._build_step_title()` reads root `title`, then falls

back to `ContentLibrary.internal_name`; the loader defaults that field to the exercise ID;

- `format_task_notification()` reads root title, instructions, and

numeric `duration_minutes/duration_estimate` instead of the nested display object.

Direct reproduction with `somatic_004_v2` produced:

```

□□□□□□□□□□□□□□□□
□□ <b>somatic_004_v2</b>
День 1 · День · 1 з 1

□ 1
□□□□□□□□□□□□□□□□

```

The actual title `Дихання`, its four instructions, and the duration label `30–60 сек` are all absent. This breaks the primary daily product loop.

Expected behavior: one canonical v5 display reader produces the trusted `title`, ordered `steps`, and `duration_label` for both finalization and Telegram rendering. Missing/invalid display content must fail visibly in logs and tests rather than silently degrade to `Завдання` or an internal ID. The same display data must later feed `current_exercise_context` (cross-reference: COACH-08).

Minimal fix: introduce one structured display extractor/validator and use it in finalization and delivery. Render the step array as concrete ordered instructions; do not convert it into an opaque legacy `instructions` field.

DEL-02 — Delivery exposes internal scheduling metadata and has a dormant pre-action rationale path

Severity: P1 Status: confirmed

The target contract is `title → steps → duration → buttons`. The current message also renders:

```
День {plan_day_number} · {DAY/EVENING label} · {task_index} з {task_total}
```

`DAY/EVENING` are internal scheduling tags under FD-03/C8, not exercise content. The day/task counter is likewise outside the accepted delivery shape and adds implementation framing to a deliberately small action.

`format_task_notification()` also contains a `Чому це працює` block before the instructions. The current v5 inventory has no rationale field, so the path is dormant today, but adding one later would violate FD-06: **rationale is not rendered before or after the exercise in MVP** (removed 100%).

Minimal fix: render only trusted display content in the exercise body and keep internal slot, mechanic, task index, and rationale out of the message entirely (per FD-06 / `delivery_contract.md`).

DEL-03 — Legacy randomized engagement/adaptation runs after complete and skip

Severity: P1 Status: confirmed

After the callback toast, both handlers send an additional chat message from `trigger_messages.json`. The catalog is old friend-bot/plan logic and contains unsupported or false claims, including:

- the same exercise returning tomorrow even though the sequence rotates;
- changing difficulty or redesigning/reviewing the plan after two skips;
- internal `focus` framing and old `plan` terminology;
- pressure and judgment such as `головне не два підряд, не зупиняйся`,

and personality conclusions from a streak.

After two explicit skips, the runtime also adds a `Переглянути план` button and routes `adapt_suggest` into Coach. This is an automatic adaptation/engagement prompt even though C7 says adaptation does not trigger at all in MVP and C3 keeps expanded closure/reflection at P2.

Expected MVP behavior (per FD-06 / `delivery_contract.md`): persist the canonical result, then **edit the same delivered message** — do not send a new one. Specifically:

- remove the Done/Skip buttons;
- show a durable in-message status `Виконано / Пропущено`;
- after `Виконано`, show the optional completed-only feedback tap

(`Допомогло?`);

- after Пропустити, show **no** feedback control;
- do **not** send any additional engagement/adaptation message, and do not

activate the random catalog or an adaptation CTA.

The exact acknowledgement copy can be polished later; disabling unsupported behavior does not need a new product decision.

DEL-04 — Delivered buttons conflict with pause and cancellation semantics

Severity: P1 Status: confirmed

The Product Map says exercise buttons remain available until the end of the user's local day. `validate_step_action()` instead requires both an active plan and `user.current_state == ACTIVE`.

Consequences:

- if the user pauses after receiving an exercise, the already-delivered

Done/Skip buttons immediately stop working and answer План зараз не активний, although pause is defined as stopping future delivery;

- `cancel_plan()` removes scheduler jobs but does not mark delivered steps

canceled and does not remove their Telegram keyboards. Stale buttons remain visible until expiry and answer the same error when pressed.

Minimal fix: make already-delivered current-day actions remain actionable through their existing expiry when delivery is paused, unless the product contract is explicitly changed. On cancellation, close pending/delivered steps and remove visible keyboards immediately.

Cross-references: COACH-04 covers future deliveries lost during a pause; LIF-17 covers the separate expiry/keyboard/ignored timing drift. This finding covers only the delivered-message interaction surface.

DEL-05 — A transient Telegram send failure permanently loses the exercise

Severity: P1 Status: confirmed

`_send_message_async()` catches a Telegram exception and returns `None`. `send_scheduled_message()` logs `task_delivery_failed`, but the APScheduler date job has already fired. The step remains pending with a `scheduled_for` timestamp in the past, while startup restoration selects only future pending steps. There is no retry or recovery path inside the two-hour late-delivery window.

Expected behavior: a bounded, idempotent retry/recovery path may re-attempt the same step within the allowed delivery grace period, records each failure, and never creates duplicate delivered events/messages.

DEL-06 — Renderer coverage is absent and callback tests are stale

Severity: P1 Status: confirmed

There is no direct regression test for `format_task_notification()` and no test that renders the actual v5 `display.*` payload. Existing lifecycle tests cover local expiry calculations, but not the final message body or keyboard cleanup behavior.

The callback suite is also not executable as a reliable guard:

- its default `test-token` is rejected by the current aiogram token

validator during collection;

- with a syntactically valid fake token, the targeted run produced

35 passed, 17 failed;

- 8 asyncio failures come from stale `DummyStep` objects that do not have canonical `step_status`; 9 more are the unconfigured/missing `trio` backend.

Minimum tests before beta:

- render every active v5 item from `display.title`, ordered

`display.steps`, and `display.duration_label`;

- assert that internal IDs/slots/mechanics and rationale are absent

(before AND after the action, per FD-06);

- assert the two callback IDs and ownership/idempotency behavior;
- assert on-tap **edit-in-place** of the same message (no new message) with

the correct keyboard replacement and durable status;

- assert the feedback tap appears **only** after *Виконано*, never after

Пропустити;

- cover active, paused, canceled, completed, skipped, and expired button

states, including that *expired/canceled* remove the keyboard and *paused* leaves an already-delivered exercise actionable;

- cover successful delivery, bounded retry, and duplicate prevention;
- cover keyboard removal at expiry and cancellation.

DEL-07 — Dynamic HTML content is not escaped

Severity: P2 hardening Status: confirmed; no current content failure

Telegram delivery uses `parse_mode="HTML"`, but title, instructions, and duration are inserted without HTML escaping. A future content edit with `<`, `>`, or `&` can turn a valid exercise into a Telegram parse failure, which currently becomes the permanent loss described in DEL-05.

The 8 current display payloads contain no HTML-sensitive characters and their core rendered content is small (the largest title/steps/duration combination is 165 characters), so this is not the cause of the current P0. Escape every dynamic display field and add a message-size assertion as part of the renderer rewrite.

DEL-08 — Delivery mechanics that already match the contract

Severity: OK Status: confirmed, with the limitations above

- the scheduled exercise is sent as one Telegram message with *Виконано*

and *Пропустити* buttons attached immediately;

- callback data identifies the exact plan step;
- ownership is checked before a step is changed;
- canonical terminal states make repeated complete/skip actions

idempotent in the runtime guard;

- `expires_at` is calculated from the user's local day;
- the expiry job marks unanswered steps expired and attempts to remove the

keyboard.

These pieces should be preserved while fixing the content and lifecycle gaps rather than replacing the whole delivery path.

Required MVP work

- fix the v5 schema mismatch and use one canonical display reader

(DEL-01);

- remove internal metadata and the pre-action rationale path from the

message (DEL-02);

- freeze the legacy post-click engagement/adaptation layer and keep one

deterministic acknowledgement (DEL-03);

- implement on-tap **same-message state transition** (edit, not new

message) with changing buttons and durable status (DEL-03, FD-06);

- implement the **completed-only feedback callback** and persist its result

(`exercise_id`, `answer`, `timestamp`); wired to storage only (FD-05/FD-06);

- align pause/cancel keyboard behavior with the Product Map (DEL-04);
- add bounded delivery retry/idempotency (DEL-05);
- replace stale callback tests and add end-to-end renderer coverage

(DEL-06).

The full delivery shape and copy rules are defined in `docs/audit/delivery_contract.md` (FD-06); this list is the code work to satisfy it.

HTML escaping and a size guard (DEL-07) are low-cost hardening and should be included in the renderer change if possible, but they are not the source of the current deterministic P0.

Scope boundary

This round covers conversion of trusted exercise data into the Telegram message and the immediate Done/Skip interaction lifecycle. Exercise selection belongs to Plan Generation; editorial adequacy and the number of steps per exercise belong to the later Content Library audit. General pause rescheduling, completion reports, and Coach context are tracked in COACH-04, Lifecycle Completion, and COACH-08 respectively.

Runtime Tools Findings

Runtime Tools Area — Audit Round 2026-07-22, full-matrix revalidation 2026-07-23

Status: completed; findings recorded only, no runtime fixes applied. **First pass was too narrow** — it only compliance-checked the evening-time rule the founder flagged, not the area. The revalidation takes the whole tool matrix (Coach state→tools + prompt rules) and checks every tool through schema, registration, backend guard, side effects, result handling, and tests.

Method: for each of the 8 Coach-callable runtime tools — what states/rules does the Coach side declare, what does the backend actually enforce, do the side effects fulfil the declared result, and can the user-facing reply be trusted?

Files inspected

- `app/plan_runtime/tools.py` — all 8 Coach-callable runtime functions
- `app/orchestrator.py` — tool registry, `_execute_plan_tool`, `result/error`

formatting, context builder, and evening cascade

- `app/workers/coach_agent.py` — prompt contract, schemas, state→tools matrix, `_coach_tools_for_state`, and API call
- `app/time_slots.py` — validation and future-step recomputation
- `app/plan_drafts/service.py` — first/follow-up plan invariants
- `app/plan_finalization.py` — activation and scheduling side effects
- `app/plan_pause.py`, `app/plan_guards.py`, and `app/scheduler.py`
- `app/session_memory.py` — pending 14-day action
- runtime-tool, Coach schema/filter, orchestrator, pause/resume, and status tests
- `conceptual_map.md`, `conceptual_map_en.md`, and the Coach system prompt

Summary — the root problem

The outer command boundary is materially better than it was before the Coach rewrite:

- only allowlisted tools are registered;
- tools are filtered before the API call by `current_state`;
- schemas are strict, reject extra properties, and require tool arguments;
- unknown DB fields are not dumped directly to the user;
- pause, resume, and cancel have backend state checks.

But the full path still has four systemic gaps:

1. tool gating is state-based while several preconditions are plan-type- or pending-flow-based, and prompt prose is the only bridge;
2. some tools commit product state before external scheduler side effects, then return success even when those side effects failed or never happened;
3. the Coach never receives the tool result, so hardcoded Ukrainian templates become a second, drifting source of user-facing truth;
4. Product Map promises one atomic 7↔14 format switch, but no such tool exists.

Per-tool matrix (Coach declaration vs backend)

Tool	Coach offers in states	Backend guard	Aligned?
create_followup_plan	IDLE_PLAN_ABORTED only	FINISHED, DROPPED, ABORTED + silent defaults	mismatch → RT-06
record_evening_time	IDLE_PLAN_ABORTED plus pending 14-day flow	no state/pending guard; format-only time validation	mismatch → RT-02 / RT-10
change_day_time	ACTIVE, ACTIVE_PAUSED, ABORTED	no state guard	partial/paused false-result risk → RT-08 / RT-12
change_evening_time	ACTIVE, ACTIVE_PAUSED, ABORTED plus configured evening	no state/plan-format guard	false success for 7-day → RT-01 / RT-12
pause_plan	ACTIVE	ACTIVE	state aligns; promised preservation does not → RT-13
resume_plan	ACTIVE_PAUSED	ACTIVE_PAUSED + profile flag	state aligns; "next remaining day" does not → RT-13
cancel_plan	ACTIVE, ACTIVE_PAUSED + extra consent	state only	consent is model-governed; cleanup incomplete → RT-13
get_plan_status	ACTIVE, ACTIVE_PAUSED, ABORTED	any state (read-only)	mostly aligned; final-day facts can be wrong → RT-14
plan switch 7↔14	—	—	does not exist → RT-07

The prompt table and `_TOOL_NAMES_BY_STATE` match each other exactly. The remaining mismatches are between that declaration and runtime truth, not between the two copies of the Coach-side matrix.

Reverse audit — Product Map promise to execution path

This pass also starts from the user-facing Product Map rather than the existing tool list. A declared capability does not automatically require a Coach tool: deterministic lifecycle operations, Telegram callbacks, and supplied runtime context are separate execution mechanisms.

Product Map capability	Correct execution owner	Current status	Missing Coach tool?
Create the first 7 working days after setup	deterministic onboarding completion	backend flow missing/incomplete; ONB-01/ONB-07	No
Automatically continue 7→7 or 14→14 after completion	deterministic completion lifecycle	not implemented; LIF-04/FD-01	No
Pause delivery	pause_plan	tool exists; preservation semantics incomplete (RT-13)	No
Resume with the next remaining day	resume_plan	tool exists; re-anchoring incomplete (RT-13)	No
Permanently cancel the current period	cancel_plan	tool exists; cleanup incomplete (RT-13/DEL-04)	No
Start a new 7- or 14-day period after cancellation	create_followup_plan	tool exists; guards/defaults drift (RT-06)	No
Change daytime or configured evening delivery time	change_day_time / change_evening_time	tools exist; plan/state/reconciliation gaps (RT-01/RT-03/RT-08/RT-12)	No
Switch 7↔14 as one confirmed action	atomic switch_plan_format	no schema, backend operation, registry entry, or test	Yes — RT-07 / COACH-11
Choose the first evening time when entering 14 days	record_evening_time plus pending creation/switch flow	tool exists; pending-flow guard incomplete (RT-02)	No
See current period status/progress	get_plan_status; completion report for closed periods	tool exists; final-day calculation gap (RT-14)	No
Mark the delivered exercise Done or Skip	Telegram callback buttons	implemented outside Coach tools	No
Treat no button response by local end of day as not completed	expiry/lifecycle job	deterministic delivery lifecycle, not a conversation action	No
Explain how to perform today's exercise	current_exercise_context supplied to Coach	context gap tracked under COACH-08	No
Send the completion summary and prepare the next period	completion/report lifecycle	lifecycle gaps already tracked	No
Direct an unanswered factual question to support	deterministic support contact or escalation flow	no real path; COACH-10 backlog	Maybe later; not currently offered as an action

Therefore the only missing Coach action required by the current Product Map is `switch_plan_format`. `create_first_plan`, automatic continuation, completion, Done/Skip, expiry, and exercise-context retrieval must not be converted into LLM tools merely because they are missing or incomplete elsewhere.

The Product Map says the user chooses working days during setup, but does not promise that working days can be changed during an active period. Likewise it does not promise user-controlled timezone changes, one-off snoozing, replaying an exercise, marking Done/Skip through free text, or retrieving arbitrary historical reports through the Coach. Possible future tools such as `change_work_days`, `change_timezone`, `snooze_current_exercise`, `record_exercise_result`, or `get_latest_completion_summary` would expand the product contract rather than close a current implementation gap.

Of those candidates, `change_work_days` is the strongest post-MVP possibility: Fogg's Ability/Routine-fit lens says a changed work schedule can make every prompt mistimed. But current users can skip, pause, or change time, and there is no usage evidence yet that runtime work-day editing is a bottleneck. Keep it as a decision/data question rather than adding another scheduling mutation before the core loop is reliable.

The Fogg working paper is an analytical lens, not the MVP source of truth. Its recommendations to let users choose exercises, generate tiny variants, or adapt content after behavior conflict with the approved Product Map and the frozen adaptation decision. They do not justify additional runtime tools in this round.

Consent boundary

Natural-language intent and consent are interpreted by the Coach. The backend can validate tool name, state, arguments, and product invariants, but it does not independently know whether a sentence was genuine confirmation. Consequently, cancellation's additional confirmation requirement is enforced by prompt/model behavior, not a durable confirmation token or callback.

This is not automatically a separate architecture bug for an AI-first MVP, but it is a real control boundary. It requires scenario/evaluation coverage. If deterministic proof of irreversible consent is later required, cancellation needs a confirmation button/token; adding another prose rule will not create backend enforcement.

Findings

RT-01 — `change_evening_time` gives a false success for 7-day users

Severity: P1 Status: confirmed

`change_evening_time` (tools.py:222-247) calls `update_user_time_slots` with `{"EVENING": hhmm}` and reschedules future steps. It never checks `evening_slot_collected` or whether the active plan is MEDIUM. For a SHORT plan there are no evening steps, so `update_user_time_slots` (time_slots.py:247, merge semantics) simply stores an EVENING value the plan never uses and `reschedule_plan_steps` gets an empty list → `rescheduled=0` — yet the function returns `{"status": "ok"}` and the orchestrator replies "Вечірній час змінено" (orchestrator.py:1230). The user is told a time changed that affects nothing in their plan.

Fix (tools.py): guard at the top — load profile + active plan; if `evening_slot_collected` is False, or the active plan is SHORT / has no EVENING slot, return a soft `{"status": "no_evening_slot"}` and persist nothing. Do not claim success.

RT-02 — `record_evening_time` sets the 14-day flag with no context guard

Severity: P2 Status: confirmed

`record_evening_time` (tools.py:167-191) persists EVENING and sets `evening_slot_collected=True` for any caller in the gated state. The MEDIUM auto-create only fires when `pending_action == collect_evening_time_for_medium` (orchestrator.py:1316), but the persistence + flag set happen unconditionally. If the tool mis-fires (LLM error) outside that flow, a 14-day-only flag is set for a user with no pending 14-day creation.

Fix: gate persistence to the pending-MEDIUM context — either the orchestrator only routes `record_evening_time` when `pending_action == collect_evening_time_for_medium`, or the tool no-ops outside it. Lower severity than RT-01 (it stores a real user-provided value), but it is a 14-day flag set outside the 14-day flow.

RT-03 — Tool gating is state-based; evening relevance is plan-type-based; only prompt prose bridges them

Severity: P1 Status: confirmed

The state→tools matrix (coach_agent.py:652-654) lists `change_evening_time` under ACTIVE and ACTIVE_PAUSED unconditionally. A user in ACTIVE may hold a SHORT or MEDIUM plan — the state does not distinguish. The prompt notes (coach_agent.py:658-660) say `change_evening_time` is "available only when an evening time is already configured"

and `record_evening_time` "only while a 14-day is pending" — but `_coach_tools_for_state` (`coach_agent.py:962-970`) filters purely by `current_state` and enforces neither. So the only thing stopping a 7-day user from getting the evening tool is the LLM obeying a prompt note. Per this audit's standing discipline, the prompt is not a guarantee.

Fix: make tool filtering plan-type-aware (needs RT-04). Drop `change_evening_time` when the active plan is SHORT / no evening configured; expose `record_evening_time` only when a MEDIUM creation is pending.

RT-04 — Coach context omits plan type, so the filter cannot be plan-type-aware

Severity: P1 (enabler for RT-03) Status: confirmed

The context payload built in `orchestrator.py:1124-1130` carries `current_state` but no `plan_type` / `total_days`. `_context_message` (`coach_agent.py:738-748`) forwards only `current_time`, `current_state`, `completion_context`. So neither the LLM nor `_coach_tools_for_state` has the information needed to distinguish 7-day from 14-day.

Fix: add the active plan's `plan_type` (or `total_days`) and `evening_slot_collected` to the payload at `orchestrator.py:1124`, thread them through to `_coach_tools_for_state`, and use them in the filter (RT-03).

RT-05 — Dead / removed-from-Coach code inventory (delete, do not patch)

Severity: P1 (correctness of the removal plan) Status: confirmed by grep + cross-referenced

Consolidated index of code that is **dead or already stripped from the Coach, but still present** in the backend. Founder directive: these are to be **removed**, not guarded/patched. Several existing findings advise patching guards on code that is actually being deleted — that framing is wrong and is corrected here.

- 1. `create_first_plan` — dead wrapper, no callers.** `grep` confirms zero callers anywhere (`app/plan_runtime/tools.py`): the only references are inside its own body (error string + log). It is not a Coach tool. ONB-07, COACH-07, and FSM-01 all describe it as "update the `IDLE_ONBOARDED` entry guard when that state is removed" — **wrong disposition**. You do not patch the guard of a function you are deleting. Correct disposition: **delete** `create_first_plan`. First-plan creation (when the real onboarding, ONB-01, is built) calls `create_plan()` in `plan_drafts/service.py` directly — it already enforces "first plan must be SHORT" (`service.py:51`). No wrapper or transition-matrix entry is needed.
- 2. `IDLE_ONBOARDED` — dead state.** Never assigned in production (`grep` for assignment finds none; only comparisons in the dead `create_first_plan` guard and the FSM transition table). Its only "consumer" is the dead wrapper above. Remove from state definitions, guards, `ONBOARDABLE_STATES`, `IDLE_STATES`, and transition tables. Tracked as FSM-01 / ONB-07.
- 3. `IDLE_DROPPED` — dead state, orphan writer.** The only writer (`orchestrator.py:767`) has no live production caller (FSM-02); explicit abandonment uses `IDLE_PLAN_ABORTED`. Remove the orphan writer and all `IDLE_DROPPED` allowances; migrate any existing rows. Tracked as FSM-02. (This corrects RT-06's earlier "add `IDLE_DROPPED` to the Coach matrix" direction — the state is removed, not wired.)
- 4. `SCHEDULE_ADJUSTMENT` — zombie subsystem.** The state is **never entered** (`grep` for assignment finds none), yet a full machinery references it across 8 files: a `stuck_schedule_adj_check` cron job that scans for users in it (`scheduler.py:829`), Telegram `sched_adj_timeout` callbacks, session-memory `schedule_adjustment_context`, `orchestrator` routing (`:1381`), FSM transitions, and constants. Remove the whole subsystem, not just the constant. Tracked as FSM-03 (and SCH-02).
- 5. `MORNING slot` — frozen, unused, still carried.** Not produced by any P1 plan (recipe is DAY/EVENING only; `schemas/planner.py:51` marks it "frozen"), yet it persists as defaults and branches across ~7 files (`db.py:157`, `plan_finalization.py:92`, `api.py:57/83/89`, `time_slots.py`, `task_notification.py:9`). **Not in Codex's inventory — added here.** Lower priority than 1–4 (it is inert, not wired to a live job), but it is a latent user-facing leak risk (a `MORNING/Панок` label could surface) and dead weight. Decide: remove, or keep explicitly frozen with a single guarded definition rather than scattered branches.
- 6. `app/fsm/guards.py / can_transition` — dead after FSM-03 and FSM-10 cleanup.** Its only call paths are the zombie `schedule-adjustment` tunnel and the unreachable `transition_signal` envelope. Delete the file, import, and generic transition helpers after those two owners are removed. Aggregate validation is a separate lifecycle concern, not a reason to preserve this matrix (FSM-04).

Distinction to keep: items 1–6 are **dead** (remove). This is different from *orphaned-but-reachable* gaps (e.g. a live state with no Coach path) — those are behavior gaps, not dead code, and are tracked in their own findings.

Fix: one removal pass that deletes 1–6 together with a forward migration for any persisted `IDLE_DROPPED` / `SCHEDULE_ADJUSTMENT` rows, and updates ONB-07 / COACH-07 / FSM-01 wording from "patch the guard" to "remove the wrapper."

RT-06 — `create_followup_plan` has stale states and fail-open defaults

Severity: P1 Status: confirmed; cross-reference COACH-03 / FSM-02

Backend `_FOLLOWUP_STATES = {IDLE_FINISHED, IDLE_DROPPED, IDLE_PLAN_ABORTED}` (`tools.py:25`). But `_TOOL_NAMES_BY_STATE` (`coach_agent.py:952`) offers `create_followup_plan` **only** in `IDLE_PLAN_ABORTED`. Under the accepted target, the Coach matrix is the correct direction: `IDLE_FINISHED` auto-continues through FD-01, and `IDLE_DROPPED` is removed. Adding those rows back to the Coach would restore the old architecture rather than reconcile it.

There are also two fail-open defaults:

- `_build_tool_registry()` uses `args.get("plan_type", "SHORT")`, silently

creating 7 days if the required model argument is missing;

- `create_followup_plan()` uses "14:00" when the saved DAY time is absent,

silently inventing a schedule instead of reporting broken setup.

Both contradict the prompt contract: the user chooses the specific format, and saved setup is the source of truth.

Fix:

- narrow `_FOLLOWUP_STATES` to `IDLE_PLAN_ABORTED`;
- keep automatic same-format continuation as a separate deterministic

lifecycle service, not a Coach call from `IDLE_FINISHED`;

- remove `IDLE_DROPPED` with FSM-02;
- require `args["plan_type"]` and fail closed when the saved DAY time is

missing; do not default either decision.

RT-07 — No plan-switch (7↔14) capability exists

Severity: P1 before beta under the current Product Map Status: confirmed; accepted target dependency, not implemented

The English and Ukrainian Product Maps explicitly tell the Coach that format can be switched from 7 to 14 or 14 to 7 as **one user-confirmed action**. `product_contract.md` also lists `switch_plan_format` among the known backend dependencies that must close before beta. No schema, registry entry, backend function, state row, or test exists.

The only current approximation is `cancel_plan` followed by `create_followup_plan(other type)`. That is not equivalent:

- it requires two actions and extra friction;
- cancellation permanently ends the current sequence and suppresses its

progress summary;

- a failure between the two actions leaves the user with no active sequence;
- first switch to 14 days needs an evening-time subflow.

This is not a quick wrapper around two existing calls. The switch must be atomic at the data level: validate all inputs first, end the old sequence and create the new format in one transaction, roll back to the old sequence if creation fails, then reconcile scheduler jobs idempotently. For the first 7→14 switch, collect evening time before ending the current sequence.

Current contract gives two honest options: implement the atomic tool before beta, or explicitly remove the capability from Product Map and the Product Contract for MVP. Leaving the claim while calling it optional is not coherent.

RT-08 — `change_day_time` / `change_evening_time` have no backend state guard

Severity: P2 Status: confirmed

Neither `change_day_time` (`tools.py:194`) nor `change_evening_time` (`tools.py:222`) checks `current_state` — they validate HH:MM and update. All state gating for them lives in the Coach matrix. So any caller path that bypasses the matrix (a direct call, a future entry point, an LLM misfire the matrix didn't catch) can change delivery times in any state. Contrast

pause/resume/cancel, which all guard `current_state` in the backend. The evening guard from RT-01 partly covers `change_evening_time`; `change_day_time` should also assert a sane state (has a plan / follow-up context) rather than trusting the matrix alone.

RT-09 — The Coach does not voice tool outcomes; a fixed template does

Severity: P1 Status: confirmed (cross-ref COACH-09)

The prompt (`coach_agent.py:669-674`) instructs the Coach to return **only** the tool call, no user-facing text, and not to claim success. The orchestrator then sends a fixed `_TOOL_REPLY_TEMPLATES` string (`orchestrator.py:1226-1235`). So the Coach never sees or interprets the actual result — the user hears a canned line. Consequences directly relevant to this round:

- soft/failure results have no natural voice. Today only two are handled

explicitly (`needs_evening_time`, the MEDIUM cascade); the RT-01 `no_evening_slot` result proposed above would have no template and fall through unless the orchestrator is extended case-by-case.

- the founder's concern is the inverse risk once the Coach *is* allowed to

respond: the backend returns one thing and the Coach, not seeing it, narrates something else ("схоже, система заблокувала...") and invents detail. The Coach is the system's face; it must speak from the real result, not around it.

Fix (COACH-09): a bounded tool-result loop — the tool result is returned to the Coach, which forms one natural response grounded in the actual outcome (success, soft, or failure), instead of the orchestrator guessing a template per tool. This is the same Bounded Tool-Result Loop already recorded as COACH-09; this round confirms the runtime tools depend on it, especially once soft results (RT-01) exist.

The current templates also bypass Section 3 language adherence: every success, soft result, and error is Ukrainian even when the user speaks English. Several templates restore old user-facing `план` terminology, and the evening-time question requires manual 20:30 input despite the prompt allowing natural language. These are additional symptoms of the same missing result loop, not reasons to grow a larger template catalog.

RT-10 — backend time validation accepts impossible values

Severity: P1 Status: confirmed and directly reproduced

`_validate_hhmm()` checks only `^\d{2}:\d{2}$`. Direct execution accepted all of 24:00, 99:99, and 12:60. The strict OpenAI schema correctly prevents those values on a compliant model call, and `change_day_time / change_evening_time` later pass through `time_slots._parse_time()`. But `record_evening_time` writes the value directly after `_validate_hhmm()`, so an invalid time can be persisted as trusted profile setup.

Expected behavior: one backend validator is authoritative for every time entry point and enforces real 24-hour time (00:00 through 23:59). The schema remains defense-in-depth, not the only semantic validation.

This validator does not choose or convert a timezone. The accepted `HH:MM` is a local wall-clock value in the user's saved `user.timezone`. `change_day_time / change_evening_time` already pass that timezone into `compute_scheduled_for()`, which converts the result to UTC; first MEDIUM-plan finalization does the same after `record_evening_time`. Correct collection and validation of `user.timezone` is a separate setup contract.

Minimal fix: replace the format-only regex helper with parsing that returns a normalized `HH:MM`, and use it in all three time tools and onboarding. Add boundary tests for 00:00, 23:59, 24:00, invalid minutes, and malformed input.

RT-11 — plan creation can report success after activation side effects failed

Severity: P1 Status: confirmed

Both creation tools commit the finalized plan and `ACTIVE` state, then call `activate_plan_side_effects()` outside that transaction. That function catches every exception, logs it, and returns `None`; it also returns normally when the plan/user cannot be loaded. The caller therefore always returns `{"status": "ok"}` and the orchestrator says the new sequence was launched, even if no exercise jobs were scheduled.

The failure can also be partial: scheduling earlier steps mutates `APScheduler`, then a later exception is swallowed. The DB may say active while only part of the sequence has jobs.

Minimal fix: make activation an idempotent, observable operation. At minimum, return/raise a structured activation result and never claim full success when job activation failed. The durable target should use an outbox/reconciliation record so a committed plan can retry job activation without duplicate jobs. Do not attempt to make a DB transaction roll back external scheduler changes.

Founder directive (2026-07-23): the silent fail-open is **not acceptable** — this is deterministic backend code, it must not swallow a failure and report success. Required: either (a) a bounded retry/reconciliation policy that guarantees the jobs eventually exist, or (b) fail **closed** — surface the failure so the plan is not reported as launched. Do not ship the current "looks OK everywhere, user gets nothing" path. Success must mean the delivery jobs actually exist.

RT-12 — time changes commit before scheduler reconciliation and fail while paused

Severity: P1 Status: confirmed; related to COACH-04 / FSM-06

`change_day_time` and `change_evening_time` first commit the profile and rewritten `scheduled_for` timestamps, then call `reschedule_plan_steps()` in a separate session. If rescheduling raises, the orchestrator reports failure although the saved time and DB schedule already changed.

For `ACTIVE_PAUSED`, the prompt explicitly offers both time tools, but `schedule_plan_step()` refuses every step because `can_deliver_tasks()` accepts only `ACTIVE`. The tool therefore commits the new times, schedules zero jobs, and returns `status=ok`. Resume only flips flags/state and does not rebuild the jobs. The promised result — future delivery at the new time after resume — is not implemented.

Minimal fix: treat schedule reconciliation as an idempotent post-commit result, not an assumed side effect. For paused users, persist the new time but defer job creation to the required resume re-anchoring flow. Return structured facts such as `saved=true`, `jobs_reconciled=true/false/deferred`, and let the bounded result loop describe only what actually happened.

RT-13 — state guards align for pause/resume/cancel, but their declared effects do not

Severity: P1 Status: confirmed; cross-reference COACH-04, DEL-04, and FSM-06

The backend correctly rejects pause outside `ACTIVE`, resume outside `ACTIVE_PAUSED`, and cancel outside those two states. That part matches the prompt. The promised consequences do not:

- pause says the remaining sequence is preserved, but due steps fire and are lost while delivery is gated;

- resume says delivery continues with the next remaining day, but it only

flips `is_paused` and `current_state`; it does not re-anchor or reschedule;

- resume does not verify that an active/paused `AIPlan` actually exists;
- cancel changes plan/user state and removes future scheduler jobs, but does

not close delivered/pending step state or remove visible Telegram keyboards (DEL-04).

Cancellation's additional verbal confirmation is also model-governed as noted under Consent boundary; the backend validates state, not conversation history.

Minimal fix: do not create another set of tool-specific work items here. Implement null-on-pause plus re-anchor/recompute-on-resume once (COACH-04 / FSM-06), implement cancelled-step/keyboard closure once (DEL-04), and add integration tests through the runtime tools so their advertised results become true.

RT-14 — `get_plan_status` can report one day remaining after the last day was delivered

Severity: P1 factual correctness Status: confirmed

`current_day` advances when all steps for a day are delivered, not when they are completed. On the final day, `maybe_advance_current_day()` caps the value at `total_days`; it cannot advance to `total_days + 1`. `get_plan_status()` then calculates:

```
days_remaining = total_days - current_day + 1
```

Therefore, after every final-day exercise has already been delivered, `current_day == total_days` and the tool still reports 1 day remaining until the plan moves out of the active state. The existing test hardcodes a middle day and does not cover this boundary.

Minimal fix: define status facts from actual scheduled days/remaining non-terminal or not-yet-delivered steps rather than inferring all semantics from a capped cursor. Add first-day, middle-day, final-day-before-delivery, final-day-after-delivery, paused, and no-plan cases. Keep the structured result; the bounded Coach result loop should verbalize it.

RT-15 — tests prove wiring, not the declared product outcomes

Severity: P1 test expansion Status: confirmed

The targeted current suite passed (44 passed with unavailable Trio cases excluded), but it primarily mocks each function's collaborators and asserts that they were called. It does not cover:

- impossible but regex-valid times;
- missing `plan_type` silently becoming `SHORT`;
- missing `DAY` time silently becoming `14:00`;
- activation-side-effect failure after plan commit;
- time-change reconciliation failure or paused time changes;
- pending-action absence/expiry around first evening collection;
- actual `_execute_plan_tool()` success/soft/error/cascade behavior;
- language adherence after a tool call;
- final-day status facts;
- cancellation consent scenarios or cleanup through the runtime boundary;
- the absent format-switch contract.

The green suite is useful regression coverage for current wiring, but it is not evidence that the prompt/tool contract is fulfilled. Add outcome-level tests alongside the fixes rather than enlarging mocks of the current behavior.

Recommended fix shape

Priority order:

1. **Close false-success paths (RT-11/RT-12/RT-13).** A created or changed

sequence must have observable, retryable scheduler reconciliation; pause, resume, and cancellation must fulfil their existing Product Map contract.

2. **Align follow-up entry points (RT-06).** Automatic continuation is not a

Coach tool; user-requested follow-up creation is only from `IDLE_PLAN_ABORTED`; remove stale states and silent defaults.

3. **Add authoritative input/context guards (RT-01/RT-02/RT-08/RT-10).**

Invalid time, wrong plan format, wrong state, or absent pending flow must fail before persistence.

4. **Make filtering plan-aware (RT-03/RT-04).** The Coach should not receive

tools that cannot be meaningful for the current plan/context.

5. **Implement the bounded tool-result loop (RT-09/COACH-09).** The Coach

receives one safe structured result and writes one grounded reply; templates remain failure fallback only.

6. **Resolve the accepted format-switch contract (RT-07).** Implement the

atomic tool before beta, or explicitly remove the promise from Product Map and Product Contract.

7. **Correct status facts and outcome tests (RT-14/RT-15).**

Per-file work list

- `app/plan_runtime/tools.py`
- `change_evening_time`: add the no-evening-slot guard (RT-01);
- `record_evening_time`: gate persistence to the pending-MEDIUM flow (RT-02);
- use one semantic time parser for record/change operations (RT-10);
- add backend state/context guards to both time-change tools (RT-08);
- narrow follow-up creation to `IDLE_PLAN_ABORTED`; require explicit `plan_type` and saved `DAY` time (RT-06);
- return structured, factual results for activation and scheduler

reconciliation instead of unconditional `status=ok` (RT-11/RT-12);

- implement `atomic switch_plan_format(plan_type)` or remove its product

promise before beta (RT-07);

- derive status fields from real remaining/delivered steps (RT-14).
- `app/orchestrator.py`
- `_execute_plan_tool / cascade`: handle the new `no_evening_slot` soft

result; refuse `record_evening_time` unless `pending_action == collect_evening_time_for_medium` (RT-02);

- remove the `SHORT` default in the `create_followup_plan` registry wrapper

(RT-06 / COACH-03);

- context builder (: 1124-1130): add `plan_type/total_days +`

`evening_slot_collected` (RT-04);

- `_humanize_tool_error`: map the new guard errors to friendly copy;
- tool-result handling: replace per-tool fixed templates with a bounded

result loop so the Coach voices the real outcome in the user's language (RT-09/COACH-09);

- emit telemetry for requested / soft-pending / succeeded / failed as

distinct outcomes, not one `plan_tool_executed` event.

- `app/workers/coach_agent.py`
- `_context_message`: forward `plan_type / evening-configured` (RT-04);
- `_coach_tools_for_state`: filter evening tools by plan type + pending

state (RT-03);

- keep `IDLE_FINISHED` and removed `IDLE_DROPPED` out of the Coach tool

matrix; prove the deterministic auto-continuation path separately (RT-06);

- add `switch_plan_format` schema/state availability only when its backend

atomic operation exists (RT-07);

- on the second bounded call, disable tools and ground the answer solely in

the safe structured result (RT-09).

- `app/plan_finalization.py / scheduler` integration:
- make activation and rescheduling idempotent and observable; do not swallow

activation failure while callers report success (RT-11/RT-12);

- implement the existing pause/resume and cancel lifecycle fixes once, then

exercise them through tool-level integration tests (RT-13).

- tests: implement the outcome coverage enumerated in RT-15, including

invalid semantic time, plan-creation activation failure, paused time change, final-day status, and the full tool-result loop.

Scope boundary

This round audits all 8 Coach-callable runtime functions and the missing format-switch capability against their declarations. First-plan creation belongs to the deterministic onboarding flow and is tracked separately under ONB-07 / FSM-01. Lifecycle modes and persistence ownership are covered by the state audit below. The Bounded Tool-Result Loop architecture is owned by COACH-09; RT-09 only confirms the runtime tools depend on it.

Lifecycle State & Persistence Findings

Lifecycle State & Persistence Area — Audit Round 2026-07-22, consistency pass 2026-07-23

Status: technical pass completed and revalidated; persistence ownership resolved by FD-08; findings recorded only, no runtime fixes applied

Files and paths inspected

- `app/fsm/states.py`
- `app/fsm/guards.py`
- `app/fsm/__init__.py`
- `app/db.py`
- `app/orchestrator.py`
- `app/plan_runtime/tools.py`
- `app/plan_pause.py`
- `app/plan_finalization.py`
- `app/scheduler.py`
- `app/telegram.py`
- `app/session_memory.py`
- FSM, schedule-adjustment, runtime-tool, onboarding, completion, and Coach tests
- FSM-related SQL migrations in `migrations/`

Accepted target

Per `C_state`, FD-01, FD-04, FD-08, and the accepted pause/cancel contract, the target MVP lifecycle is:

```
incomplete onboarding -> current_mode=ONBOARDING
current plan active    -> current_mode=ACTIVE
current plan paused    -> current_mode=ACTIVE_PAUSED
no current plan        -> current_mode=NO_ACTIVE_PLAN

active plan -> completed plan + active same-format successor
active / paused plan -> abandoned plan -> NO_ACTIVE_PLAN
NO_ACTIVE_PLAN -> active plan (explicit new sequence)
```

These are derived product modes, not values stored in `users.current_state`. `IDLE_ONBOARDED`, `IDLE_FINISHED`, `IDLE_PLAN_ABORTED`, `IDLE_DROPPED`, and `SCHEDULE_ADJUSTMENT` are not target persisted user states. Automatic continuation recovery uses an idempotent outbox/reconciliation record, not an `IDLE_FINISHED` queue state. Plan format remains plan data, not a separate mode.

Summary

The three states accepted for removal are still represented across the ORM constraint, guards, runtime branches, tool entry conditions, scheduler, Telegram callbacks, Redis session memory, and tests. They are not equally dead:

- `IDLE_ONBOARDED` is absent from the current onboarding path and survives

mainly as the precondition of the dead `create_first_plan` wrapper;

- `IDLE_DROPPED` has an orphan writer with no production caller, but is still

accepted by follow-up creation and transition guards;

- `SCHEDULE_ADJUSTMENT` is a zombie subsystem: the old entry dispatcher has

no production caller, while callbacks, timeout recovery, session storage, and an active scheduler job remain.

`app/fsm/guards.py` is not an authority over any reachable lifecycle operation; it protects only the two dead paths tracked in FSM-03 and FSM-10 and should be removed with them. The live problem is different: the runtime has no coherent lifecycle aggregate. `User.current_state`, `UserProfile.is_paused`, `AIPlan.status`, and duplicated end-date fields can describe different realities for the same user. This has already allowed the pause, delivery, cancellation, and completion contracts to drift apart.

Findings

FSM-01 — `IDLE_ONBOARDED` and its only runtime wrapper are target-dead

Severity: P1 Status: confirmed; cross-reference ONB-07 / COACH-07

Current behavior:

- a new Telegram user is created directly in `ONBOARDING:START`;
- the current mock onboarding branch does not persist a transition to

`IDLE_ONBOARDED` or create a plan;

- the uncalled `create_first_plan()` wrapper rejects every state except

`IDLE_ONBOARDED`;

- `states.py`, `app/fsm/guards.py`, the ORM constraint, and tests still treat the

state as live.

The canonical `create_plan()` service already enforces first-plan SHORT. The legacy state/wrapper pair is therefore not a backend capability to preserve; the missing capability is the deterministic onboarding-completion transaction.

Expected behavior: deterministic onboarding saves the required setup and creates the first SHORT sequence idempotently, ending directly in `ACTIVE`.

Minimal fix: implement the accepted onboarding-completion transaction through `create_plan()` directly; then delete `create_first_plan()` and remove `IDLE_ONBOARDED` from state definitions, the dead guard matrix, tests, and the DB constraint. Do not expose first-plan creation to Coach.

FSM-02 — `IDLE_DROPPED` has no live entry path

Severity: P1 cleanup before beta Status: confirmed

`_auto_drop_plan_for_new_flow()` is the only runtime writer of `IDLE_DROPPED`, and it has no production or test caller.

Explicit cancellation uses `IDLE_PLAN_ABORTED`. Nevertheless, `IDLE_DROPPED` remains in `PLAN_CREATION_ENTRY_STATES`, `_FOLLOWUP_STATES`, end-state guards, the ORM constraint, migrations, and FSM tests.

Expected behavior: one user-visible stopped-sequence state, `IDLE_PLAN_ABORTED`, covers the post-cancellation return path. There is no background-drop behavior in the accepted MVP.

Minimal fix: remove the orphan helper and all `IDLE_DROPPED` allowances. In a forward migration, map any existing `IDLE_DROPPED` rows to `IDLE_PLAN_ABORTED` before replacing the constraint. Log the affected row count; do not silently discard plan records.

FSM-03 — `SCHEDULE_ADJUSTMENT` is a zombie subsystem, not one dead constant

Severity: P1 Status: confirmed; expands SCH-02

The old `run_plan_tool_call()` dispatcher has no production caller, so the normal target flow cannot enter `SCHEDULE_ADJUSTMENT`. Time changes now use the direct `change_day_time` and `change_evening_time` runtime tools.

However, the legacy subsystem still includes:

- transition constants and guard branches;
- four orchestrator handlers plus old keyboard builders;
- Telegram `sched_task:*`, `sched_time:*`, and timeout callbacks;
- Redis context, last-active, and soft-prompt keys/methods;
- the active `stuck_schedule_adj_check` scheduler job and hard-reset path;
- two dedicated test modules.

The DB definition is internally inconsistent as well: `app/db.py` permits `SCHEDULE_ADJUSTMENT`, but no checked SQL migration adds it. Its availability therefore depends on whether a database was created from ORM metadata or evolved through migrations.

Expected behavior: time changes are atomic deterministic tool operations and do not change FSM state.

Minimal fix, in safe order:

1. remove any persisted `stuck_schedule_adj_check` job and disable its registration;
2. inspect/reset any existing `SCHEDULE_ADJUSTMENT` rows to their recorded pre-tunnel `ACTIVE` or `ACTIVE_PAUSED` state before dropping the state;
3. remove the legacy dispatcher, handlers, callbacks, Redis methods, constants, and tests;
4. keep and test the direct time-change tools instead.

Do not add a migration that makes the obsolete tunnel valid merely to repair the current ORM/migration mismatch.

FSM-04 — `app/fsm/guards.py` protects only dead paths and should be removed

Severity: P1 Status: confirmed

`can_transition()` is reached only through:

- the legacy schedule-adjustment handlers (`_commit_fsm_transition`, FSM-03);
- `_guard_fsm_transition` for `transition_signal`, a field current

`coach_agent()` cannot emit (FSM-10).

Every reachable lifecycle operation — plan creation/finalization, pause/resume, cancellation, and completion — bypasses the matrix. Its current enforcement value is therefore zero.

The matrix also checks the wrong level of truth. The confirmed failures in FSM-06/08/09/13 are not illegal `from -> to` pairs; they are missing plans, field drift, stale scheduling data, and concurrent writes.

Expected behavior: deterministic lifecycle operations lock the relevant record(s), validate the whole lifecycle aggregate, persist the business change atomically, and log the result. This is an operation boundary, not a generic transition matrix or workflow engine.

Minimal fix: remove FSM-03 and FSM-10 first; then delete `can_transition`, `_commit_fsm_transition`, `_guard_fsm_transition`, and `app/fsm/guards.py`. Build only the operation-specific aggregate validation/locking selected under FSM-08. Deleting the matrix does not mean accepting invalid lifecycle data.

FSM-05 — the dead generic transition path fails open on invalid source state

Severity: P1 (must be resolved with onboarding) Status: confirmed

`is_valid_fsm_state("ONBOARDING:START")` returns `True`, and the DB constraint accepts `ONBOARDING:*`. `_normalize_fsm_state()` instead checks exact membership in `FSM_ALLOWED_STATES`, so every `ONBOARDING:*` value normalizes to `None`.

The downstream guard then fails open: `_guard_fsm_transition()` treats a missing or unrecognized normalized current state as permission to accept any valid target. Direct reproduction produced all of:

```
None -> ACTIVE
CORRUPT -> ACTIVE
ONBOARDING:START -> ACTIVE
```

The generic onboarding transition appears to work only because its current state was discarded, not because the transition matrix authorized it. Current Coach cannot emit `transition_signal`, so this is part of the dead mutation path rather than a mechanism to repair for production.

Minimal fix: delete `_normalize_fsm_state()` and `_guard_fsm_transition()` with FSM-10 instead of repairing them. When onboarding is implemented, test its direct deterministic completion transaction and invalid/NULL persisted setup data; do not route onboarding through a generic transition signal.

Founder note (2026-07-23): dead-path removal belongs to FSM-10. The real onboarding integration test belongs to ONB-01. This is not a reason to harden the obsolete generic guard now.

FSM-06 — a paused sequence can reach natural completion while still paused

Severity: P1 Status: confirmed; related to COACH-04

Pause changes the user/profile state but leaves the plan `active` and does not move `plan_end_date`. The completion cron selects active plans whose end date has passed without excluding `ACTIVE_PAUSED`, and `_auto_complete_plan_if_needed()` then writes `IDLE_FINISHED`. `app/fsm/guards.py` also explicitly permits `ACTIVE_PAUSED -> IDLE_FINISHED`.

This conflicts with the accepted pause meaning: future delivery stops and the remaining sequence is preserved for resume. A sufficiently long pause can instead finish the sequence in the background.

Minimal fix:

- on pause, clear `plan_end_date` and stop future scheduled jobs; pause duration

is unbounded, so there is no known offset to apply;

- keep an already-delivered current-day exercise actionable through its

existing expiry (DEL-04);

- exclude paused plans from completion candidates as defense-in-depth;
- on user-activated resume, re-anchor only the remaining future steps from the

next selected working day, schedule their jobs, and derive the new end date from the actual final re-anchored step;

- cancellation from pause remains valid; natural completion from pause does

not.

No guard-matrix edit is required because `app/fsm/guards.py` is removed under FSM-04.

FSM-07 — current tests and module docstrings preserve the obsolete architecture

Severity: P1 test replacement Status: confirmed

`tests/test_fsm_states.py` passes all 8 tests, but three of those tests explicitly assert the legacy `IDLE_ONBOARDED`, `IDLE_DROPPED`, and `SCHEDULE_ADJUSTMENT` transitions. The green suite therefore protects the old FSM rather than the accepted one.

The module docs do the same: `states.py`'s header declares "Active FSM (9 state groups)" and lists `IDLE_ONBOARDED`, `IDLE_DROPPED`, and `SCHEDULE_ADJUSTMENT` as live. `app/fsm/guards.py` also documents the zombie model, but that entire file is deleted under FSM-04 rather than rewritten. Anyone reading either module today is taught the dead architecture.

The dedicated schedule-adjustment tests are already stale against other accepted changes. With a syntactically valid test bot token and the unavailable Trio parametrization excluded, the targeted run produced 13 passed, 7 failed; the failures expect the removed user-facing `MORNING` slot and old tunnel behavior. The default test token also fails aiogram validation during collection.

Minimal fix: rewrite `states.py` documentation for the selected persisted or derived lifecycle model; replace matrix tests with operation-level aggregate invariant tests, direct time-tool tests, onboarding-to-first-plan integration, paused-completion prevention, concurrency cases, and a DB migration test. Remove schedule-adjustment tests with their subsystem and delete FSM guard tests with `app/fsm/guards.py`. This does not apply to `app/plan_guards.py`.

FSM-08 — lifecycle truth is duplicated across unsynchronized fields

Severity: P1 Status: confirmed; **resolved by FD-08** (2026-07-23) — plan-centric ownership

This finding diagnoses the problem (three parallel lifecycle sources). **FD-08** is the accepted decision that fixes it: `AIPlan.status` + onboarding progress own lifecycle truth; `current_mode` is derived by one function; `users.current_state`, `UserProfile.is_paused`, and `User.plan_end_date` are removed as duplicate authorities. See FD-08 for the hard dependencies and sequencing.

The same lifecycle is represented independently in several places:

- `User.current_state` gates Coach tools, scheduler delivery, and step actions;

- `UserProfile.is_paused` is described in code and migration comments as the

authoritative persistent pause flag, but the scheduler does not read it;

- `AIPlan.status` permits `active`, `completed`, `paused`, and `abandoned`;

helpers accept `paused`, but no production writer sets a plan to `paused`;

- `User.plan_end_date` and `AIPlan.end_date` duplicate completion timing, and

cancellation updates only the plan field, leaving the user field stale until a later completion check clears it.

The status vocabulary is also internally inconsistent: the unused Python `PlanStatus` enum defines `CANCELED`, while the ORM enum and runtime use `abandoned`. No cross-table constraint or startup reconciliation verifies that these values form one valid aggregate.

The guard-removal review reopened this decision; FD-08 then resolved it on 2026-07-23. The product still has lifecycle **modes**, but they are derived from onboarding progress plus the current plan rather than persisted independently.

Accepted FD-08 target:

- onboarding progress owns setup state;
- `AIPlan.status` owns plan lifecycle (`active`, `paused`, `completed`, `abandoned`);
- Coach/tool code receives a derived `current_mode` rather than treating `users.current_state` as a second source of truth;
- `UserProfile.is_paused` is removed;
- completion timing has one authoritative source derived from actual steps.

Do not ship the current hybrid. Implement the accepted plan-centric model through the FSM-11 migration and operation-level aggregate boundaries.

FSM-09 — state-only checks can manufacture valid-looking states without a plan

Severity: P1 Status: confirmed; expands RT-13

Pause, resume, and cancellation validate fragments of state rather than the whole lifecycle aggregate:

- `pause_plan` accepts `User.current_state == ACTIVE` without requiring an active plan;
- `resume_plan` accepts `ACTIVE_PAUSED` plus `profile.is_paused=True` without requiring a plan;
- `cancel_plan` accepts `ACTIVE/ACTIVE_PAUSED`, and if no plan is found still

commits `IDLE_PLAN_ABORTED` and returns success.

The inverse inconsistency is also possible: an active plan paired with an idle or paused-mirror mismatch is stranded because scheduler and Coach gating trust `current_state`. There is no repair or explicit failure mode for either direction.

Minimal fix: lifecycle operations must lock and load the user/setup data and exactly one current plan, validate the selected FSM-08 ownership model, then apply the operation atomically. On mismatch, fail honestly and emit reconciliation telemetry; never report a successful pause/resume/cancel for a plan that does not exist.

FSM-10 — the orchestrator retains an unreachable second mutation architecture

Severity: P1 cleanup before beta Status: confirmed

The current `coach_agent()` can return only text or a function tool call. Nevertheless, `handle_incoming_message()` still processes legacy generated `plan_object`, `plan_updates`, and `transition_signal` fields that the Coach cannot emit. Those branches can:

- abandon and replace an active plan using the old load/focus plan schema;
- apply plan adaptations that are disabled for MVP;

- directly rewrite `user.plan_end_date`;
- commit generic FSM transitions outside runtime tools.

Related preview/deep-link/action handlers remain in Telegram and orchestrator code even though the v5 plan service creates plans deterministically and `build_plan_draft_preview()` is frozen to return an empty string.

This is dead code today, but it preserves a conflicting lifecycle model and creates a dangerous accidental re-entry point if a future worker begins returning similarly named fields.

Minimal fix: remove the legacy envelope branches, persistence helpers, draft preview/deep-link/action flow, and their tests after confirming no external caller. The orchestrator's plan-changing surface should be deterministic onboarding, completion/continuation, and the allowlisted runtime tools only.

FSM-11 — database schema history has no authoritative FSM migration

Severity: P1 before beta deployment Status: confirmed

The ORM, SQL files, and deployed-schema bootstrap do not define one FSM:

- current SQL files variously permit `PLAN_FLOW:*`, `ADAPTATION_FLOW`,

adaptation substates, and confirmation states;

- the ORM permits a different set and adds `SCHEDULE_ADJUSTMENT`;
- no checked migration installs the ORM's current constraint;
- startup uses `Base.metadata.create_all()`, which does not replace constraints

on an existing database;

- `audit_startup_schema()` checks only three columns on `plan_instances`, not

FSM constraints or lifecycle enums;

- both `users.current_state` and `ai_plans.status` are nullable, so their CHECK

or enum declarations do not enforce a complete lifecycle value.

Consequently, behavior depends on whether a database was created from current ORM metadata or evolved through an undocumented subset/order of raw SQL files.

Minimal fix: implement FD-08 through one explicit forward migration that remaps legacy rows, removes `users.current_state` and `UserProfile.is_paused`, removes the duplicate `User.plan_end_date`, makes the retained lifecycle columns non-null, and normalizes plan-status values. Record applied migration/version state and add a startup assertion for the selected lifecycle contract. Do not rely on `create_all()` to upgrade an existing database.

FSM-12 — the database does not enforce one current plan per user

Severity: P1 Status: confirmed; cross-reference LIF-09 / LIF-11

`finalize_plan()` locks the user and checks for an active plan, which protects the canonical creation path. The database itself has no partial unique constraint for one current `AIPlan` per user, however, and legacy writers or race paths can still leave multiple current rows.

The runtime already anticipates this invalid state: completion logs a warning and completes only the newest active plan. Scheduler queries can meanwhile restore or deliver steps for every active plan belonging to the same `ACTIVE` user. One user-level FSM state cannot identify which plan is authoritative.

Minimal fix: clean any existing duplicates and add a PostgreSQL partial unique index covering every status that owns the current sequence (`active` and, under the preferred plan-centric model, `paused`). Retain the plan-scoped idempotency key required by LIF-11 for automatic continuation. Treat duplicate detection as an invariant breach, not a normal "choose latest" condition.

FSM-13 — lifecycle mutations are not serialized against each other

Severity: P1 Status: confirmed

Plan finalization uses row locks, but pause, resume, cancellation, and completion do not lock the user/current plan before reading and writing their state. Telegram messages, completion cron, and message-entry completion checks can run concurrently.

For example, cancellation and completion can both observe `ACTIVE`; one writes `abandoned/IDLE_PLAN_ABORTED`, while the other writes `completed/IDLE_FINISHED` and may initiate a report. Under ordinary read-committed transactions, the last commit wins individual fields without preserving the intended operation-level outcome.

Minimal fix: serialize lifecycle mutations by locking the current plan plus any retained user/setup row inside one short DB transaction, validate the complete operation preconditions, and commit lifecycle metadata together. External Telegram and scheduler side effects must remain outside the lock and use idempotent outbox/reconciliation records rather than extending the DB transaction.

Cross-domain FSM consequences already owned elsewhere

Do not create duplicate fixes for these; the consistency pass confirmed them:

- DEL-04 owns the rule that an already-delivered current-day exercise remains actionable through expiry while future delivery is paused;
- COACH-04 / RT-12 / RT-13 own pause/resume re-anchoring, end-date recomputation, and scheduler reconciliation;
- LIF-03 / LIF-04 / LIF-09 / LIF-11 own factual completion, same-format continuation, explicit plan identity, and continuation idempotency;
- DEL-04 owns cancellation of open steps and visible Telegram keyboards.

Verification evidence

- Current core state/tool/guard tests: 67 passed.
- The green tests assert the current architecture, including obsolete states;

they are not evidence of target-FSM correctness.

- Callback plus legacy schedule-adjustment revalidation:

14 passed, 15 failed, 23 deselected. Failures are stale step doubles and removed `MORNING/legacy tunnel` assumptions, consistent with DEL-06 and FSM-03/FSM-07.

- Direct guard reproduction confirmed that `NULL`, corrupt, and prefixed onboarding source states all currently pass through to `ACTIVE`.

Required MVP work and order

1. implement accepted FD-08 ownership: plan-centric persistence plus one derived `current_mode` function;
2. implement deterministic onboarding completion through `create_plan()` directly and delete the dead first-plan wrapper (ONB-07 / FSM-01);
3. prevent paused plans from completing and implement null-on-pause plus re-anchor/recompute-on-resume (COACH-04 / FSM-06);
4. disable the legacy schedule-adjustment scheduler job, inspect legacy rows, then remove the full tunnel (SCH-02 / FSM-03);
5. remove `IDLE_DROPPED` and merge any rows into `IDLE_PLAN_ABORTED` (FSM-02);
6. remove the unreachable Coach mutation/draft path, then delete `_guard_fsm_transition`, `_commit_fsm_transition`, and `app/fsm/guards.py` (FSM-04 / FSM-05 / FSM-10);
7. ship one forward migration for the accepted ownership model, non-null lifecycle fields, legacy-row remapping, and normalized plan statuses (FSM-11);
8. align `states.py` or the derived-mode builder, runtime tools/services, Coach tool filtering, and tests to that same lifecycle contract;

9. enforce aggregate existence, one current plan per user, operation-level

locking, idempotency, and telemetry without adding a workflow framework (FSM-09 / FSM-12 / FSM-13).

What is not a finding

`IDLE_NEW` and `IDLE_FINISHED` remain facts of the current implementation until FD-08 and FD-01 are implemented. They are not target persisted user states: incomplete setup derives `ONBOARDING`, while completion/continuation is an atomic lifecycle operation with durable outbox recovery.

Privacy & Data Boundaries Findings

Privacy & Data Boundaries Area — Audit Round 2026-07-30

Status: product boundary accepted under FD-09; code findings recorded only, no runtime fixes applied

Files and paths inspected

- `app/db.py`
- `app/telegram.py`
- `app/orchestrator.py`
- `app/session_memory.py`
- `app/workers/coach_agent.py`
- `app/ai.py`
- `app/api.py`
- `app/plan_completion/tokens.py`
- `app/telemetry.py`
- `app/plan_metrics.py`
- `resource/assets/product/conceptual_map.md`
- `resource/assets/product/conceptual_map_en.md`
- `docs/audit/product_contract.md`

External facts used

- The current Ukrainian Law "On Personal Data Protection" requires notice

about the controller, collected data, purpose, recipients, and data-subject rights at collection when data comes from the person (Article 12), and gives the person access to their data and a response/content right within 30 calendar days (Article 8):

<https://zakon.rada.gov.ua/laws/show/2297-17>.

- OpenAI states that API data is not used for model training unless the

customer opts in, while abuse-monitoring logs may contain prompts/responses and are retained for up to 30 days by default, subject to stated exceptions: <https://developers.openai.com/api/docs/guides/your-data>.

These facts define engineering requirements and disclosure inputs. They are not a substitute for legal review of the final Privacy Policy, processing basis, contracts, or cross-border data-transfer setup.

Accepted retention and access model

- raw user/Coach conversations: retain for **90 days**, then delete from

Postgres, Redis, and any application logs that duplicate content;

- user-linked product and behavioral history: retain while the account exists,

subject to the final Privacy Policy and deletion workflow;

- FD-18 roster and identity records: separate, purpose-limited access data

retained only for the active entitlement, commercial relationship, and a bounded security/audit need. Raw roster uploads have a shorter documented retention than normalized access records and never enter behavioral stores;

- independent aggregates produced under FD-09: may be retained indefinitely

and survive account deletion only after they are no longer linked or reasonably reconstructable as a person's history;

- user access/export/deletion may be handled manually during beta through a

real support path; a self-service privacy center is not required for MVP.

PRIV-01 — No user-facing privacy notice or recorded notice version

Severity: BLOCKER before beta Status: confirmed

No Privacy Policy, onboarding privacy notice, notice-version record, or acceptance/acknowledgement path exists in the inspected product code. The system nevertheless collects Telegram identifiers, profile/schedule data, conversation text, plan activity, and behavioral events, and sends conversation context to OpenAI.

FD-18 additionally requires production collection of an employer-supplied email roster and either minimum Google/Microsoft identity claims or a verified email challenge. These are access-control data, not behavioral telemetry, but they are still personal data and must be disclosed rather than hidden behind the word SSO.

Minimal fix:

- publish a concise Privacy Policy and onboarding notice that identify the controller/contact path, data categories, purposes, recipients/processors (including Telegram and OpenAI), retention periods, user rights, and whether any company-facing analytics exists;
- describe roster email/identity use as limited to authentication, entitlement, revocation, support, fraud prevention, and security; state that the employer receives no individual product behavior under FD-09;
- record which notice version and timestamp were shown/acknowledged before free-form Coach use;
- have counsel confirm the processing basis and any required processor / cross-border terms; do not describe a checkbox as solving every legal basis.

PRIV-02 — Raw conversations and content-bearing logs have indefinite retention

Severity: BLOCKER before beta Status: confirmed

Every incoming user message and several assistant reply paths write full text to `ChatHistory(telegram.py)`. `get_stm_history()` reads it as the Postgres fallback, so the table is live, not legacy. There is no age filter or cleanup job.

Redis keeps the last 20 messages with `RPUSH + LTRIM`, but the message key has no `EXPIRE`; the 1-2 hour TTLs elsewhere in `session_memory.py` belong to pending actions and schedule-adjustment state, not chat history.

`coach_agent.py` also logs the first 500 characters of every model response. That can create a third content copy with a separate, undefined log-retention policy.

Minimal fix:

- add a scheduled, observable 90-day deletion job for `ChatHistory`;
- make the Postgres fallback query exclude rows outside the retention window;
- set an explicit expiry on Redis message keys and delete the key during account deletion;
- stop logging response bodies in production, or redact/structure them so conversational content is not copied into logs;
- define and enforce application-log retention separately.

PRIV-03 — OpenAI storage controls and payload minimization are not explicit

Severity: P1 before beta Status: confirmed

The Responses API calls in `app/ai.py` and `coach_agent.py` do not explicitly set `store=False`. The Coach sends the current message, up to 20 recent messages, Product Map, prompt, and bounded runtime facts. Direct Telegram identity fields are not currently serialized by `_compose_messages():user_id` is used locally to build context but is omitted from `_context_message()`. This is a good current property but is not enforced by a contract test.

Minimal fix:

- set `store=False` on every compatible OpenAI call and audit all AI entry

points, not only Coach;

- keep names, Telegram IDs/usernames, email, employer identity, and other

direct identifiers out of model input unless a specific product need is accepted;

- add a test that inspects the final request payload and fails if direct

identity fields leak;

- disclose OpenAI processing accurately: no training by default is not the

same promise as zero provider retention.

PRIV-04 — No access, export, correction, or deletion request workflow exists

Severity: BLOCKER before beta Status: confirmed

No product command, API, support runbook, or tested service handles a request to access, export, correct, or delete the user's data. The statutory access/content deadline cannot be operationalized by a Privacy Policy alone.

Minimal beta fix:

- provide one real support contact and identity-verification procedure;
- document a manual export format covering profile/schedule, plans, exercise

events, conversations still inside retention, and feedback;

- document correction and deletion handling, responsible owner, request log,

and deadline tracking;

- do not claim that a request was received or completed unless a real path

records it.

PRIV-05 — Account deletion is not one tested data operation

Severity: P1 before beta Status: confirmed

There is no account-deletion service or endpoint. Several ORM relationships have delete-orphan cascade, but `TaskStats` and `FailureSignal` reference users without corresponding `User` relationships/cascade ownership. Redis session keys, scheduler jobs, report access, and content-bearing logs are outside ORM cascade entirely. Deleting a `User` directly is therefore not a deletion contract and may fail, leave dependent data, or leave live access paths.

Minimal fix:

- implement one idempotent deletion service and manual beta runbook;
- stop delivery and scheduler jobs, invalidate report access, delete Redis

keys, then remove all user-linked database rows in a defined transaction / reconciliation sequence;

- preserve only aggregates already made independent under FD-09;
- add an integration test that seeds every user-linked table and external key,

deletes the account, and proves no user-level record or access path remains.

PRIV-06 — Internal telemetry is personal data, not anonymous analytics

Severity: P1 contract enforcement before beta Status: confirmed; collection accepted under FD-09

`UserEvent`, `TaskStats`, `FailureSignal`, plan rows, and completion metrics are linked to `user_id` or a user-owned plan execution. They can support product operation and learning, but they remain user-level data even if the UI omits a name. A stable pseudonym would still be linkable and would not make the longitudinal history anonymous.

Expected boundary:

- user-linked telemetry is product-internal and disclosed to the user;
- access is least-privilege and never available through a company/HR surface;
- raw Coach text and free-form feedback are not analytics dimensions;

- company-facing reporting, if built later, reads only the independent

aggregate layer under FD-09.

PRIV-07 — Independent idempotent aggregation is not implemented

Severity: P1 data architecture before beta Status: confirmed; target accepted under FD-09

Current telemetry writes only user-linked rows and computes metrics from them. No independent aggregate table, aggregation ledger, idempotency key, or outbox exists. Consequently, deleting personal events either destroys all historical product learning or tempts the system to retain linkable data indefinitely.

Minimal fix:

- give each accepted behavioral event one stable identity;
- in the same reliable operation, persist the personal event and its bounded

aggregate contribution, with retry-safe uniqueness;

- store aggregate counts/buckets without user identifiers, stable hashes,

exact user timelines, or raw text;

- treat any event-ID/idempotency ledger as operational data with bounded

retention; do not copy that join key into indefinite aggregate counters;

- include completion/skip/ignore, response-latency buckets, streak buckets,

plan completion, activation, and D3/D7 where definitions are stable;

- reconcile failed aggregate writes observably; never rebuild them during

account deletion as an ad hoc migration.

Three-layer clarification (2026-07-30): dual-write is right, but **not every counter without a `user_id` is anonymous**. A cell like `company_id + day + exercise + count=1` has no identifier yet deanonymizes trivially by side knowledge. Distinguish three layers:

1. **User-linked events** — personal, retained until account deletion.
2. **Restricted operational aggregates** — company/time/exercise buckets that

may still be small (low-N). These are **not** company-facing and **not** treated as anonymous; they follow the same bounded retention and deletion as personal data.

3. **Sealed aggregates** — have passed the cohort/cell threshold, contain no

join keys, and cannot be reversed to individuals. **Only these** may be retained indefinitely and may ever appear in a report.

So the founder's "aggregate survives deletion" applies **only to sealed aggregates**, not to every counter. The dual-write persists to the restricted layer; sealing is a separate gated step.

Related telemetry ceiling (2026-07-30): the bot cannot measure exercise **execution** or **read** — Telegram Bot API has no read receipts for bot-sent messages, and a `Виконано` tap does not prove the exercise was done. So `ignored` means only "no registered response," not "did not recover." The chain *no tap* → *not resting* → *overloaded* → *risk* has three unmeasured jumps. "Read" and real "duration" become measurable only inside a Mini App the user opens (post-beta) — do not design metrics that assume either.

PRIV-08 — Completion and pulse report bearer links do not expire or revoke

Severity: P1 before beta Status: confirmed

`make_report_token()` signs only `plan_id`. `verify_report_token()` validates that signature with no issued-at time, expiry, token version, or revocation state. The same token is accepted by report/pulse routes. A copied URL therefore remains usable indefinitely, including after chat retention expires or an account is deleted, unless the plan/database disappears.

Minimal fix:

- define a bounded report-link lifetime;
- include expiry and token version in the signed payload, or store a

revocable access grant;

- invalidate access on account deletion and secret/token rotation;
- add expired, revoked, deleted-user, tampered, and valid-token tests;
- do not put additional personal data into the token itself.

PRIV-09 — Dead sensitive-looking schema expands the risk surface

Severity: P2 cleanup; complete during the database audit Status: confirmed

`FactCategory.MEDICAL / UserFact` and `UserDailyLog.stress_level, energy_level, and mood_note` exist in the ORM, but no production write path was found. These fields are not harmless documentation: they imply planned storage of health/mental-state information, expand future misuse risk, and make the real data inventory harder to explain.

Minimal fix: confirm the tables/columns are unused in deployed data, then remove them through the database migration rather than preserving speculative "memory" or mood-tracking schema. Reintroduce such data only after a concrete use case, legal basis, retention rule, and user-facing contract exist.

PRIV-10 — `/user/time-slots` can mutate another user's schedule without authorization

Severity: BLOCKER before beta Status: confirmed

`POST /user/time-slots` accepts a caller-supplied internal `user_id`, performs no authentication or authorization, updates that user's saved slots and active steps, and then reschedules jobs. It also reveals whether an internal user ID exists through its 404 behavior. This is a direct integrity and privacy boundary failure, independent of future company analytics.

Minimal fix: if the endpoint is legacy/not required by MVP, remove it. If it must remain, authenticate the caller, derive the user identity from a verified session or signed Telegram WebApp payload rather than a query parameter, authorize the operation, rate-limit it, and add cross-user denial tests.

Confirmed safe current property

No company model, HR dashboard, company-facing analytics endpoint, or user-level reporting API was found. Preserve the absence of live and user-level company views. The only accepted exception is FD-09's single gated post-pilot aggregate product-usage summary, built from sealed aggregates rather than an ad hoc query over user-linked rows.

Required MVP work and order

1. publish the privacy notice and establish the manual rights-request path
(PRIV-01 / PRIV-04);
2. remove or secure the unauthenticated time-slot endpoint (PRIV-10);
3. enforce 90-day conversation retention across Postgres, Redis, and logs
(PRIV-02);
4. add `OpenAI store=False`, request-payload minimization tests, and accurate disclosure (PRIV-03);
5. implement the idempotent personal-event + independent-aggregate operation before beta telemetry begins (PRIV-06 / PRIV-07 / FD-09);
6. implement and test account deletion across DB, Redis, scheduler, logs, and report access (PRIV-05);
7. add bounded, revocable report tokens (PRIV-08);
8. remove unused sensitive-looking schema during the database migration
(PRIV-09);
9. keep live, sliced, invitation-level, and user-level company reporting disabled; implement FD-09's centralized gate before producing the one accepted post-pilot aggregate summary.

What this round does not decide

The mere presence of a free-text Coach does not by itself prove that Love Yourself systematically processes health, violence, or other special-risk categories. The product does not solicit those categories for analytics and should not add classification or storage for them by implication. Reassess regulatory notification and special-category requirements if later features deliberately collect, classify, or operationalize that information.

Content Library Findings

Content Library Area — Audit Round 2026-08-06

Status: audit complete; target accepted under FD-10; implementation deferred

Files and paths inspected

- resource/assets/content_library/tasks/burnout_combined_content_library.json
- resource/assets/product/product_internal_spec.md
- docs/audit/product_contract.md
- app/content_library.py
- app/db.py (ContentLibrary, AIPlanStep, UserEvent, TaskStats,

FailureSignal)

- app/plan_drafts/plan_builder_v5.py
- app/ux/task_notification.py
- app/telemetry.py
- plan-generation and delivery-renderer findings already recorded above
- supplied evidence reviews and Fogg/behavior-design material as supporting

context, not as final external expert approval

Accepted exercise catalogue

The following is the canonical target catalogue. IDs are stable and do not contain a version suffix; instruction changes increment `content_version`. Durations are explicit user-facing execution contracts, not clinical doses or claims that a particular duration guarantees an effect.

exercise_id	User-facing title and instructions	Duration	mechanic	modality
breathing_sigh	Дихання Повільно вдихни носом. Не видихаючи, зроби ще один короткий вдих. Повільно видихни ротом до кінця. Повтори ще двічі.	30 sec	switch	breathing
pmr_fist	Кулак Стисни один кулак із помірною силою на 5 секунд. На видиху повністю розтисни й зачекай 5 секунд. Повтори ще двічі.	30 sec	switch	muscle
tactile_surface	Дотик Поклади долоню на поверхню поруч. Зверни увагу на її температуру. Потім — на текстуру.	20 sec	switch	tactile
visual_distance	Погляд вдалину Відведи погляд від екрана. Знайди найдавший об'єкт, який бачиш. Дай очам сфокусуватися на ньому.	20 sec	switch	visual
auditory_sound	Один звук Знайди один фоновий звук, який уже чуєш. Залиш увагу на ньому. Поміть, чи він рівний, чи змінюється.	20 sec	switch	auditory
cold_water_face	Холодна вода Набери в долоні прохолодну воду. Затримай дихання й занур обличчя у воду на 2–3 секунди. Підними голову й повернися до звичайного дихання. Повтори ще двічі.	15 sec	switch	thermal
brain_dump	Brain Dump Напиши сюди або в нотатку одну річ, яка зараз крутиться в голові. Без структури. Як є.	60 sec	unload	writing
one_thing	Одна річ Напиши сюди або в нотатку одну річ, яка сьогодні вдалася — будь-яку. Зав'язати шнурки теж підходить.	30 sec	unload	writing
first_step_tomorrow	Перший крок завтра Напиши сюди або в нотатку перший конкретний крок, з якого почнеш наступний робочий день.	60 sec	unload	writing

User-facing duration labels are flat and exact (Тривалість: 20 секунд, Тривалість: 1 хвилина). Do not add орієнтовно, приблизно, or комфортно. For sensory exercises, duration is displayed separately from the steps; do not turn it into a counting instruction unless counting is itself the protocol.

Accepted requirements matrix

Requirements describe execution constraints, not user traits. They may later support explicit eligibility/preferences, but the system must not infer a disability, diagnosis, or preference from completion behavior.

exercise_id	capabilities	environment	friction
breathing_sigh	breath_control	—	—
pmr_fist	hand_use	—	—
tactile_surface	hand_use	touchable_surface	—
visual_distance	vision	distant_focus_target	—
auditory_sound	hearing	audible_sound	headphones_may_interfere
cold_water_face	hand_use, breath_hold, facial_water_contact	sink, water	leave_desk, shared_resource, socially_visible
brain_dump	text_entry	chat_or_note	written_response
one_thing	text_entry	chat_or_note	written_response
first_step_tomorrow	text_entry	chat_or_note	written_response

The attention shift requested by every `switch` exercise is part of the global content contract, not a discriminating per-exercise requirement.

Target record shape

```
{
  "id": "stable_exercise_id",
  "content_version": 1,
  "is_active": true,
  "mechanic": "switch",
  "modality": "tactile",
  "requirements": {
    "capabilities": [],
    "environment": [],
    "friction": []
  },
  "duration_seconds": 20,
  "cooldown_days": 1,
  "review_required": false,
  "review_status": "unreviewed",
  "display": {
    "title": "...",
    "steps": ["..."],
    "duration_label": "20 секунд"
  }
}
```

Common initial values: `content_version=1`, `is_active=true`, and `cooldown_days=1`. Cooldown remains a beta variety hypothesis, not a claimed therapeutic interval.

Remove rather than repurpose:

```
variations
weight
duration_minutes
extended_minutes
category
difficulty
energy_cost
logic_tags
slot / hardlock / office / remote legacy classification
```

`mechanic`, `modality`, and `structured requirements` replace the overloaded legacy tags. No category or requirement is user-facing.

Findings

CONTENT-01 — Current library structure encodes the abandoned model

Severity: P1 migration before beta Status: confirmed; target accepted under FD-10

The current JSON contains eight parent exercises plus nested runtime variations, minute-based durations, weight, and legacy classification. The builder ignores nested variations, uses `weight`, and routes only by `mechanic`; the ORM/loader preserve

required `category`, `difficulty`, `energy_cost`, and `logic_tags` fields that no longer express product decisions. Keeping this shape would make the target catalogue appear implemented while runtime still selects the old parents.

Required implementation: replace the library as one versioned migration, update loader validation and builder selection, and remove the dead fields from JSON, ORM, database, documentation, and tests. Do not layer the new axes into `logic_tags` as another compatibility blob.

CONTENT-02 — Exact instructions and durations are now a versioned contract

Severity: P1 editorial/data contract before beta Status: accepted

The canonical table above replaces parent/variation copy. The text is short, imperative, self-contained, and does not prescribe an emotion or promise what the user will feel. The renderer must read the exact selected content version; content edits increment `content_version` and must remain distinguishable in telemetry.

`Brain Dump`, `Одна річ`, and `Перший крок завтра` are accepted beta `unload` hypotheses. In particular, the first-step action may unload an open loop or may restart work planning; wording alone cannot settle that question. Measure it rather than adding a rationale sentence that claims the outcome.

CONTENT-03 — The 10/20/30-second sensory dose is not evidence-derived

Severity: founder decision / beta calibration Status: accepted initial protocol; re-evaluate with evidence

No reviewed evidence establishes that tactile, visual, or auditory attention requires exactly 10, 20, or 30 seconds to "switch focus." Experimental orienting can occur much faster, but that is not equivalent to a useful workday pause, subjective relief, or restored performance. Microbreak studies use heterogeneous protocols, and the familiar 20-second visual-break rule does not establish a universal cognitive threshold.

Twenty seconds is accepted as the shared initial dose for touch, distant gaze, and sound because it is a reasonable friction/effect midpoint, not because it is scientifically optimal. Calibrate with timed usability observation and versioned beta comparisons. Telegram delivery-to-tap latency must never be used as measured execution duration.

CONTENT-04 — Requirements need first-class structure, but must not become profiling

Severity: P1 schema requirement; eligibility use may remain post-MVP Status: accepted

The exercises do not have identical physical and environmental requirements. Hand use, vision, hearing, a distant target, headphones, text entry, and sink access materially affect feasibility. Recording the requirements matrix makes these constraints inspectable and prevents future code from hiding them in labels such as `office`, `remote`, or a guessed user persona.

For MVP the metadata does not authorize inferred personalization. A skip or slow response cannot be interpreted as inability, disability, aversion, or a mental-state signal. Later exclusion/preferences must be explicit and user-controlled.

CONTENT-05 — Review state must enforce a real release gate

Severity: P1 for cool-water activation; external review before broad claims Status: accepted; reviews outstanding

A decorative `safety_status` is insufficient. All exercises start `review_status=unreviewed`. `review_required` is the runtime gate:

```
eligible = is_active
          AND (not review_required OR review_status == approved)
```

Only `cold_water_face` starts with `review_required=true`; therefore it is present in the target catalogue but cannot be selected or delivered until approved. Obtain a short review from a qualified clinician with relevant cardiovascular/arrhythmia or sports-medicine competence, covering cool (not ice) water, facial immersion, breath-hold wording, repetitions, contraindications, and the corporate self-guided context.

The remaining exercises have `review_required=false` and may enter a limited beta while `unreviewed`. Before broad market claims, seek an external review from a behavioral scientist/psychologist experienced in brief workplace interventions, behavior design, and evidence appraisal. Until then, claim only the intended `switch/unload` mechanism and product rhythm — not treatment, stress reduction, focus recovery, nervous-system regulation, or company performance effects as guaranteed outcomes.

CONTENT-06 — Visual assets are optional versioned delivery aids

Severity: P2 delivery experiment, not an MVP content blocker Status: accepted boundary; asset design deferred

GIFs or short visuals may demonstrate the non-obvious motion or make a trivial action easier to enter, especially breathing, fist tension/release, and the walk-to-sink context around cool water. They must not carry instructions that are absent from text, act as a synchronized timer, imply a guaranteed internal effect, or make the exercise unusable when media fails.

If introduced, store a versioned asset reference and accessibility text with the content version, and record the renderer/delivery variant in telemetry. Exact message hierarchy, notification preview, media placement, buttons, fallback, and Telegram/Slack/app behavior belong to the separate Delivery UX / Exercise Presentation audit.

CONTENT-07 — `UserEvent.step_id` mixes plan-step and exercise identity

Severity: P1 data integrity before beta telemetry Status: confirmed; target accepted, implementation not yet verified

`UserEvent.step_id` is a `Text` field with a foreign key to `content_library.id`, but current task paths pass integer `AIPlanStep.id`. `log_user_event()` resolves either a content ID or a plan-step ID into the same value and may manufacture an inactive `ContentLibrary` stub for a numeric plan step. Event-to-plan joins require guarded casts; exercise-level `TaskStats` and `FailureSignal` can be keyed by a unique occurrence; future category analysis can therefore look precise while joining the wrong entity.

Accepted target:

- `user_events.plan_step_id`: nullable integer foreign key to `ai_plan_steps.id` — the scheduled occurrence;
- `user_events.exercise_id`: nullable text foreign key to `content_library.id` — the reusable exercise;
- `content_version`: the exact instructions delivered;
- later delivery experiments use an explicit renderer/delivery variant.

Migration requirements:

1. add both columns/indexes without changing user-facing behavior;
2. backfill numeric values only when they match a real plan step, then derive the exercise from `AIPlanStep.exercise_id`;
3. recover exercise IDs only from validated context or genuine legacy content IDs;
4. move all new writes and joins to explicit columns;
5. keep old nullable `step_id` only for a bounded compatibility window;
6. add migration, completion/skip/ignore, metrics, and legacy backfill tests.

A local uncommitted implementation draft was accidentally created during the audit. It is not an accepted production fix and must remain uncommitted until the migration and behavior-preservation tests are reviewed.

CONTENT-08 — Product contracts and specifications still describe the old library

Severity: P1 contract synchronization with implementation Status: confirmed

`product_contract.md` still states eight exercises and five `switch` exercises, and preserves `variants[]/legacy` slot metadata. The internal product specification still describes old parent IDs, minute durations, micro-walking, the removed thought exercise, and the legacy schema. The JSON, database model, loader, tests, Product Map references, and documentation must be updated together when FD-10 is implemented. Until then, the audit decision is the target and current runtime remains explicitly legacy.

CONTENT-09 — Exercise presentation requires its own UX audit

Severity: P1 product-touchpoint design before beta Status: founder accepted as a separate upcoming audit stage

The earlier Delivery Renderer audit established technical correctness and the basic Done/Skip lifecycle, but did not lock the full user-facing presentation: title/duration/steps hierarchy, Telegram notification preview, button labels, GIF placement and fallback, media failure, accessibility copy, or portability to Slack/a future app.

Content Library owns **what** the renderer receives: title, exact steps, duration, and optional versioned asset/alt text. Delivery UX owns **how** those fields become a channel interaction. Complete that audit before implementing the final renderer/content migration so neither side silently dictates the other.

Required implementation work and order

1. complete Delivery UX / Exercise Presentation audit;
2. complete the required cool-water medical review or keep it gated out;
3. design one forward content/DB migration from the legacy library;
4. replace JSON and loader validation with the target record shape;
5. update builder selection to independent exercises and enforce the review gate; remove variations/weight/legacy metadata;
6. update renderer to consume exact `display` and optional asset metadata;
7. normalize event identity and record `exercise_id`, `plan_step_id`, `content_version`, and delivery variant;
8. implement the completed-only `better/same/worse` feedback path under FD-05/FD-07 without changing plan generation;
9. synchronize Product Map, product contract, internal specification, and tests;
10. run schema, migration, generation, renderer, callback, review-gate, and telemetry tests before enabling the new catalogue.

Explicitly deferred

- behavior-based personalization or plan adaptation;
- exercise levels/unlocks;
- runtime variations;
- expanding the catalogue to rescue retention before beta evidence;
- Mini App timer or measured execution duration;
- inferred accessibility profiles;
- clinical, neurological, productivity, or company-ROI claims without appropriate evidence review.

Telemetry Area — Audit Round 2026-08-08

Files and paths inspected

- `app/telemetry.py`
- `app/db.py`
- `app/telegram.py`
- `app/scheduler.py`
- `app/orchestrator.py`
- `app/plan_runtime/tools.py`
- `app/plan_pause.py`
- `app/plan_finalization.py`
- `app/plan_completion/metrics.py`

- `app/plan_metrics.py`
- `app/api.py`
- telemetry, task-lifecycle, completion, dashboard, and orchestrator tests
- `docs/audit/product_contract.md`
- FD-05, FD-07, FD-08, FD-09, FD-10, PRIV-06/07, and CONTENT-07

The code inspection used committed HEAD (`ac2711d`) because a local uncommitted draft already changes event identity fields. That draft is not treated as implemented or verified.

Audit objective

Define the minimum trustworthy event contract needed to validate corporate deployment/activation (C1) and continued rhythm (C3) from the [Riskiest Assumptions → MVP Test Map](#), then compare it with the current runtime. The audit deliberately separates:

- operational facts the application can observe;
- proxies such as a registered completion tap;
- product interpretations such as continued rhythm;
- health, psychological, and company-performance claims the product cannot

infer from interaction telemetry.

Target event envelope

Every persisted event must use the same explicit envelope:

<code>event_id</code>	UUID generated once per source operation
<code>event_name</code>	versioned, allow-listed name
<code>event_schema_version</code>	integer
<code>occurred_at</code>	timestamp of the real operation
<code>recorded_at</code>	ingestion timestamp
<code>source_operation_id</code>	stable idempotency key
<code>user_id</code>	nullable only for pre-user deployment facts
<code>organization_id</code>	nullable internal organization reference
<code>deployment_id</code>	nullable opaque launch/cohort reference
<code>plan_id</code>	nullable FK to <code>ai_plans</code>
<code>plan_cycle_number</code>	nullable integer
<code>plan_step_id</code>	nullable FK to <code>ai_plan_steps</code>
<code>exercise_id</code>	nullable FK to <code>content_library</code>
<code>content_version</code>	nullable integer
<code>delivery_variant</code>	nullable versioned renderer/channel variant
<code>properties</code>	allow-listed event-specific fields only

`source_operation_id` is unique within an event source. Examples are Telegram `update_id/callback_query.id`, scheduler job plus step plus event name, runtime tool execution ID, and plan/cycle plus lifecycle event. A retry reuses the key; it never manufactures another product event.

Free text, names, Telegram handles, Coach content, brain-dump content, inferred mood, diagnosis, or skip reason are not valid `properties`. Raw conversation and feedback content remain in their separately retained personal stores.

Target event catalogue

Event	Exact occurrence	Required linkage
bot_started	Telegram delivers a /start update	deployment_id when present, user
roster_import_applied	a validated and explicitly confirmed roster version atomically becomes current	organization, deployment, roster version, explicit import mode, add/keep/revoke counts; no email in event properties
access_entitlement_granted / access_entitlement_extended / access_entitlement_revoked	the access-control operation commits the new entitlement boundary	deployment, entitlement, source operation; no behavior linkage in event properties
invitation_issued	roster-gated corporate authentication or testnet founder issuance creates one expiring Telegram handoff token	deployment, entitlement, invitation operation ID; no raw token
deployment_enrollment_created	a valid invitation redemption atomically creates one deployment enrollment	user, deployment, enrollment; invitation operation ID retained only inside the bounded idempotency boundary
onboarding_started	deterministic onboarding first screen is accepted for processing	user, deployment
onboarding_completed	all required setup is durably committed	user, deployment
plan_created	target AIPlan and steps commit	user, plan, cycle
plan_scheduling_succeeded	all required delivery jobs are durably registered	user, plan, cycle
task_delivered	Telegram confirms the exercise message was sent	plan, step, exercise, content/renderer version
task_delivery_failed	a delivery attempt reaches a terminal failure for that attempt	same as delivery plus bounded error class
task_completed	valid Done callback commits the step as completed	same as delivery plus callback operation ID
task_skipped	valid Skip callback commits the step as skipped	same as delivery plus callback operation ID
task_expired	the real action deadline commits an unanswered step as expired	same as delivery
plan_paused / plan_resumed	the runtime action commits the new plan status	user, plan, tool execution ID
plan_cancelled	cancellation and step closure commit	user, plan, tool execution ID
delivery_time_changed	the new time commits; reschedule outcome is recorded separately	user, plan if active, tool execution ID
plan_format_switched	an atomic active-format switch commits	user, old/new plan, tool execution ID
plan_completed	cycle finalization commits	user, plan, cycle
completion_report_sent	Telegram confirms report delivery	user, plan, cycle
next_cycle_created	automatic same-format next cycle commits	user, prior/new plan, cycle
user_message_received	a non-command inbound message is durably accepted	user only; no semantic claim
feedback_submitted	an explicit FD-07 feedback action commits	source type plus allowed target IDs

The product may use more operational events later, but no metric may depend on an undocumented event or an ad-hoc JSON key.

Metric definitions and denominators

Deployment funnel

Production `eligible_count_at_launch` is the count of active access-eligible records in the accepted launch roster version. Testnet is excluded. The company still confirms the intended launch audience and announcement time, because a roster proves eligibility but not that employees saw the enrollment message.

```
SSO authorization rate = distinct invitation_issued entitlements
                        / eligible_count_at_launch

deployment start rate = distinct deployment_enrollment_created users
                      / eligible_count_at_launch

onboarding conversion = distinct onboarding_completed users
                      / distinct deployment_enrollment_created users
```

Successful roster/SSO authorization and Telegram Start/redemption are separate observable steps. Opening the issued Telegram invitation without pressing Start remains unobservable through the bot and must not be reported as enrollment.

Product activation funnel

Report each stage, not one blended percentage:

```
onboarding_completed
→ plan_created
→ plan_scheduling_succeeded
→ first task delivered
→ first explicit response (task_completed or task_skipped)
→ first registered task_completed
```

first-completion activation uses users with a successfully delivered first task as its immediate denominator.

Deployment activation remains a separate eligible-population metric so channel failure cannot disappear from the numbers.

Working-day retention and rhythm

D1, D3, and D7 mean the first, third, and seventh configured **plan working day**, not calendar days and not rolling 24/72/168-hour windows.

```
Dn response retention = activated users with task_completed or task_skipped
                        on plan working day N
                        / activated cohort whose day-N action window elapsed

Dn completion retention = activated users with task_completed on day N
                        / the same cohort denominator
```

Pause and cancellation remain visible outcomes in that denominator; delivery failures are reported separately and must not be misclassified as user silence. Always show numerator, denominator, raw n , cohort start, configured work-day rule, and observation cutoff.

For each completed cycle also report:

```
completed-day coverage = plan working days with >=1 registered completion
                        / elapsed eligible plan working days

task completion rate = task_completed
                      / successfully delivered, non-cancelled tasks whose
                        action window ended

explicit skip rate = task_skipped / the same task denominator
silent expiry rate = task_expired / the same task denominator

next-cycle response retention = users with an explicit response in the
                              automatic next cycle
                              / users whose prior cycle completed and whose
                                next-cycle first action window elapsed
```

Day coverage and task completion are separate because the 14-day format can deliver two actions per day.

Content and reactive diagnostics

Per-exercise results require a `task_delivered` exposure denominator and are grouped by `exercise_id`, `content_version`, and delivery variant. Tiny beta samples are shown as counts, not ranked as winners and losers.

`user_message_received` measures Coach traffic only. It is not self-started exercise use, need, distress, or value. A future `start exercise now` feature must emit its own explicit event before self-initiated use becomes a metric. Response latency measures time from confirmed delivery to callback; it is not exercise duration or proof that the exercise happened.

Inference guardrails

Telemetry must never turn the following proxies into stronger claims:

Observed fact	Allowed interpretation	Forbidden interpretation
<code>task_completed</code>	user registered completion	exercise was performed correctly or helped
<code>task_skipped</code>	user explicitly skipped	exercise was bad, inaccessible, or user lacked motivation
<code>task_expired</code>	no registered action before deadline	user ignored the bot, was overloaded, or received no value
fast callback	quick registered response	exercise was short, easy, or effective
repeated scheduled response	product rhythm continued	habit is autonomous or user is dependent
Coach message	user contacted the Coach	user needed emotional support or started an exercise
company aggregate change	interaction pattern changed	burnout, health, or productivity changed

Findings

TEL-01 — The deployment and activation funnel is not instrumented

Severity: experiment-validity BLOCKER before beta Status: confirmed

`/start` creates or retrieves a user but emits no `bot_started`, deployment, onboarding-start, or onboarding-completed events. There is no organization or deployment attribution model and no eligible-population denominator. The code can therefore calculate neither cold-deployment adoption nor onboarding drop-off. `/start → first completed` would also hide everyone who never pressed Start.

Required implementation: create deployment, roster version, restricted access identity, entitlement, and FD-18 invitation records; derive `eligible_count_at_launch` from the accepted roster and store announcement confirmation; instrument roster/SSO authorization and invitation issuance; redeem the opaque `?start=<invitation_token>` into one attributed enrollment; and instrument deterministic onboarding stages. Testnet uses the same events under an isolated testnet deployment and is excluded from production metrics. The native Telegram Start button remains the user action; the visible command message may be deleted after the event is durably captured if the UX audit chooses that presentation.

TEL-02 — D1/D3/D7 and completion metrics are not one stable contract

Severity: experiment-validity BLOCKER before beta Status: confirmed

The product contract names D3/D7 without defining calendar versus working-day semantics, activity event, denominator, pause/cancel handling, or observation cutoff. It describes first-plan completion relative to active days, while runtime completion code calculates completed task steps over terminal eligible steps. Those definitions diverge for the two-action 14-day format.

Required implementation: adopt the definitions in this round, centralize the queries, and test weekends, custom work days, pause, cancellation, failed delivery, one-action and two-action formats, and incomplete observation windows.

TEL-03 — Events are detached from the authoritative plan aggregate

Severity: data-integrity BLOCKER before beta Status: confirmed; overlaps DB audit, FD-08, and CONTENT-07

Current telemetry auto-creates/reuses legacy `PlanInstance` and an open `PlanExecutionWindow` instead of linking directly to the actual `AIPlan` cycle. The window is not reliably closed. `UserEvent.step_id` mixes scheduled step and exercise identity and can manufacture fake inactive content rows. A `plan_activated` event stores `plan_id` in a content foreign-key field.

Required implementation: use the FD-08 plan aggregate and explicit target linkage from the event envelope. Complete and verify the CONTENT-07 migration; remove the legacy instance/window compatibility layer after backfill.

TEL-04 — Event ingestion is not idempotent

Severity: data-integrity BLOCKER before beta Status: confirmed; overlaps DB audit and PRIV-07

Each call generates a new random UUID and has no source-operation uniqueness. Telegram retries, scheduler retries, completion retries, or repeated tool execution can produce duplicate events, counters, and future aggregate contributions.

Required implementation: enforce a unique source operation key, make the user-linked event and independent aggregate contribution one idempotent transaction, and add duplicate-update/retry tests.

TEL-05 — `task_ignored` does not represent the real expiry transition

Severity: experiment-validity BLOCKER before beta Status: confirmed

`check_ignored_tasks()` scans a sliding 24-hour delivery-event window at a fixed UTC job time and logs `task_ignored` without changing step status. The separate `expire_overdue_steps()` applies the real per-step deadline and marks `expired` without emitting the corresponding event. A task can therefore be logged ignored before its real deadline and later be completed, or expire with no ignored event.

Required implementation: delete the independent 24-hour inference. Emit one idempotent `task_expired` from the same transaction that changes the step from `delivered` to `expired`. Enforce one terminal user outcome per plan step.

TEL-06 — Runtime actions are not logged consistently

Severity: P1 before beta Status: confirmed

Target runtime tools commit pause, resume, cancellation, first evening time, day/evening time changes, and plan creation without one consistent telemetry path. Old adaptation code logs some pause/resume events but is not the target tool authority. Cancellation and direct time changes can succeed with no product event.

Required implementation: emit action events from the authoritative runtime service only after the database transition commits, using tool execution IDs. Record scheduler/reschedule success or failure as a linked operational result, not as proof that the user action failed after its DB state committed.

TEL-07 — Automatic continuation makes the old continuation metric invalid

Severity: P1 contract correction before beta Status: confirmed

The old contract measures whether the user chose another 7/14-day plan. FD-01 now creates the next same-format cycle automatically. Creation and delivery can happen with no user choice, so counting them as retention would mechanically inflate success.

Required implementation: record `next_cycle_created` as an operational fact and use the first explicit response in that cycle as behavioral retention. Format switching, pause, and cancellation remain separate choices.

TEL-08 — `user_message` is not a reactive-use metric

Severity: P1 measurement correction Status: confirmed

Every inbound text is logged as `user_message`, regardless of whether it is product support, a plan-control request, workday emotional support, feedback, or unrelated text. Its only context is message length. Counting it as self-initiated recovery would be false, while classifying raw text into psychological categories would violate the product/privacy boundary.

Required implementation: retain a neutral inbound-message operational event if useful. Count explicit runtime actions and FD-07 feedback through their own events. Do not create a self-start metric until a distinct self-start feature exists.

TEL-09 — Legacy inference scores and streak incentives contradict FD-11

Severity: P1 removal before beta Status: confirmed

`hidden_compensation_score` combines night completions, an ad-hoc edge-of-day JSON flag, and batch completion into an inferred score with no validated meaning. `EngagementStatus`, `FailureSignal`, skip-streak helpers, and several friction labels preserve the same legacy adaptation worldview. Success streak is also used in completion messages, reports, Coach context, templates, and tests, so removing it as a product mechanism is cross-domain work rather than dropping one telemetry column.

Required implementation: remove inferred-state scores, failure labels, and adaptation consumers. Remove user-facing/streak-trigger behavior and stored streak optimization; keep dated terminal actions so descriptive run lengths remain derivable if later analysis needs them.

TEL-10 — Operational reliability and user behavior are mixed

Severity: P1 before beta interpretation Status: confirmed

Delivery can succeed before its telemetry transaction commits; plan creation can commit before scheduler activation fails; time changes can commit before rescheduling fails. Conversely, `task_delivery_failed` is not handled through the same task-event linkage as successful delivery. Without reconciliation, a missing response can mean either user silence or a broken mechanism.

Required implementation: preserve separate requested/committed/effect-applied operational facts, use an outbox/reconciliation path for post-commit side effects, and exclude unconfirmed deliveries from user-response denominators.

TEL-11 — Event names and JSON properties have no enforced schema

Severity: P1 before beta data collection Status: confirmed

`event_type` is arbitrary text and `context` is unrestricted JSONB. Existing constants name events that are never emitted, while emitted names are not one catalogue. Context contains legacy focus/load/adaptation fields and unstable ad-hoc keys. A GIN index makes the blob queryable but not semantically valid.

Required implementation: version and validate the event catalogue, validate required/allowed fields per event, reject unknown analytics properties, and write contract tests for every producer.

TEL-12 — Privacy-preserving aggregates do not yet exist

Severity: P1 architecture before beta telemetry Status: confirmed; owned jointly with PRIV-06/07 and FD-09

Current analytics are user-linked rows and user-level derived tables. There is no independent aggregate contribution, restricted operational aggregate, or sealed aggregate layer. Deleting a user would delete the only contribution; retaining those rows indefinitely would instead violate the accepted privacy contract.

Required implementation: add the idempotent user-event plus independent aggregate write from day one. No live or user-level company view is allowed; only FD-09's gated post-pilot summary may read sealed aggregates. Raw text and small-N operational aggregates never enter the sealed layer.

TEL-13 — User has no immutable first-seen timestamp

Severity: P1 schema requirement before beta Status: confirmed

The current `users` table has no `created_at` or `first_seen_at`. An immutable account-origin timestamp is useful for lifecycle audits, migration checks, and reconstructing when the system first knew the user. It is not, however, the anchor for working-day D1/D3/D7: those cohorts start from attributed `bot_started`, activation, and the actual plan working-day schedule.

Required implementation: add timezone-aware, server-generated `first_seen_at`, backfill legacy users conservatively from the earliest trusted user event/chat/plan timestamp, and preserve an `unknown`/backfilled marker when the exact first-seen moment cannot be proved. Do not silently use migration time as historical registration time.

TEL-14 — There is no internal beta analytics view

Severity: P1 before interpreting beta results Status: confirmed; implementation follows data-integrity blockers

A correct event store without a repeatable way to inspect it leaves the founder with ad-hoc SQL and inconsistent manual calculations. No current internal view shows the deployment funnel, activation stages, working-day D1/D3/D7, completed day coverage, terminal task outcomes, next-cycle response, and delivery/runtime failures under the definitions in FD-11.

Required implementation: after TEL-01..05 and canonical metric queries are verified, add a minimal founder-only read-only report or CSV export. It must support deployment/cohort and observation-cutoff selection and show raw numerators/denominators, percentages, delivery failures, and pseudonymous individual trajectories for the small internal beta. It is not a company dashboard, must not expose conversation/free-text content, and requires restricted access. Prefer one canonical query/service used by both screen and export rather than calculations in templates.

Review discipline: inspect at the end of a working-day cohort or beta checkpoint, not as a daily scorecard. Do not encode automatic $x \rightarrow z$ product decisions for the first 10-15 users. First verify event integrity and delivery, then distinguish deployment, onboarding, operational, content, continued-use, and unknown causes.

TEL-15 — `avg_reaction_sec` is misleading and currently ungrounded

Severity: P1 measurement cleanup Status: confirmed

`TaskStats.avg_reaction_sec` sounds like exercise reaction or execution time, but the only defensible quantity is response latency from confirmed Telegram delivery to a valid callback. Current completion/skip producers do not supply a stable `reaction_sec`, and the aggregate table is already tied to the mixed `step_id` model.

Required implementation: do not migrate this mutable average as-is. Derive or record `response_latency_seconds` from linked `task_delivered` and terminal callback timestamps after idempotent event linkage exists. Keep the user-facing and analytic definition explicit: latency to tap, not exercise duration, difficulty, effect, or attention. Prefer bounded buckets in sealed aggregates.

Required implementation work and order

1. implement the canonical event envelope, catalogue, and source-operation idempotency;
2. finish FD-08/CONTENT-07 linkage so events reference real plan cycles, plan steps, exercises, and content versions;
3. add deployment records/deep-link attribution and instrument deterministic onboarding;
4. unify expiry status and `task_expired` emission; remove 24-hour ignored inference;
5. instrument authoritative runtime actions and automatic continuation;
6. remove hidden compensation, legacy failure/engagement inference, and user-facing streak consumers;

7. implement FD-09/PRIV-07 independent aggregation in the same idempotent ingestion operation;
8. add and conservatively backfill immutable `User.first_seen_at`;
9. implement and test canonical activation, working-day D1/D3/D7, cycle coverage, per-task outcome, next-cycle response, and operational reliability queries;
10. derive response latency from linked delivery/callback facts rather than the legacy average;
11. reconcile a seeded end-to-end beta cohort against plan/step state;
12. expose the verified canonical queries through a restricted founder-only report/CSV before interpreting beta results.

Explicitly deferred

- D14/D30 decision thresholds until the first seven-working-day cohort and automatic next-cycle behavior are trustworthy;
- self-start exercise metrics until the product has an explicit self-start action;
- company-facing dashboards or reporting;
- ML personalization, inferred user traits, churn scores, or adaptation;
- claims about stress, burnout, health, productivity, or ROI from interaction telemetry alone;
- read receipts and measured exercise duration, which the Telegram bot does not observe.

Database & Redis Integrity Findings

Database & Redis Area — Audit Round 2026-08-12

Status: static audit complete against committed application HEAD `ac2711d`; Railway production metadata checked read-only; physical SQL introspection deferred until the database implementation round because production services are stopped. No production service, migration, or data was changed.

Scope and evidence

The round covered PostgreSQL models and access paths, Redis usage, scheduler persistence, plan lifecycle ownership, telemetry linkage, content persistence, privacy retention, migration authority, indexes, constraints, connection pools, and Railway storage/recovery metadata.

Production metadata observed on 2026-08-12:

- Railway project: `Love Yourself BOT`;
- Postgres, Redis, and bot had no active deployment; previous Postgres

deployments were `REMOVED`;

- the Postgres volume was `READY`, 500 MB capacity, approximately 118 MB used;
- the Redis volume was `READY`, 500 MB capacity, approximately 49 MB used;
- scheduled volume backups: none; existing backups: none; PITR: disabled;
- the last Postgres deployment configuration used one replica.

The physical production tables, columns, constraints, migration ledger, and row counts were not inspected because the database was stopped. Static findings are confirmed against HEAD; destructive migration details remain conditional on later read-only introspection of a restored copy.

Recovery dependency

FD-12 is the standing operational backup policy. This module does not redefine it. Database implementation must follow FD-12 before physical introspection or the first production migration.

Findings

DB-01 — Production schema is not reproducible from the repository

Severity: beta blocker before the first schema change Status: confirmed in HEAD; physical production delta not yet inspected

Runtime startup relies on `create_all()`, which creates missing tables but does not migrate existing ones. The repository does not contain an authoritative, continuous Alembic ledger capable of reconstructing the current production schema.

Required work: follow FD-12 and create a baseline from the restored physical schema. Do not replay historical ad-hoc SQL blindly.

DB-02 — Lifecycle truth is duplicated across several records

Severity: P1 Status: confirmed; implementation owned by FD-08 / FSM-01...08

`User.current_state`, `UserProfile.is_paused`, plan status/end fields, step states, and scheduler jobs can disagree. The accepted target is plan-centric: one authoritative plan aggregate with user mode derived from it.

Blanket reference: implement through FD-08 and the Lifecycle State & Persistence round rather than inventing a second DB-specific lifecycle contract.

DB-03 — The database does not enforce one current plan per user

Severity: beta blocker Status: confirmed

Application checks are insufficient against retries or concurrent callbacks. Add the narrow database constraint/index implied by FD-08 so a user cannot have two current plans. This is a cheap integrity guard, not a scale optimization.

DB-04 — Lifecycle writes are only partially serialized

Severity: P1, minimal beta protection required; full framework deferred Status: confirmed

V5 plan creation locks relevant rows, but pause/resume/cancel, completion, and task callbacks do not share one locking/idempotency discipline. Retries and double taps can affect one user; the risk is not dependent on a large audience.

Required work: make critical callbacks atomic and idempotent. A universal locking framework may wait.

DB-05 — Telemetry cycles are detached from the authoritative V5 plan

Severity: beta blocker for trustworthy telemetry Status: confirmed; jointly owned by TEL-02/03 and FD-08

V5 creates `AIPlan`, while telemetry creates/reuses legacy `PlanInstance` and `PlanExecutionWindow`; execution windows are not reliably closed. Events from separate cycles can therefore be attributed to one telemetry window.

Blanket reference: use the canonical plan/cycle linkage specified by TEL-02/03.

DB-06 — `UserEvent.step_id` mixes plan-step and exercise identities

Severity: beta blocker for trustworthy telemetry Status: confirmed in committed HEAD; a local uncommitted split exists but was not treated as production truth

Required work: separate `plan_step_id` and `exercise_id`, preserve current runtime behavior through a forward migration, and include `content_version` as specified by CONTENT-07 and the Event Contract.

DB-07 — Event ingestion has no stable idempotency key

Severity: beta blocker Status: confirmed; owned jointly by TEL-04 and PRIV-07

Random event UUIDs identify insert attempts, not source operations. A retried callback or job can create duplicate facts and duplicate aggregate contributions.

Required work: persist a stable `source_operation_id` with a unique constraint; write the user-linked event and independent aggregate contribution in one idempotent operation.

DB-08 — PostgreSQL and external side effects cannot be reconciled

Severity: P1 before beta for critical paths Status: confirmed

Telegram delivery, scheduler mutation, Redis state, and PostgreSQL commits can succeed independently. Critical flows need retry-safe reconciliation: plan scheduling, exercise delivery logging, and delivery-time rescheduling. A generic outbox framework is not required before beta.

DB-09 — Content has two competing sources of truth

Severity: P1 before content rollout Status: confirmed; owned by FD-10 / CONTENT-01...09

Generation reads JSON while plan steps reference DB `content_library` rows, and startup does not establish a versioned import contract. Use one versioned deploy/import path and snapshot the delivered content/version. Do not resolve this separately from the accepted Content Library overhaul.

DB-10 — Legacy and sensitive schema remains present

Severity: mixed: sensitive fields before beta; harmless legacy backlog Status: confirmed

Dead structures include old plan/telemetry paths and sensitive-looking health fields such as medical facts, stress, energy, and mood notes. Remove unused sensitive fields before real employee data is collected. Non-sensitive legacy tables may be removed after the authoritative schema and row use are verified.

DB-11 — Accepted data mechanisms do not yet exist

Severity: beta blocker/P1 depending on mechanism Status: confirmed; blanket-owned by existing audit rounds

The schema lacks accepted feedback storage, privacy notice acknowledgement, independent aggregate ingestion, event idempotency, review gating, and an outbox/reconciliation mechanism.

Blanket references:

- privacy acknowledgement, retention, access/export/delete: PRIV-01...10;
- independent aggregates: FD-09 / PRIV-07;
- feedback records: FD-05 / FD-07;

- content review gates and versions: FD-10 / CONTENT-01...09;
- canonical events: FD-11 / TEL-01...15.

This finding inventories missing persistence; those source sections remain the behavioral contracts.

DB-12 — Domain constraints are too weak

Severity: P1 Status: confirmed

Key enums, date/time ranges, completion values, relationships, and one-current- plan invariants rely too heavily on application code. Add focused foreign-key, unique, check, and partial-index constraints during the baseline migration series. Do not blanket-constrain uncertain legacy data before introspection.

DB-13 — High-value compound indexes are missing

Severity: backlog/P2 for the small beta Status: confirmed

Add indexes only from verified query paths and `EXPLAIN` evidence after the schema is authoritative. Likely candidates cover current-plan lookup, pending/delivered tasks, scheduler due work, chat history ordering, and canonical event queries. This is not a reason to delay the beta integrity fixes.

DB-14 — Mutable derived fields can drift from source facts

Severity: P1 Status: confirmed; jointly owned by FD-08 and TEL-02

Duplicated end dates, percentages, states, and counters can become stale. Retain a stored derived value only where it is transactionally maintained or explicitly rebuildable; otherwise calculate it from authoritative facts.

DB-15 — `User` lacks immutable cohort/deployment chronology

Severity: beta blocker for cohort reconstruction Status: confirmed; owned by TEL-01/13

Add and conservatively backfill immutable `first_seen_at` (and explicit deployment attribution). Working-day D1/D3/D7 anchors to activation/plan days, not this timestamp; `first_seen_at` exists to reconstruct acquisition cohorts and data provenance.

DB-16 — Privacy retention and user-rights lifecycle are absent

Severity: legal/product beta blocker Status: confirmed; owned by Privacy round

Chat history has no 90-day purge, access/export/delete workflow is absent, and report grants are not expiring/revocable. Implement the accepted Privacy round as one lifecycle. Do not reinterpret this finding as a new privacy policy.

DB-17 — Redis conversation history can become stale authority

Severity: P1 Status: confirmed

Redis retains the last messages without TTL and is preferred over PostgreSQL whenever non-empty, even when partial or stale. Define one bounded cache contract: namespaced keys, TTL aligned with chat retention, versioning, and explicit fallback/rebuild semantics. PostgreSQL remains durable truth.

DB-18 — Redis contains legacy namespaces and unused FSM storage

Severity: P2, with cheap cleanup during migration Status: confirmed

Old schedule-adjustment state and unused aiogram FSM storage remain. Inventory actual production keys before deletion, version active namespaces, and remove legacy writers. Do not flush Redis wholesale.

DB-19 — Application and scheduler create separate PostgreSQL pools

Severity: backlog/P2 for current topology Status: confirmed

Separate engines/pools are acceptable for one small bot deployment but need explicit limits, `pool_pre_ping`, observability, and later consolidation if connection pressure appears. This is not a beta blocker by itself.

DB-20 — APScheduler assumes a single writer process

Severity: deployment invariant before beta; leader election deferred Status: confirmed

Run exactly one bot replica / scheduler writer for beta and verify this in deployment configuration. Multiple writers require leader election or an external scheduler later; do not build that framework now.

Required work before beta, in order

1. **Recovery:** implement FD-12; test restore; introspect the restored physical schema.
2. **Migration authority:** create the Alembic baseline and a forward-only migration sequence.
3. **Plan integrity:** implement FD-08, one-current-plan protection, and minimal atomic/idempotent lifecycle callbacks.
4. **Telemetry integrity:** implement TEL-02/03/04/13 and CONTENT-07 linkage.
5. **Content consistency:** implement the accepted versioned content source, delivery snapshot, and review gating.
6. **Critical reconciliation:** make scheduling, delivery logging, and time rescheduling retry-safe.
7. **Privacy minimum:** implement notice acknowledgement, 90-day chat retention, manual access/export/delete, revocable report grants, and remove unused sensitive fields.
8. **Deployment guard:** one bot replica / one scheduler writer.

Explicitly deferred

- full cleanup of harmless legacy tables;
- broad index tuning without query evidence;
- connection-pool unification;
- scheduler leader election for multiple replicas;
- a universal outbox/locking framework beyond critical beta paths;
- PlanDraft simplification;
- ML, a data warehouse, or vector storage;
- broad Redis redesign beyond TTL, namespace, and stale-authority fixes;
- PITR while the MVP data volume and recovery objective do not justify it.

Delivery UX & User-Facing Product Surface Findings

Delivery UX Area — Audit Round 2026-08-13

Scope

This round audits every material employee-facing touchpoint around the current Telegram MVP, not only exercise copy:

- entry and handoff into onboarding;
- notification preview and scheduled exercise presentation;
- complete, skip, expiry, pause, cancel, and failure states;
- Coach-wrapper degradation and deterministic controls;
- completion artifact;
- media delivery;
- cross-channel presentation boundaries;
- end-to-end UX verification.

The onboarding conversation itself remains owned by the Onboarding round and its accepted rewrite. Product positioning beyond the accepted FD-13 name is not part of this code-audit area.

Accepted target behavior

The employee-facing MVP is **LY Workday**. A scheduled action appears first as a neutral notification preview and then as a complete in-chat exercise. The interaction stays bounded:

```
notification
→ one exercise
→ one explicit outcome
→ closed state
```

The bot proactively sends only scheduled exercises and cycle summaries. It does not revive the removed pulse/persona/re-engagement branch. Plan controls remain deterministic and usable without the Coach model. The Coach receives the same canonical exercise facts as the renderer and may explain or execute allowed runtime actions within its existing scope.

UX-01 — Entry into the product is not one finished flow

Severity: P1 before beta Status: confirmed

Current behavior: `/start` presents generic legacy bot copy, while old `newplan_*` deep links enter the superseded parameter-selection model. There is no FD-18 roster/SSO authorization, entitlement-bound invitation redemption, testnet issuance path, or canonical handoff into the new onboarding flow.

Expected behavior: in production the employee passes roster/SSO authorization, opens the issued Telegram deep link, presses the native Start action, and enters deterministic onboarding without obsolete plan setup. Testnet begins with a founder-issued test token and is identical from Telegram Start onward. Authorization failure, repeat account entry, spent/invalid/revoked invitation, visible command behavior, and already-onboarded paths must be verified.

Minimal fix: replace legacy start/deep-link routing with the canonical deployment and onboarding entry contract; do not redesign the onboarding conversation inside this finding.

UX-02 — Lock-screen preview behavior is unknown on real devices

Severity: P1 before beta Status: confirmed

Current behavior: no device-level test establishes which sender name, first line, truncation, media preview, or privacy-sensitive text appears on iOS and Android lock screens.

Expected behavior: implement FD-13/14, then verify the neutral first line, sender identity, truncation, hidden-preview settings, and media behavior on real supported devices. Notification copy is a privacy and action-entry surface, not a place for exercise rationale or emotional labels.

Minimal fix: create a small device test matrix and lock one verified beta copy after seeing the actual notifications.

UX-03 — The exercise renderer still implements the old delivery contract

Severity: P1 before beta Status: confirmed; expands DEL-01, DEL-02, and DEL-07

Current behavior: delivery can expose day/slot/count metadata, decorative framing, and rationale while failing to render the accepted `display.steps` payload consistently. Dynamic HTML is not safely escaped.

Expected behavior: the opened chat message renders the neutral preview label, exercise title, exact duration, current versioned steps, required instructional media when applicable, and Виконано / Пропустити. Internal category, slot, selection, and scheduling metadata remain hidden.

Minimal fix: replace the legacy formatter with one escaped canonical presentation renderer based on FD-06, FD-10, and FD-16.

UX-04 — Telegram cannot complete the exercise from the OS notification

Severity: platform constraint; beta UX must account for it Status: confirmed

Telegram Bot API inline buttons are available in the chat message, not as the bot's custom action buttons in the operating-system notification. Therefore a user must open Telegram before seeing and pressing Виконано or Пропустити.

Expected behavior: treat the extra open as a real funnel step, keep the in-chat exercise glanceable, and preserve its buttons after screen lock/reopen until the true action deadline. Do not claim notification-level completion and do not distort exercise duration to work around screen timeout.

Minimal fix: test the accepted FD-14 path end to end and instrument only facts the Telegram wrapper can actually observe.

UX-05 — Complete and skip still enter the removed engagement flow

Severity: P1 before beta Status: confirmed; expands DEL-03

Current behavior: completion/skip handlers can send new praise messages, calculate streaks, select persona triggers, and suggest adaptation after repeated skips.

Expected behavior: edit or close the original exercise message, remove its buttons, and show one neutral registered state: Виконано or Пропущено. Optional better / same / worse feedback appears only after registered completion and remains part of the same bounded interaction under FD-07.

Minimal fix: remove persona/streak/adaptation branches from callbacks and make the callback transition idempotent.

UX-06 — Delivered exercise states have no one presentation contract

Severity: P1 before beta Status: confirmed; expands DEL-04 and the lifecycle findings

Current behavior: expiry may only remove buttons; expired or cancelled callbacks can produce silence; pause and cancellation do not consistently close or preserve already-delivered actions.

Expected behavior: define and render delivered, completed, skipped, expired, and cancelled states consistently. An already-delivered action remains actionable through its existing deadline when the plan is paused. Cancellation closes pending/delivered actions immediately. A late tap receives a brief factual response rather than silence.

Minimal fix: centralize visible state transitions and test each transition against the authoritative plan lifecycle.

UX-07 — Exercise delivery has no bounded retry or reconciliation

Severity: P1 before beta Status: confirmed; same underlying defect as DEL-05 and DB-08

Current behavior: a transient Telegram send failure is logged and can permanently lose the scheduled exercise. Database, scheduler, and Telegram side effects do not establish one reconcilable delivery result.

Expected behavior: bounded retry, stable source-operation identity, and reconciliation distinguish delivered, retryable, terminal failure, and duplicate attempts without double-sending.

Minimal fix: implement retry-safe delivery logging for the beta path rather than a universal messaging framework.

UX-08 — The Coach wrapper does not fail gracefully across input and API states

Severity: P1 before beta Status: confirmed

Current behavior: the primary wrapper handles text, while photo, voice, file, sticker, and other unsupported inputs may receive no useful response. OpenAI failure and rate limiting do not have one complete user-facing fallback.

Expected behavior: unsupported inputs and Coach outages receive short, deterministic explanations. Menu and lifecycle controls remain functional without an LLM response. The Coach does not browse, generate arbitrary media, write code, or perform tasks outside the accepted prompt scope.

Minimal fix: add deterministic fallback handlers around the existing scoped Coach; do not expand it into a general-purpose assistant.

UX-09 — The abandoned engagement layer remains reachable

Severity: P1 before beta Status: confirmed; cross-reference to SCH-01 and related legacy findings

Current behavior: silent checks, pulse snapshots/routes/templates, persona, streak, comeback, quotation, silence, and adaptation remnants remain in runtime or user-facing paths.

Expected behavior: the bot initiates only scheduled exercise delivery and the cycle summary. Reactive Coach replies remain allowed when the user writes first. No NPC-like chatter or generic re-engagement survives accidentally.

Minimal fix: remove the legacy writers, routes, templates, callbacks, jobs, and dead persona source as one deletion pass with regression tests.

UX-10 — Plan and schedule controls expose obsolete interaction models

Severity: P1 before beta Status: confirmed; cross-reference to FSM-03, RT-05, and RT-07...13

Current behavior: English actions such as `Confirm plan`, `Regenerate`, `Change parameters`, and `Restart from scratch`, plus old `SCHEDULE_ADJUSTMENT`, slot selection, and multi-task controls remain in code.

Expected behavior: only current actions remain: status, delivery-time change, pause, resume, cancel, and explicit format switch when implemented. Their availability and wording follow the canonical plan lifecycle and Coach tool contract.

Minimal fix: delete old callback surfaces and connect current controls to one authoritative runtime-action layer.

UX-11 — Tool outcomes are hardcoded instead of returned to the Coach

Severity: P1 architecture before first-class Coach UX Status: confirmed; same defect as RT-09 and COACH-09

Current behavior: the model emits a tool call but does not receive the actual execution result; fixed templates voice success/failure and can diverge from runtime truth.

Expected behavior: a bounded tool-result loop returns a structured, allowlisted result to the Coach, which then writes one natural user-facing reply grounded in the actual result. Failed actions must not be described as successful.

Minimal fix: implement the already-recorded COACH-09 architecture with strict result schemas, one follow-up model turn, and no recursive tool chain.

UX-12 — The accepted deterministic user menu is not implemented

Severity: P1 before beta Status: confirmed

Current behavior: plan control and product information depend too heavily on free-text interpretation and legacy keyboards. No stable user menu exists.

Expected behavior: a deterministic menu available without the Coach exposes the current cycle and next delivery, time management, pause/resume/cancel, format switch when available, privacy/data information, support, and about. Exact labels, grouping, and callback layout are implementation design work, not a new founder decision.

Minimal fix: build the smallest menu over canonical runtime actions; it must remain usable during Coach/API degradation.

UX-13 — The user-facing privacy promise contradicts the accepted MVP boundary

Severity: P1 trust and contract issue before beta Status: confirmed; cross-reference to FD-09 and the Privacy round

Current behavior: Product Map copy states broadly that the company receives aggregate analytics, without FD-09's post-pilot timing, 100/50 gate, three allowed headline values, or prohibition on live/sliced reporting.

Expected behavior: entry, onboarding, privacy menu, support copy, Product Map, and sales-facing facts all state the same boundary: no live or individual view; one gated aggregate product-usage summary after the pilot; no wellbeing, productivity, Coach-text, or small-group reporting.

Minimal fix: synchronize both Product Maps and related product-contract copy before exposing the privacy surface.

UX-14 — The current completion report contradicts the accepted lifecycle

Severity: P1 before beta Status: confirmed

Current behavior: the HTML report and its supporting code contain evaluative tiers, streaks, adaptation counts, dominant-slot/persona interpretation, and old next-plan choice actions. Its bearer link also inherits PRIV-08 risk.

Expected behavior: implement the deterministic Telegram image defined by FD-15 for every completed cycle. It reports controlled facts, functions as the cycle's tangible result, and bridges directly into FD-01 automatic same-format continuation.

Minimal fix: replace the MVP report delivery path with a tested application- rendered image and factual text fallback; retire the old user-facing report route after required compatibility handling.

UX-15 — Instructional media delivery does not exist

Severity: P1 for the FD-16 exercises before they enter beta Status: confirmed; expands CONTENT-06

Current behavior: there is no animation/photo send path, versioned asset linkage, caption contract, media fallback, or telemetry linkage to the content version shown.

Expected behavior: breathing, fist PMR, and approved cool-water delivery use their versioned instructional GIF under FD-16. The exact text remains self-sufficient, and media failure degrades to text without blocking action. Other exercises ship without mandatory GIFs.

Minimal fix: implement the smallest versioned `sendAnimation/fallback` path and cover both successful and failed media delivery.

UX-16 — Renderer, Coach, and future channels do not share presentation truth

Severity: P1 for system consistency; channel expansion is deferred Status: confirmed

Current behavior: Telegram builds a message directly from content and stores rendered text, while the Coach receives separately assembled context. A future Slack, WhatsApp, browser, or native-app adapter would likely duplicate these rules again.

Expected behavior: one canonical structured `ExercisePresentation` supplies exercise ID, content version, title, duration, exact steps, media metadata, status, action deadline, and available actions. Each channel renders that structure; the Coach receives the same facts as `current_exercise_context`.

Minimal fix: introduce the shared presentation object while implementing the new renderer. Do not build speculative channel adapters or a mini app now.

UX-17 — There is no end-to-end user-facing beta QA path

Severity: beta blocker after implementation Status: confirmed

Current tests do not prove the complete real Telegram journey:

```
deployment link
→ onboarding handoff
→ first scheduled exercise
→ complete / skip / optional feedback
→ expiry
→ time change
→ pause / resume
→ cancel
→ automatic continuation
→ cycle summary
→ Coach and media failure
```

Expected behavior: run this path on real iOS and Android Telegram clients and verify copy, callback states, persistence, notification preview, failure fallbacks, and duplicate-tap behavior.

Minimal fix: create one scenario checklist plus automated integration coverage for deterministic transitions; perform device QA before beta invitations.

UX-18 — Delivery-time ownership remains an explicit MVP hypothesis

Severity: P1 product/UX hypothesis; exact policy not yet accepted Status: open; linked to ONB-04 and MISC-02

Current behavior: time handling is inconsistent across legacy onboarding, Coach natural-language tools, and deferred picker concepts. The current system does not implement workflow-aware timing.

Working MVP hypothesis: the employee chooses an individual exact delivery time inside a company/deployment workday window. The window prevents out-of-scope late-night scheduling while individual choice avoids synchronized office-wide exercise behavior and preserves autonomy. The exact window, configurability, and exception policy remain unapproved and must not be hardcoded from an audit example.

Future option: delivery anchored to workflow events such as the end of a focus block, meeting, deployment, IDE session, or calendar event. This remains deferred because it adds substantial integration, consent, privacy, and individual-model complexity. MVP copy must not imply that the system detects the objectively correct moment.

Minimal next step: validate the window model with the first beta company and employees, then record the exact policy before implementing onboarding/time- picker constraints.

Open implementation choices, not founder decisions

- the exact neutral preview word after device testing (Пауза vs Перерва);
 - the exact allowed delivery window and whether a company can configure it;
 - detailed menu information architecture and labels;
 - final GIF art direction and asset-production process;
 - future workflow-trigger integrations and their consent/privacy model.
-

Security & Configuration Findings

Security & Configuration Area — Audit Round 2026-08-15

Scope and files inspected

This round covers repository, AI-agent, transport, API, and runtime configuration boundaries rather than re-auditing product behavior:

- `.env`, `.env.example`, `.gitignore`, Docker build context, and deployment

entrypoints;

- `app/config.py`, `app/ai.py`, `app/api.py`, `app/main.py`,

`app/redis_client.py`, dependency declarations, and container execution;

- Coach prompt/tool exposure, orchestrator allowlisting, runtime-tool backend

guards, and the future tool-result loop boundary;

- Telegram messages, callbacks, deep links, update replay/idempotency, and the

admin command surface;

- model usage propagation, content-bearing logs, alerting, kill switches, and

incident recovery.

Threat actors considered: a normal user sending accidental bursts; an authenticated beta user deliberately automating messages or prompt-injection attempts; a user replaying stale buttons; a person obtaining a bearer report or enrollment link; and an attacker obtaining a committed credential. Telegram, OpenAI, Railway, and PostgreSQL provider compromise is outside this code audit.

This is a static repository audit against the current local worktree and committed baseline. Live Railway ingress, environment variables, service visibility, provider budgets, database roles, and deployed image contents must still be verified at the Release & Operations gate. No static review can make the system literally bulletproof; the target is bounded blast radius, layered authorization, observable failure, and recoverability.

Privacy, report-token, and endpoint findings already owned by another area are referenced below instead of duplicated.

Findings

SEC-01 — Production-like secrets are tracked and enter the Docker build context

Severity: BLOCKER Status: confirmed

Current behavior: `.env` has been tracked since the initial commit and contains non-placeholder Telegram and OpenAI credentials. `.gitignore` does not exclude environment files, no `.dockerignore` exists, and `Dockerfile` uses `COPY . .`. The build context therefore also includes local environments, Git metadata, development artifacts, and any tracked or untracked secret files not excluded by Docker.

Expected behavior: no runtime secret is committed, copied into an image, or used as a repository default. Railway/environment secret variables are the runtime source; the repository contains placeholders only.

Minimal fix:

1. treat the Telegram and OpenAI credentials as compromised and rotate them;
2. remove `.env` from Git tracking and add environment-file rules to `.gitignore`;
3. add a narrow `.dockerignore` for `.env*` except the explicit example, Git metadata, virtual environments, caches, local R&D, logs, and OS artifacts;
4. verify the built image contains no credential file;
5. decide separately whether coordinated history rewriting is worthwhile.

History rewriting does not replace rotation and must not be the first response.

SEC-02 — Production configuration fails open to development behavior

Severity: P1 before beta deployment Status: confirmed

Current behavior: `ENVIRONMENT` defaults to `dev`, arbitrary values are accepted, and production-only validation runs only for the exact string `prod`. `APP_BASE_URL` and `BOT_USERNAME` have placeholder defaults. Missing Redis configuration silently falls back to process-local storage or disables Redis-backed context rather than establishing whether Redis is required for the selected environment.

Expected behavior: environment identity and required infrastructure are explicit. A production typo or missing critical setting stops startup instead of producing partially working behavior or invalid user-facing links.

Minimal fix: allow only `dev`, `staging`, and `prod`; define required settings per environment; reject placeholder URLs, bot names, secrets, and missing production dependencies at startup; keep deliberate local fallbacks limited to development and tests.

SEC-03 — There is no complete authoritative configuration contract

Severity: P1 before beta deployment Status: confirmed

Current behavior: `.env.example` omits several settings used by runtime code, while `README.md` describes an older product and configuration model. Numeric parsers silently replace malformed values with defaults and do not enforce semantic ranges. Configuration therefore depends on code archaeology and can drift from Railway without an obvious failure.

Expected behavior: one typed configuration contract names every supported setting, whether it is required, its safe default where one exists, its valid range, and the environments in which it applies. Secret values are never printed during validation.

Minimal fix: synchronize `.env.example` with the typed settings model, validate ranges and cross-field requirements at startup, remove obsolete settings such as `DEFAULT_DAILY_LIMIT` under FD-17, and keep the deployment variable list as the canonical operational checklist.

SEC-04 — Coach token usage is calculated and then discarded

Severity: P1 before interpreting beta operating cost Status: confirmed

Current behavior: the OpenAI response adapter extracts input, output, and total token usage, and `coach_agent` returns it. `handle_incoming_message` reduces the worker result to reply and UI fields, so usage is neither persisted nor emitted as a reliable operational metric. Actual unrestricted usage and cost therefore cannot be observed under FD-17.

Expected behavior: unrestricted Coach access remains unchanged, while every model request produces privacy-safe operational usage data. The measurement must not include conversation text and must not become a user score or a company-facing metric.

Minimal fix: propagate model name, token counts, outcome, latency, and estimated cost into the operational telemetry path; separate model failures from successful zero-usage responses; expose aggregate founder-only totals and provider budget alerts. Do not add automatic user quotas or refusals.

SEC-05 — Model tool output is constrained but not fully re-authorized at execution

Severity: P1 before external beta Status: confirmed; blast radius is currently narrow

Confirmed safe properties: user content is sent as a user message rather than system context; only state-filtered tools are exposed to the model; tool schemas are strict; the orchestrator accepts only names from a fixed registry; and the runtime injects the authenticated current user's internal ID instead of letting the model choose a user. Coach has no browser, arbitrary HTTP, filesystem, shell, email, or raw database tool. A prompt injection therefore cannot directly turn Coach into a general remote-execution agent or target another user by supplying their ID.

The remaining gap is at the final execution boundary. `_execute_plan_tool()` rebuilds the registry containing all known Coach tools and does not independently verify that the selected name was in the exact state-filtered set offered for this request. Several backend tools re-check state, but `change_day_time`, `change_evening_time`, and first-time evening collection have the prerequisite gaps already recorded in RT-01 and RT-08. The unused `_detect_foreign_instructions()` substring scanner is not called and would not be an authorization boundary if it were.

Expected behavior: treat every model-produced tool call as untrusted input. Immediately before mutation, the executor must re-read the authoritative user, state, current plan, ownership, consent prerequisite, and tool-specific context; reject any tool not in the request's exact allowed set; validate arguments semantically in the backend; and return a bounded structured result. The second COACH-09 model pass receives no tools. Add adversarial tests for prompt-leak requests, role-change instructions,

fabricated tool names, valid tool names in the wrong state, missing cancellation confirmation, malformed arguments, and cross-user identifiers.

Do not add a keyword-based jailbreak detector as the primary fix. Least privilege, fresh authorization, backend invariants, bounded agency, and tests are the security boundary.

SEC-06 — Coach has no concurrency, rate, timeout, or cost-abuse boundary

Severity: P1 before external beta Status: confirmed; target accepted in FD-17

Current behavior: several Telegram paths can enter `handle_incoming_message`, including legacy deterministic callbacks that unnecessarily convert button presses into Coach text. There is no per-user in-flight lock, inbound Coach admission guard, explicit model-call timeout, global token/cost circuit breaker, or persisted usage accounting. The output ceiling is not a fixed literal: `settings.MAX_TOKENS` defaults to 300 but can be overridden from the environment through an integer parser with no semantic range validation. It limits output size only; it does not bound the large static input, concurrent requests, retries, complete tool-result turns, or total paid calls.

Minimal beta fix: first complete the accepted legacy cleanup so deterministic commands and callbacks invoke backend operations directly. At the free-text Coach boundary, deduplicate Telegram updates and apply admission immediately: start the first turn without debounce when the user is idle; while it runs, enqueue at most nine later messages in a process-local FIFO. Preserve each message as a separate turn and its arrival order. Refuse additional pending messages with the deterministic FD-17 overflow response without stopping or changing accepted work. After transport deduplication, count every attempted user-authored free-text message against rolling flood windows, including queue-rejected attempts; exclude deterministic commands, callbacks, and internal model continuations.

Then implement FD-17 in this order: enforce one per-user worker/lock consuming the bounded FIFO, add explicit model timeout and bounded retry policy, persist privacy-safe usage from SEC-04, enforce attempted-text flood limits of 30 per rolling minute and 300 per rolling hour per user, add the global cost circuit breaker, and return the neutral fallback when admission is refused. Validate `MAX_TOKENS` and all numeric safety settings against explicit startup ranges. Hold the lock across the complete turn, not only the model request, and remove idle per-user lock/FIFO entries through race-safe registry cleanup.

The two rolling limits are founder-selected beta abuse thresholds, not generic industry defaults. Observe their false-positive rate and full-turn token/cost distribution, then revise them from evidence. Do not add a debounce window or merge separate user messages merely to optimize an edge case: the first turn starts immediately and up to nine later turns wait in order. Do not build a durable queue for the one-process beta. Before adding another process, replace the process-local FIFO and counters with cross-process coordination.

SEC-07 — Task callback ownership is enforced, but replay safety is not atomic

Severity: P1 before external beta Status: confirmed; cross-reference DB-04, DB-07, and the Event Contract

Confirmed safe properties: Done and Skip load the referenced plan step, compare the plan owner's Telegram ID with the callback sender, require an active plan, and reject terminal steps. Existing tests cover cross-user denial and repeated sequential clicks.

The check and write are not one atomic operation. Neither callback locks the step row nor performs a conditional terminal-state update, and telemetry lacks a stable unique operation key. Two concurrent callbacks can both read the step as actionable, write conflicting or duplicate terminal outcomes, and emit two events. Sequential idempotency tests do not cover this race. Telegram `update_id` helps the transport order/confirm updates, but business mutations still need application-level idempotency across retries, crashes, and double taps.

Minimal fix: make the terminal transition a row-locked or conditional atomic write, enforce one accepted terminal outcome per plan step at the database boundary, and write the lifecycle event with a stable unique idempotency key in the same reliable operation. Add concurrent Done/Done, Skip/Skip, and Done/Skip tests. Do not add signatures to current private-chat callback data as a substitute for ownership and atomicity.

SEC-08 — Dependency and container security are not reproducible or scanned

Severity: P1 Release & Operations gate before external beta Status: confirmed; no specific dependency vulnerability asserted by this audit

Current behavior: several Python dependencies are not exactly pinned, one uses a lower bound, no hash-locked dependency artifact or automated vulnerability scan is configured, and the mutable `python:3.11-slim` image runs the application as root. SEC-01 separately covers secrets and the over-broad Docker context. The repository therefore cannot currently prove which dependency/image build was reviewed or whether known advisories were checked.

Minimal fix at the final release gate: resolve and lock exact tested dependencies, run an SCA audit against the resolved environment, review rather than blindly apply upgrades, pin/rebuild the base image deliberately, run the container as a non-root user, scan the final image, and make dependency/security checks part of CI. Record accepted exceptions with an owner and review date.

SEC-09 — Security logging and incident response are not operationalized

Severity: P1 before external beta Status: confirmed

Current behavior: logs contain user IDs, full error payloads, tool arguments, and up to 500 characters of Coach output, while there is no central redaction allowlist, security-event taxonomy, access/retention rule, alert destination, credential-rotation checklist, or tested kill-switch procedure. Privacy findings PRIV-02 and PRIV-03 own removal of conversational content; this finding owns the ability to detect and contain an incident without collecting more sensitive text.

Minimal beta runbook:

1. inventory Telegram, OpenAI, database, Redis, Railway, and report-token credentials; name the owner and verified revocation/rotation path for each;
2. provide independent emergency controls for Coach calls and scheduled delivery, plus the FD-17 cost circuit breaker;
3. alert on spend thresholds, request/error spikes, circuit-breaker activation, authorization denials, unknown tool names, duplicate callback conflicts, and repeated startup/restart failure;
4. log structured identifiers and outcomes only; redact secrets and user text, restrict log access, and enforce retention;
5. document containment, key rotation, Railway rollback/redeploy, evidence preservation, data-impact assessment, user/regulatory notification decision, recovery, and post-incident verification;
6. connect database recovery to FD-12 and test the runbook once before beta.

A small beta does not need a SOC or 24/7 monitoring contract. It does need one named owner, reachable alerts, revocable credentials, working stop controls, and a written recovery sequence.

SEC-10 — FD-18 production enrollment has no implemented identity or token boundary

Severity: BLOCKER before external testnet beta for token/environment controls; remaining roster/identity controls before the first production company enrollment Status: confirmed absent; implements FD-18 / COMP-04

Current behavior: the application has no roster importer, OIDC/email challenge callback, access identity, entitlement authorization, production/testnet token separation, or secure account-handoff flow. The only current `/start` grammar is the unrelated legacy `newplan_*` path.

Expected behavior: roster import, Google/Microsoft authentication or verified email fallback, entitlement creation/reconciliation, invitation issuance, and Telegram redemption form one explicit authorization chain. No browser parameter, unverified email string, Telegram sender, or token alone grants production access.

Minimal implementation and threat-model checks:

- authenticate the operator performing roster imports; restrict file size and type; require an explicit `full_snapshot` or `delta` mode; normalize and deduplicate the expected domain; validate and preview exact add/keep/revoke counts; require explicit confirmation; audit versions without copying email into telemetry or general logs;
- use authorization-code OIDC with server-side token validation and state, nonce, CSRF, redirect-URI, issuer, audience, expiry, tenant/domain, stable subject, and verified-email checks; request no mailbox, calendar, directory, or profile permission that access verification does not require;
- for roster-listed addresses outside the supported OIDC providers, use a short-lived single-use email challenge with rate limits, hashed challenge storage, and a neutral response that does not reveal roster membership;
- authorize against the current roster and deployment on the backend; bind the

entitlement and invitation to one environment and deployment;

- generate high-entropy, short-lived, single-use handoff tokens; persist only a

hash; make redemption and enrollment idempotent and atomic; reject replay and cross-environment use with the same neutral response;

- ensure roster revocation closes company-sponsored product access without

deleting history, modifying behavioral records, or exposing the affected identity to company analytics;

- test token leakage through URL/referrer/browser cache, logs, traces, exception

payloads, analytics, and support tooling; rotate signing/HMAC keys through a documented emergency path;

- prove production/testnet separation at startup and in CI with negative tests

for credentials, databases, Redis namespaces, Telegram bots, OpenAI projects, token keys, URLs, and aggregate sinks.

The company agreement and privacy notice are necessary purpose restrictions, but they do not replace these controls.

Conversely, technical separation does not authorize reuse of corporate identity for behavioral analysis.

Verified perimeter boundaries and deferred requirements

- The unauthenticated mutating `/user/time-slots` route remains the direct

blocker in PRIV-10. Remove it if unused; otherwise derive identity from a verified caller rather than `?user_id=` and add authorization/rate-limit tests.

- Completion and pulse URLs are bearer capabilities. Expiry, revocation, account

deletion, and secret rotation remain owned by PRIV-08.

- The current `newplan_*` Telegram deep link uses the authenticated Telegram

sender and checks that sender's current state; it does not expose a cross-user mutation. It is nevertheless legacy product flow and should be removed under Legacy Reachability Cleanup. FD-18's production enrollment-page URL is not itself a credential. Each roster/SSO-authorized or testnet-issued Telegram invitation uses an opaque, entitlement-bound, scoped, expiring, single-use token, stores no raw token in logs, and embeds no employee or company facts in the URL. Production and testnet token namespaces and keys are disjoint.

- The bot currently uses long polling, so there is no public Telegram webhook

endpoint to authenticate. If deployment later switches to webhooks, require HTTPS, validate Telegram's webhook secret header, constrain the route, and preserve update idempotency.

- No dynamic SQL built from user input, arbitrary URL fetch, user file upload,

shell execution, or general-purpose external tool was found in the reviewed Coach path. Preserve this narrow capability surface.

- Production OpenAPI/docs exposure, trusted-host/proxy settings, health checks,

Railway service visibility, and final network policy belong to the Release & Operations gate and must be verified against the deployed service rather than inferred from repository code.

Blanket references — do not duplicate

- OpenAI payload minimization, retention controls, and explicit provider

storage behavior remain owned by PRIV-03.

- Conversation-body logging and its retention/access boundary remain owned by

PRIV-02; SEC-09 adds the security-event and incident-response contract without duplicating that privacy fix.

- Expiry and revocation of completion/pulse bearer links remain owned by

PRIV-08.

- Authorization for the mutating `/user/time-slots` endpoint remains owned by

PRIV-10 and must be resolved before that endpoint is exposed.

- The pulse endpoint and legacy agent/router artifacts belong to the accepted Legacy Reachability Cleanup, not to a new security subsystem.

- Telegram /spawn is currently allowlisted through ADMIN_IDS, rejects non-admin callers, and caps creation at 20 tasks; no new finding is required.

- CI, full test execution, health/readiness endpoints, graceful shutdown, Railway verification, and a restore drill belong to the final Release & Operations gate after product implementation. SEC-08 defines the security acceptance criteria for dependencies and the final container at that gate.

Legacy Reachability Cleanup Findings

Legacy Reachability Cleanup Area — Audit Round 2026-08-15

Scope and method

This area does not audit product behavior and does not discover dead code. The audit already owns nearly all of it: `RT-05` is a six-item dead-code inventory, `FSM-03` owns the `SCHEDULE_ADJUSTMENT` zombie subsystem, `FSM-10` the unreachable mutation architecture, `UX-09` the reachable-but-abandoned engagement layer, `PRIV-09` and `DB-10` the dead sensitive schema, `TEL-09` the failure-inference model, and `LIF-05` the dead completion CTA.

What no document currently holds is the **removal order**. Deletions in those findings depend on one another, several are only safe after an accepted decision lands, and two would be re-written if removed early. This area supplies that sequencing, plus the small amount of new reachability evidence found while building it.

Method: import-graph sweep over all 57 `app/` modules; caller sweep per symbol across `app/` and `tests/`; and producer-side verification for data-driven branches, since a branch keyed on a value no producer writes is unreachable regardless of its guards.

Rules for this area:

1. **Severity is owned by the source finding.** This area never assigns or changes one, except for findings that originate here.
2. **No new founder decisions.** Where a decision exists it is applied, not re-opened; where one is explicitly open it stays open.
3. **Tests are not callers.** A test that is the only caller proves the code runs, not that it is reachable. Delete tests with the code; repointing them at a replacement is how this material survived (`FSM-07`).

Reachability taxonomy

Three states are routinely collapsed into "legacy" and require different handling. The distinction is the substance of this area:

Category	Meaning	Handling
Dead	no caller, or keyed on a value no producer writes	delete
Registered but unreachable	handler is registered with the framework, but nothing live produces the input that reaches it	delete — registration is not a working feature
Reachable but rejected	runs in production, but the product decision has retired it	delete per the owning decision, not per reachability

The middle category is the trap. A registered `aiogram` callback handler looks live in every editor and every grep; whether it can be entered depends on who produces the `callback_data`, which is a different file.

Findings

LEG-01 — Producer-side proof that the legacy worker envelope cannot be entered

Severity: evidence only; severity owned by `FSM-10` Status: confirmed

`FSM-10` establishes that the legacy worker-envelope architecture is dead. The decisive evidence was not previously recorded and is supplied here: **no worker emits any of the three envelope keys**. A grep for `plan_updates|generated_plan_object|transition_signal` across `app/workers/` returns zero hits, while `handle_incoming_message()` reads all three (`app/orchestrator.py:1489-1620`).

The branches are therefore not rarely-taken paths that might still fire under unusual input — they are keyed on values nothing writes. This closes `FSM-10` from the producing side and removes any need for a runtime experiment to confirm it before deletion.

LEG-02 — `_resume_plan_if_paused` is an unlisted orphan in the `FSM-03` tunnel

Severity: owned by `FSM-03` Status: confirmed

`_resume_plan_if_paused()` (`app/orchestrator.py:154`) has **zero callers**, including tests. It is tagged `[SCHED_ADJ]` and belongs to the `SCHEDULE_ADJUSTMENT` tunnel, but does not appear in `FSM-03`'s component inventory. Delete it with that subsystem so the tunnel removal is complete.

LEG-03 — Six unimported modules and one orphan symbol

Severity: P2 cleanup Status: confirmed

An import-graph sweep across all 57 `app/` modules found six that production application code does not import, excluding the `app/main.py` entrypoint:

Module	Lines	Disposition
<code>app/logic/__init__.py</code>	1	empty package — delete
<code>app/logging/llm_response_logging.py</code>	59	delete — see LEG-04
<code>app/plan_drafts/activation_alignment.py</code>	126	unimported draft-alignment module alongside the live <code>app/plan_activation/activation_anchor.py</code> (used at <code>app/plan_finalization.py:211</code>); delete after a final import and behavior-ownership check
<code>app/plan_parser.py</code>	157	test-only parser for the retired free-form <code>/plan</code> command; delete with <code>tests/test_plan_parser.py</code> rather than preserving a second plan-entry model
<code>app/plan_normalizer.py</code>	164	test-only normalizer for the retired generated-plan payload; delete with <code>tests/test_plan_normalizer.py</code> after confirming no migration or offline fixture still consumes it
<code>app/content_library.py</code>	56	not a deletion candidate — <code>CONTENT-01</code> owns its loader validation; ownership is Content implementation

Additionally, `_detect_foreign_instructions()` (`app/workers/coach_agent.py:772`) has **zero callers**. `SEC-05` states it would not be an authorization boundary even if wired up. Delete it, so that it cannot later be "just enabled" as a jailbreak fix.

LEG-04 — An unimported logger would emit model output text

Severity: P2 cleanup; coordinate with `PRIV-02` and `SEC-09` Status: confirmed

Current behavior: `log_llm_text_candidates()` (`app/logging/llm_response_logging.py:24`) builds `"preview": part_text[:300]` — up to 300 characters of model output — and emits it as a JSON log line. The module is currently unimported, so nothing leaks today.

Expected behavior: conversational content does not reach logs, and no dormant code exists that would start emitting it on a single import.

Minimal fix: delete the module rather than leaving it as inert debug tooling. This is a privacy cleanup rather than tidiness, and `SEC-09`'s redaction allowlist should be written so that re-introducing content-bearing previews fails review. `PRIV-02` continues to own live conversation-body logging.

Removal-ordering index

Rationale and severity belong to the owner column; this index adds only order and dependency.

#	What	Owner	Depends on
1	<code>start_plan</code> : buttons in the completion report (<code>app/orchestrator.py:629,633</code>) — remove the buttons, add no handler	LIF-05	none
2	<code>app/logic/</code> , <code>llm_response_logging.py</code> , <code>_detect_foreign_instructions</code> , <code>activation_alignment.py</code> , <code>plan_parser.py</code> , <code>plan_normalizer.py</code> , and the two test-only legacy plan tests	LEG-03, LEG-04	final import/fixture check
3	The whole <code>SCHEDULE_ADJUSTMENT</code> tunnel — dispatcher, four handlers, keyboard builders, <code>sched_task:sched time:timeout</code> callbacks, Redis keys, <code>stuck_schedule_adj_check</code> job, FSM state, tests — plus <code>_resume_plan_if_paused</code>	FSM-03, LEG-02	none; do not partially retain
4	Legacy worker-envelope architecture; then re-check whether <code>app/plan_adaptations.py</code> retains a live caller	FSM-10, LEG-01	after 3
5	<code>AIPlanVersion</code> — already has no reader; the single writer is <code>app/plan_adaptations.py:252</code> , so the table becomes fully dead once the adaptation path is removed	FSM-10, DB-10	after 4
6	<code>create_first_plan</code> , <code>IDLE_ONBOARDED</code> , <code>IDLE_DROPPED</code> , <code>app/fsm/guards.py</code>	RT-05 items 1–3, 6	inside the <code>FD-08</code> migration

#	What	Owner	Depends on
7	UserFact/FactCategory, UserDailyLog — verify production rows are empty, then drop by migration	PRIV-09, DB-10	with 6
8	FailureSignal and the failure-inference model	TEL-09	independent
9	Reachable-but-rejected engagement layer: pulse, persona, streak, comeback, silence checks, adapt_suggest	UX-09	independent
10	Preview/deep-link/action tail: _PLAN_ACTIONS, newplan_*, build_plan_draft_preview, show_plan_actions, plan_draft_parameters	FSM-10, UX-10	after 4

Step 3 must not be partially retained

The `sched_task:` and `sched_time:` handlers in `app/telegram.py:297` and `:350` are **registered but unreachable**. The only initial producer of their `callback_data` is `_build_task_select_keyboard/_build_time_select_keyboard` called from `_handle_schedule_adjustment_init` (`app/orchestrator.py:203`) and `_handle_schedule_adjustment_record(:260)` — both inside the dispatcher `FSM-03` already establishes has no production caller. Once entered, the handlers rebuild their own keyboards (`app/telegram.py:329, 343, 390`), which makes the subsystem appear self-sustaining when read in isolation. It is a closed loop with no entrance.

Consequently the live-looking UI must not be preserved, and must not be rewritten on top of `change_day_time / change_evening_time:` either would restore a tunnel `FSM-03` has decided to remove. A replacement time picker is separate future work, already specified as `MISC-02` — buttons calling runtime tools directly with no LLM round-trip — and is built new rather than recovered from this code.

Open decision — MORNING is not a cleanup item

`MORNING` is deliberately absent from the index above. `FD-03` keeps it as frozen internal metadata, and `RT-05` item 5 leaves an explicit open choice: remove it, or keep it frozen behind a single guarded definition instead of branches scattered across roughly seven files. That question is undecided, and this area does not resolve it. It requires a founder decision before it can enter any cleanup pass.

Handed to the Security area

Two items surfaced while counting Coach entry points. They belong to `SEC-06` and are recorded here only as their origin:

- **Coach entry points are broader than inbound text.**

`handle_incoming_message()` is called from seven sites in `app/telegram.py`, of which only `:276` is user-authored text; the remainder are `/start(:151)` and inline buttons (`:290, :308, :366, :399, :640`). Steps 3, 9, and 10 above remove five of the six non-text entries. The ordering consequence matters: deterministic callbacks bypassing Coach is the **end** state, not the current one, so an admission check written before that cleanup must identify user-authored turns by an explicit parameter rather than by assuming callbacks never reach Coach.

- **MAX_TOKENS is deployment-configurable, not remotely controlled.**

`max_output_tokens` resolves to `settings.MAX_TOKENS` (`app/config.py:60`; used at `app/ai.py:88` and `app/workers/coach_agent.py:993`) via the unvalidated numeric parser described in `SEC-03`. There is no attacker or user path to it; the defect is missing range validation on a value that bounds per-request output, so a deployment typo silently raises the only output ceiling that exists. It does not bound total spend.

Blanket references — do not duplicate

- Component inventories for the `SCHEDULE_ADJUSTMENT` tunnel and the legacy

mutation architecture remain owned by `FSM-03` and `FSM-10`; this area adds ordering and two missing components only.

- The dead-code inventory itself remains owned by `RT-05`.
- Dead sensitive schema remains owned by `PRIV-09` and `DB-10`; live

conversation-body logging remains owned by `PRIV-02`.

- Content Library loader ownership remains with `CONTENT-01` under `FD-10`.
- The abandoned engagement layer remains owned by `UX-09`, and obsolete plan and

schedule controls by `UX-10`.

- Rate limiting, concurrency, and cost controls remain owned by SEC-06 under FD-17; callback race atomicity by SEC-07.
-

Company Deployment Findings

Company / B2B Onboarding Area — Audit Round 2026-08-15

Scope and method

This area covers everything required to launch with a real company and learn something trustworthy from the result: the organization/deployment record, the roster/SSO enrollment gateway accepted in FD-18, revocable entitlement, one-time Telegram handoff, attribution, `eligible_count`, timezone mode, privacy-notice binding, testnet isolation, operational controls, and the pre-launch smoke test.

Method: grep for any company, organization, tenant, employer, or deployment concept across `app/`; read the pre-existing deployment funnel in `product_contract.md` §2.10 and the B2B privacy contract in §2.9 together with the later accepted FD-09 and FD-18 amendments; and check the existing `/start` and deep-link paths for anything attribution-shaped.

Result of the sweep: zero. `company`, `organization`, `employer`, `deployment_id`, `org_id`, and `tenant` return **no matches anywhere** in `app/`. There is no model, no column, no identifier, and no attribution. This area therefore audits an accepted contract against an empty implementation rather than against defective code.

Severity has two gates. Before the individual developer beta, implement the isolated testnet plus the shared deployment/entitlement/token/enrollment spine; the founder-issued test entitlement replaces roster/SSO only at the acquisition edge. Before the first company production deployment, add production roster, OIDC, reconciliation, company legal/configuration, and launch controls. The accepted sequencing still tests the product with individual developers first, but it no longer does so inside production or on a disposable data model.

Why this area is load-bearing

C1 — will employees open a corporate tool — is the pre-registered kill/continue test, and `product_contract.md` states it cannot be tested by interview, only by live deploy. Under FD-18, its accepted measurement is `deployment_start_rate = distinct deployment_enrollment_created users / eligible_count_at_launch`. A raw `/start` update and a successfully attributed company enrollment are separate events and must not be substituted for one another.

Both sides of that fraction are missing. The numerator needs attribution that does not exist; the denominator needs an `eligible_count` that has nowhere to live. A company deployment run today would produce a number that cannot be computed, on a cohort that cannot be identified, against a baseline nobody recorded — and the founder would face a continue/kill decision with no evidence, having spent the scarcest asset in the plan: a real HR relationship.

Findings

COMP-01 — No organization or deployment record exists

Severity: BLOCKER before individual testnet beta (shared data spine) Status: confirmed absent

Current behavior: no organization, deployment, or tenant entity exists in the schema or the code. Users are created by `_ensure_user()` from a Telegram identity alone (`app/telegram.py`), with no field describing where they came from or which company context they belong to.

Expected behavior: one `deployment` record is the unit of a corporate launch and the join key for every other item in this area. It holds at minimum an internal ID, the organization it belongs to, operational dates and controls, accepted roster version and reconciliation cadence, timezone mode/default, the privacy-notice version in force at launch, a named champion contact, and environment identity. It does **not** hold one public enrollment token.

Minimal fix: add `organization`, `deployment`, `versioned roster/access-identity`, `access_entitlement`, `deployment_invitation`, and `deployment_enrollment` records. Do not add `user.deployment_id`: people can move between employers or individual/company access over time, while historical attribution belongs on events and must not be rewritten with current membership.

COMP-02 — User acquisition is unattributed, so C1 cannot be computed

Severity: BLOCKER before individual testnet beta for token attribution; production roster/SSO linkage before the first company deployment Status: confirmed absent

Current behavior: `cmd_start` (`app/telegram.py:105`) parses only the `newplan_` argument and otherwise discards `/start` payloads. A user arriving from a company link is indistinguishable from an organic one. No `bot_started` attribution is recorded anywhere.

Expected behavior: the invitation token is captured at `/start`, resolved, and validated against its active entitlement before it creates a `deployment_enrollment` record. The same accepted operation emits `deployment_enrollment_created`; `bot_started` remains the raw Telegram-entry event and is not the company-start numerator. Attribution is recorded on later events with the deployment and enrollment that were current **at the moment of the event**, so the accepted deployment funnel can be computed per launch.

Minimal fix: capture and resolve the token in `cmd_start`; create one enrollment; emit idempotent `bot_started` and `deployment_enrollment_created` events with their distinct meanings.

Do not put a permanent `deployment_id` on User. An earlier draft of this finding did, and it was wrong: people may change employers, replace an individual entitlement with a future company entitlement, or take part in a second pilot. A lifetime foreign key on the user conflates *how this person entered* with *which employer relationship is current*. The correct shape is a narrow enrollment record:

```
user_id, deployment_id, entitlement_id, enrolled_at, ended_at,
status, attribution_source
```

with **one active enrollment at a time**. What must be immutable is the attribution stored **on historical events**, not the user's employer membership. Moving to a new deployment is an explicit act that opens a new enrollment and does not rewrite prior history; re-opening the same invitation is idempotent.

Do not treat a deep-link open as an observed event: §2.10 states explicitly that opening a link without pressing Telegram Start is not observable and must not be presented as measured.

COMP-03 — `eligible_count` has no home, no integrity, and no as-of date

Severity: P1 before the first company deployment Status: confirmed absent

Current behavior: nothing stores `eligible_count`. It exists only as a term in the accepted contract.

Expected behavior: `eligible_count` is **not the company's total headcount**. It is the number of active, deduplicated, access-eligible corporate identities in the accepted roster version for this deployment at launch. That forms the denominator of C1; announcement exposure is recorded separately because a roster does not prove that an employee saw the launch message.

Minimal fix: record it once per deployment, at launch, as a small fixed set:

```
eligible_count_at_launch, roster_version_id, source, as_of,
announcement_channel, announcement_at
```

A versioned roster history records import mode and is required because later reconciliation changes entitlements but must not rewrite the launch denominator used for the original decision. Record the champion's announcement confirmation and timestamp separately. Never convert roster membership, SSO authorization, or tokens issued into `people_reached` by assumption.

COMP-04 — The FD-18 roster/SSO enrollment gateway is not implemented

Severity: BLOCKER before individual testnet beta for entitlement/token/enrollment; production roster/SSO gateway before the first company deployment Status: confirmed absent; implements FD-18

Current behavior: the only deep link that exists is `newplan_*`, which encodes plan parameters in clear text and is itself legacy flow scheduled for removal (Legacy Reachability Cleanup, step 10). There is no enrollment mechanism.

Expected behavior: the company provides a current email roster. On the deployment enrollment page, Google or Microsoft OIDC preferably proves control of the roster-listed identity; a one-time email challenge supports other providers and personal addresses listed by the company. The backend matches the verified address to the active roster and creates or retrieves one entitlement. It then mints one entitlement-bound Telegram token, shows the raw value once for immediate open, and stores only the hash. HR never assigns tokens and never receives a token list.

Redemption validates token and entitlement in the same transaction, consumes the token, and creates or resumes one enrollment. The restricted identity-to-entitlement-to-enrollment mapping survives because roster reconciliation must be able to revoke company-funded access. Returning users resume through the account while the entitlement remains active; invalid invitation handling does not silently create an unentitled production user.

Minimal fix: implement safe roster import/reconciliation, minimum-scope Google and Microsoft OIDC plus the email-verification fallback, access identity and entitlement records, handoff-token hash and state, atomic redemption, enrollment creation, neutral failure response, no-store/log-redaction controls, and the isolated testnet issuance path. Build it after the legacy `newplan_*` grammar is removed. The product-level contract remains in FD-18; this finding does not restate or reopen it.

COMP-05 — Deployment lifecycle and entitlement are undefined

Severity: BLOCKER before individual testnet beta for entitlement lifecycle; full roster reconciliation before the first company deployment Status: confirmed absent; implements FD-18

Current behavior: nothing expresses that a company launch starts, pauses, or ends, and no company enrollment or entitlement exists.

Expected behavior: a deployment carries **orthogonal operational fields**, not a single lifecycle enum:

`enrollment_open, delivery_enabled, starts_at, ends_at, renewal_due_at`

Minimal fix: implement these fields directly. A single `draft/active/paused/ended` enum was rejected because it collapses three independent axes — whether new people may join, whether delivery runs, and the commercial period — into one dimension, which then forces invalid combinations and has to be unpicked later. Pausing delivery during an incident and closing enrollment at the end of a launch window are different acts and must be separately expressible. Individual invitation state (`available, claimed, redeemed, expired, revoked`) belongs to invitation records, not one deployment-level `invite_status`.

`access_entitlement` separately carries at least `granted_at, revoked_at, source`, and the roster version that last confirmed it. Roster cadence is not an entitlement expiry: no roster or a rejected import leaves every existing entitlement unchanged. Only omission from an explicitly confirmed `full_snapshot`, an explicit revoke operation in an accepted `delta`, an authorized manual request, or the deployment's commercial end revokes company-sponsored access. Commercial renewal extends the deployment centrally without SSO, another token, or user action. The personal account and history remain. Future individual paid access requires its own payment, verified-email, and personal-entitlement path; it is not part of MVP and is not merely an undecided price.

COMP-06 — Deployment timezone mode is decided but not implemented

Severity: P1 before any deployment outside `Europe/Kyiv` Status: confirmed; product decision made 2026-07-15, not implemented

Relocated from MISC-01 now that this area exists, per the Miscellaneous section's own filing rule.

Current behavior: every new user is silently defaulted to `Europe/Kyiv`, and timezone is never collected anywhere.

Expected behavior (decided 2026-07-15, clarified 2026-08-15): the timezone question is asked **of the company, at company onboarding** — is everyone in one timezone, or is the team distributed? The answer decides whether the employee is asked anything at all:

- **single timezone** — the company has already answered for everyone, so the

deployment default is applied silently and the employee is never asked;

- **distributed** — the employee sets their own timezone during their

onboarding. This is deliberate: asking the one person who knows is cheaper and more accurate than making the company attach timezone data to its roster or produce a list of per-office defaults, and it is the only option that handles remote employees who belong to no office.

A per-user override exists in Settings for travel; there is no automatic geolocation or travel detection in MVP.

Minimal fix: two deployment fields — `default_timezone` and a mode flag (`single / distributed`) — set at company onboarding. In `single` mode resolve the user's timezone from the deployment at attribution time; in `distributed` mode add one timezone step to employee onboarding. The per-user override remains available in both. No office hierarchy or per-office default list is required.

Not a correctness bug today under a single-timezone launch — it becomes one the moment a company outside that timezone signs up, and it fails silently by delivering exercises at the wrong local hour, which reads to the employee as a broken product rather than a misconfiguration.

COMP-07 — The privacy notice is not bound to a deployment

Severity: P1 before the first company deployment Status: confirmed absent; extends `PRIV-01` into the company dimension

Current behavior: `PRIV-01` already owns the absence of a privacy notice and notice-version record. Nothing connects a notice version to a company launch.

Expected behavior: the notice version in force is pinned on the deployment at launch and on each user at acknowledgement. When the notice changes, existing deployments keep the version their employees actually accepted until re-acknowledgement, rather than retroactively appearing to have accepted new terms.

Minimal fix: record `notice_version` on both the deployment and the user acknowledgement, and define the re-consent trigger. The notice and company agreement state the FD-18 purpose boundary: roster and corporate identity are used for access, licensing, revocation, support, fraud prevention, and security, not employee scoring or company-facing individual behavior. The user-facing notice mechanism itself remains owned by `PRIV-01`; only the deployment binding and access-identity purpose are new here.

This matters commercially as well as legally: the first thing an HR buyer's legal contact asks is which version of the notice their employees agreed to, and "the current one" is not an answer if the text has changed since launch.

Legal review is required before the first company deployment — this must be decided, not deferred. An earlier position deferred all data-protection paperwork until a paying client existed. That is wrong: an unpaid pilot still processes employee personal data, so the absence of an invoice changes nothing.

The roles are genuinely unclear and are determined by the actual purposes and means of processing rather than by what a contract calls them. Because Love Yourself uses the data for its own telemetry and product learning, it may be a separate controller at least for that part, rather than a pure processor acting on the company's instruction. The likely outcome is that **both** are needed — an employee-facing privacy notice and a contractual delineation of responsibility with the company — but the exact construction is a lawyer's call, not this audit's.

What this area records is only the gate: **no company deployment starts until that determination exists.** ISO, SOC 2, and large security questionnaires remain correctly deferred until traction or a concrete procurement request.

COMP-08 — Test and demo deployments would pollute real metrics and aggregates

Severity: BLOCKER before individual testnet beta Status: confirmed absent

Current behavior: no test/production distinction exists anywhere. Founder testing, demos, and screenshots would be indistinguishable from real employee behavior.

Expected behavior: FD-18 testnet and production are isolated environments built from the same artifact, not rows mixed in one production database. They use different databases, Redis namespaces, Telegram bots, OpenAI projects, token keys, URLs, backups, alerts, and aggregate sinks. An immutable environment ID is still attached to deployments/events as defense-in-depth; production rejects testnet tokens and vice versa.

Minimal fix: create both environment configurations and enforce isolation at startup, token validation, persistence, telemetry ingestion, backup, and reporting. This is effectively unfixable after contamination: `FD-09` aggregates are deliberately non-reversible and retain no join key back to the personal event, so a test run entering production aggregates cannot be subtracted afterwards.

COMP-09 — No deployment-scoped operational controls or support path

Severity: P1 before the first company deployment Status: confirmed absent

Current behavior: `SEC-09` requires global emergency controls for Coach and scheduled delivery. There is no way to act on a single company: no way to pause one deployment's delivery, and no defined support route for an employee who has a problem or wants their data deleted.

Expected behavior: delivery and enrollment can be paused for one deployment without affecting others, and every deployment has a named champion contact on the company side plus a stated support route for employees.

Minimal fix: scope the pause control by deployment; record the champion contact on the deployment record; state the employee support and deletion route in the onboarding copy. The deletion mechanism itself remains owned by the Privacy round — this finding only requires that a company's employees know where to send the request.

The champion contact is a product requirement, not administrative tidiness: the accepted product position is that a champion is needed for install and that the ritual must then self-sustain. A deployment with no named human on the company side is a launch with no way to diagnose zero activation.

COMP-10 — No pre-launch verification for a company deployment

Severity: P1 before the first company deployment Status: confirmed absent

Current behavior: nothing verifies that a deployment works end to end before the enrollment page is opened to employees.

Expected behavior: a fixed smoke test is executed through the isolated **testnet** on the exact release artifact and its result recorded before any production enrollment link is announced. A final production-safe check verifies configuration without creating behavioral production data.

Minimal fix: adopt the checklist below and require a recorded pass. The asymmetry justifies the ceremony — the smoke test costs under an hour, and a failed first impression on the first HR relationship cannot be retried.

Company Deployment Checklist

The operational artifact this area exists to produce. Run per deployment; keep the completed copy with the deployment record.

Before production enrollment is enabled

1. Organization record exists: legal name, champion contact with a working direct channel, and the agreed support route for employees.
2. **Legal determination in place** — roles, employee notice, and any contractual delineation (COMP-07). This is a gate, not a formality.
3. A normalized and deduplicated roster version with explicit `full_snapshot` or `delta` mode is validated, previewed, explicitly accepted, and bound to the deployment; the launch full snapshot's active count becomes `eligible_count_at_launch` (COMP-03).
4. Timezone mode confirmed: single or distributed (COMP-06).
5. Privacy-notice version pinned to the deployment (COMP-07).
6. Pilot terms recorded without assuming a price model: free/paid status, pricing basis if any, `starts_at / ends_at`, annual renewal boundary, and the observation window. Roster-reconciliation cadence and owner are recorded separately and never act as an automatic entitlement-expiry boundary.
7. **The pilot question written down** — this pilot tests C1 and C3. No health, wellbeing, stress, or productivity outcome is promised or measured.
8. What the company receives after the pilot, and what happens to delivery and data when it ends, agreed in writing (COMP-05, post-pilot report).
9. Subprocessor list and a short privacy/security factsheet ready — these get asked for, and improvising them is how over-promising happens.
10. Production and testnet environment IDs, credentials, stores, token keys, aggregate sinks, and bot identities verified as distinct (COMP-08).

Before employees can enroll

11. Smoke test executed on **testnet**, end to end on a real phone: founder test entitlement → raw handoff token displayed once → Telegram Start → enrollment created → onboarding completes → first exercise delivered at the correct local time → response registered → Coach replies → cycle summary renders.
12. Production FD-18 gateway verified with test identities: supported OIDC and roster-listed email-verification fallback succeed; absent, wrong-domain, expired, revoked, and malformed identities fail neutrally; raw token is absent from logs and later views; one redemption only; an already-enrolled user resumes without repeated SSO or another token. Roster tests prove both directions: no roster or an invalid import changes nothing, while an accepted `full_snapshot` revokes only omitted entitlements and an accepted `delta` changes only explicit rows. Neither path deletes account/history or exposes behavior to the company, and a missing mode is rejected.
13. Delivery timing verified in the employee's resolved timezone, not the server's.
14. **Tested on a real corporate device and network** — Telegram reachable, not blocked by policy or VPN (C5).
15. Language and supported-device baseline confirmed for this workforce.

16. Company confirms the enrollment page will be placed in an existing private employee channel and that the selected OIDC or email-verification path is allowed on corporate devices.

17. Deployment-scoped pause verified to stop delivery for that deployment only (COMP-09).

18. Rate-limit and cost controls active per FD-17; provider budget alert reachable by a human who is awake.

19. Love Yourself operational owner and incident contact named for this deployment.

At launch

20. Baseline recorded: launch timestamp, accepted roster version/count, initial entitlement/invitation/enrollment counters, notice version, content version, private announcement channel, exact launch copy, and intended reminder count.

21. Champion confirms when the enrollment page was announced to the launch roster; roster membership alone is not treated as message exposure.

22. Champion briefed on what employees receive and what the company will and will not see (FD-09 §2.9).

After launch

23. Funnel reviewed internally at D1, D3, D7 per deployment: eligible roster, successful identity authorizations/invitations, redemptions/enrollments, onboarding conversion, first delivery success, response and completion. This internal founder view is not the company report.

24. Failures separated from rejection — SSO/roster denial, issued-but-unredeemed invitations, undelivered messages, blocked bot, and wrong-timezone delivery are not evidence about the same funnel step or about product value.

25. **Abnormal weeks recorded explicitly** — public holidays, a release crunch, an outage, or mass vacation. A pilot that lands on one of these produces misleading C3 data, and without a note it will later be misread as the product failing.

26. Deployment retrospective recorded before the next launch, so the second company benefits from the first.

27. Roster-reconciliation due dates produce founder/champion reminders and an overdue alert. Missing or rejected input never disables existing users.

Method note. A synchronous launch — everyone scanning a QR code together in a room — is a *different experiment*, not an improvement to this one. It will raise activation and will not answer C1 as pre-registered, which is specifically whether people authorize and open a corporate tool after the enrollment page appears in their normal private company channel and the decision is theirs. Run it as a deliberate follow-up if the cold result is poor; do not mix the two.

Post-pilot company report — implementation of FD-09

After the pilot, and only once FD-09's **≥100 eligible / ≥50 distinct actual contributors** gate is met, the company may receive one aggregate product-usage summary:

Employees eligible at launch:	200
Deployment enrollments created:	150
Active in the stated 14-day window:	90

The report prints the exact 7- or 14-day window and the FD-09 definition of *active*: at least one accepted exercise response or one user-authored Coach turn after onboarding. It does not count automated delivery or continuation.

Not provided, at any cohort size: names; teams, departments or offices; completion and skip breakdown; separate Coach/exercise counts; Coach text; invitation-level status; weekly or finer dynamics; any real-time dashboard; any ability for HR to lower the privacy threshold.

Do not defend this by claiming an aggregate count is inherently anonymous. It is not — a single number can identify through a small group or through what the employer already knows. The protection here is the combination of the existing gate, a single post-pilot delivery rather than live monitoring, and the absence of any slice. The public wording remains the one required by §2.9: aggregate data without individual or small-group views.

The reason this boundary is necessary is commercial as much as ethical. A buyer asking whether their people use the product is asking a legitimate question, and answering "that is private" is both untrue and fatal to renewal. Defining the answer in advance is what prevents it being improvised, and over-shared, under pressure on a sales call.

Accepted scope in this round

- Production enrollment implements the FD-18 roster/SSO gateway, revocable entitlement, and one-time Telegram handoff; HR supplies the roster but never assigns or receives handoff tokens.
- Individual beta runs on the isolated FD-18 testnet with founder-issued test entitlements and the same post-redemption product behavior.
- Telegram remains the employee delivery channel for MVP;

`deployment.channel` is recorded, while the company's internal enrollment page placement is a deployment field and discovery input.

Open founder decisions

Listed, not decided. Each changes what gets built:

1. **Pilot pricing and unit of sale** — the first company pilot may be free, flat-fee, or use another agreed basis. This audit does not decide it and no per-seat billing system is built from an assumption.

Blanket references — do not duplicate

- The company-facing analytics boundary, the aggregate layer, and the 100-eligible/50-contributor privacy gate remain owned by FD-09; this area never widens what a company may see.
- The privacy notice, notice-version record, and consent mechanism remain owned by PRIV-01; the data-deletion mechanism by the Privacy round. COMP-07 and COMP-09 add only the company binding and the support route.
- OIDC and opaque handoff-token security originate in the Security round; FD-18 owns the roster/entitlement/privacy model and COMP-04 implements it.
- Global kill switches, alerting, and incident response remain owned by SEC-09; COMP-09 adds only per-deployment scoping.
- Rate limiting, concurrency, and cost control remain owned by FD-17 and SEC-06.
- Removal of the legacy `newplan_*` deep link remains owned by the Legacy Reachability Cleanup index, step 10.
- Controller/processor roles, necessary contractual terms, and whether a DPIA is required are a legal gate under COMP-07 before the first company deployment. ISO/SOC certification, large procurement questionnaires, and an HR dashboard remain deferred until traction or a concrete buyer request.
- `product_contract.md` §2.9/§2.10 still contains the pre-FD-09/FD-18

company-reporting and shared-deep-link model. Updating that document and the website remains the accepted final documentation/product-language gate after implementation stabilizes. Until then, FD-09, FD-18, and the Telemetry Event

Contract in this audit govern implementation; the stale contract text must not be used to recreate a shared company credential, anonymous dispenser, or company-view-zero rule.

Release & Operations Findings

Release & Operations Area — Audit Round 2026-08-15

Scope and method

This area is the final release gate after the accepted product and code changes have been implemented. It covers:

- process topology and Railway deployment behavior;
- startup, health, readiness, and graceful shutdown;
- schema migration, rollback, backup, and restore;
- reproducible builds, tests, CI, and configuration;
- monitoring, incident diagnosis, and release provenance;
- delivery capacity and external-provider degradation.

The review inspected `app/main.py`, `app/api.py`, `app/db.py`, `app/scheduler.py`, Docker and process declarations, dependency files, the test suite, and the current Railway metadata observed during the Database round. It did not mutate production or inspect the stopped physical production schema.

Accepted operational posture

The MVP runs as **one bot process, one Telegram polling owner, and one scheduler writer**. It does not introduce leader election, multiple replicas, a durable message broker, or speculative channel infrastructure before those become necessary.

The existing delivery grace remains **two hours**. This is accepted behavior, not a new founder decision in this round. A delivery must nevertheless never cross the step's own `expires_at`; the effective deadline is:

```
min(scheduled_for + 2 hours, expires_at)
```

No scheduling jitter is added for MVP. If the capacity test in OPS-11 fails, a controlled delivery window is a known later lever, not an implicit change to the user's selected time.

Scheduled exercise delivery is deliberately deterministic and does not call OpenAI at delivery time. That property is preserved explicitly in OPS-12.

Findings

OPS-01 — Railway deployment can violate the single-runtime-owner invariant

Severity: BLOCKER before production release Status: confirmed

Current behavior: `app/main.py` starts Telegram long polling, APScheduler, and Uvicorn in one process. APScheduler uses a persistent SQLAlchemy job store in the same PostgreSQL database as application data. Railway deployment handover can briefly overlap the old and new processes, while both Telegram polling and the scheduler require one authoritative owner. DB-20 already accepts one replica / one scheduler writer for beta, but no release procedure enforces it.

Expected behavior: at every moment there is at most one polling owner and one scheduler writer. A release cannot rely on a nominal replica count while old and new deployments overlap during handover.

Minimal fix for MVP: keep one replica, disable unattended production autodeploy, and use a controlled `stop old → deploy/start new → verify ready` procedure. The brief downtime is accepted for beta. Record the actual Railway service settings and verify from logs that no overlap occurred. Add a leader lease or move scheduling behind one external owner only when zero-downtime or multiple replicas become a real requirement.

OPS-02 — Application startup mutates schema and there is no canonical migration runner

Severity: BLOCKER before the first database-changing release Status: confirmed

Current behavior: `main()` calls `init_db()`, which executes `Base.metadata.create_all(bind=engine)` on every process start. This can create missing tables but cannot safely evolve, backfill, contract, or prove the version of an existing production schema. SQL migration files and ORM metadata do not form one enforced ledger or release command.

Expected behavior: application startup verifies schema compatibility and fails closed; it does not perform schema mutation. Database changes run once, separately from application startup, after backup and restored-copy rehearsal.

Minimal fix: establish one migration ledger and one migration command; remove `create_all()` from production startup; add a read-only schema/version check. Use `expand` → `backfill` → `switch` → `contract` for incompatible changes. Establish this process **before** the FD-08 lifecycle rebuild, event-ID split, sensitive column removal, or legacy table drops — the first large migration must not be the rehearsal.

`apscheduler_jobs` is scheduler-owned infrastructure in the shared database. Exclude it from application-table cleanup and migration ownership; operate on it only through the explicit scheduler recovery procedure in OPS-07.

OPS-03 — The process has no truthful liveness or readiness contract

Severity: BLOCKER before production release Status: confirmed

Current behavior: `app/api.py` has no liveness or readiness endpoint, and Uvicorn binds to hardcoded port `8000` instead of Railway's injected `PORT`. The HTTP server can remain reachable while polling or the unreferenced scheduler task has already died, so an HTTP-only check would report a healthy product that no longer delivers exercises.

Expected behavior:

- `/live` proves the process and event loop are alive;
- `/ready` proves the schema version is compatible, required PostgreSQL and

Redis dependencies are reachable, and the polling and scheduler owners are initialized and still running;

- the service binds to `PORT` with a local fallback;
- health checks are read-only and do not create schema or user data.

Minimal fix: expose both endpoints, retain observable task handles, configure Railway's deployment health check against `/ready`, and add continuous application monitoring because Railway's deployment health check alone does not prove that polling and scheduling remain alive after rollout.

OPS-04 — Core tasks and graceful shutdown have no one lifecycle supervisor

Severity: P1 before beta Status: confirmed

Current behavior: `asyncio.create_task(schedule_daily_loop())` discards the task handle; polling and Uvicorn are joined with `asyncio.gather`; scheduler shutdown is defined but never called. `APScheduler` is a `BackgroundScheduler` whose thread-pool jobs cross into the `asyncio` loop through `run_coroutine_threadsafe()`. A `SIGTERM` can therefore close one concurrency domain while work in the other is still waiting on it.

Expected behavior: one lifecycle supervisor owns polling, HTTP, scheduler, and dependency cleanup. Shutdown follows an explicit order:

1. reject new work and mark readiness false;
2. pause the scheduler so no new jobs begin;
3. keep the event loop alive while bounded in-flight scheduler threads finish;
4. stop polling and HTTP intake;
5. close Telegram, Redis, and database resources;
6. cancel any remainder after a bounded deadline and exit non-zero if shutdown

was incomplete.

Minimal fix: implement one top-level lifecycle/task-group or equivalent supervisor, wire `SIGTERM` once, retain all task handles, call `shutdown_scheduler(wait=True)` while the loop is still alive, and configure a Railway drain/termination window long enough for the bounded shutdown path.

OPS-05 — There is no reproducible passing release-test baseline

Severity: BLOCKER for a failing or partially collected suite; P1 for CI automation Status: confirmed

Current behavior: no canonical environment and command reproduce a complete passing suite. The audit collected 172 tests but hit seven collection errors, including tests that still import removed `app.ai_plans`, missing runtime dependencies in the invoked environment, and Telegram bot construction during collection with an invalid token. A test count is not evidence when part of the suite never collected. There is no `.github/workflows` release check.

Expected behavior: one clean command installs the locked runtime and test dependencies, collects the entire suite, and passes without production credentials. The release baseline covers PostgreSQL and Redis integration, migrations against an

earlier schema, Docker startup, scheduler recovery, delivery idempotency, lifecycle transitions, and the deterministic user smoke path.

Minimal fix: delete or rewrite the stale tests under the Legacy cleanup owner; make imports side-effect free; provide valid test configuration and ephemeral PostgreSQL/Redis services; then run the same command in CI. A manually executed canonical check may support the first founder-only beta, but no production release proceeds with collection errors, and repeated releases require CI.

OPS-06 — The production artifact and Railway configuration are not reproducible

Severity: P1 before beta Status: confirmed

Current behavior: the repository has both a Docker `CMD` and a `Procfile`, no `railway.toml`, no `.dockerignore`, mutable base-image and dependency ranges, and no checked release declaration for health path, restart behavior, replica count, drain time, start command, or pre-deploy migration command. The running service can therefore differ from what repository review implies.

Expected behavior: one build artifact and one start contract define the runtime. Operational settings that change correctness are versioned as code or captured in a release manifest and reviewed with the commit.

Minimal fix: select Docker as the canonical build/start path; remove the competing declaration; pin the runtime and dependencies under `SEC-08`; add a minimal Railway config or verified manifest for `PORT`, health path, restart policy, one replica, drain window, and migration command. Test the built image, not only the host checkout.

OPS-07 — Backup policy exists, but restore is not yet a safe product operation

Severity: BLOCKER before database-changing production work Status: confirmed; implements FD-12 and expands DB-08 / DEL-05

Current behavior: FD-12 accepts scheduled Railway backups, a manual backup before migration, and restore verification. The Railway inspection on 2026-08-12 found no scheduled backup, existing backup, or PITR. In addition, the APScheduler job store lives inside the restored database. Restoring an old snapshot can therefore restore stale date jobs and stale plan state together.

Most old jobs are bounded by the existing two-hour application guard, but a restore can still replay a recent delivery or revive future work from a plan that was changed after the snapshot. The callback currently rejects `completed`, `skipped`, `expired`, and `canceled`, but not `delivered`, so a restored job for an already-delivered step can send the same exercise again.

Required implementation:

- enable FD-12's daily/weekly Railway backups and a manual pre-migration

backup;

- measure and record actual RPO and RTO in one restore drill;
- add `delivered` to the delivery callback's no-send statuses;
- after restore, keep the bot stopped, clear scheduler-owned

`apscheduler_jobs`, reconcile restored user/plan/step state, and recreate only valid future jobs;

- start exactly one runtime owner and verify that no past, canceled, completed,

or already-delivered exercise is sent;

- retain the runbook even after the one-line `delivered` guard, because restore

can revive broader stale plan and enrollment state, not only one job.

OPS-08 — Code rollback, database rollback, and content rollback are conflated

Severity: P1 before the first database-changing release Status: confirmed

Current behavior: Git can restore application code, but no contract describes which schema versions that code can read, whether a migration is reversible, or which content version an already-created plan contains. Exercise notification text is intentionally rendered and stored in the APScheduler job at planning time, while no explicit release provenance ties that snapshot to its source content.

Expected behavior: normal rollback deploys compatible previous code without rewinding user data. Destructive rollback uses the tested restore runbook and is treated as data loss according to the measured RPO. Every release records at least Git SHA, image identifier, schema migration/version, configuration version, and content-library version.

Minimal fix: adopt backward-compatible migrations and a release manifest. Persist the plan/step content snapshot and its **planning-time** content version explicitly. Do not relabel an old scheduled snapshot as the current release version, and do not re-render already-created plans from the newest library during delivery unless a future product decision deliberately changes that immutability contract.

OPS-09 — Monitoring cannot detect product failure before users report it

Severity: P1 before beta Status: confirmed

Current behavior: logs exist, but there is no defined alert or operational view for a dead polling task, dead scheduler, overdue delivery backlog, repeated Telegram failure, dependency outage, cost-circuit-breaker trip, backup age, or crash loop. Railway CPU/RAM metrics cannot establish that the product is delivering the right exercise at the right time.

Expected behavior: founder-facing monitoring covers at minimum:

- process restarts and deployed release identity;
- polling and scheduler heartbeats;
- pending/overdue delivery count and oldest age;
- delivery success, retry, terminal failure, and duplicate-attempt signals;
- PostgreSQL, Redis, Telegram, and OpenAI availability;
- Coach rate-limit and global cost-circuit-breaker activation;
- latest successful backup age and restore-drill status.

Minimal fix: emit structured operational metrics and alerts with documented owners and response actions. Do not include conversation bodies, feedback text, Telegram IDs, tokens, or other sensitive payloads in alerts; privacy-safe logging remains owned by [PRIV-02](#) and [SEC-09](#).

OPS-10 — Release smoke tests and failure drills are not one fixed gate

Severity: P1 before beta Status: confirmed absent

Current behavior: there is no recorded Go/No-Go artifact proving that the built production image, migrations, dependencies, and user-facing lifecycle work together. Individual tests do not cover Railway process behavior or the failure boundaries between PostgreSQL, Redis, Telegram, OpenAI, and the scheduler.

Expected behavior: every production release runs one fixed smoke path on the isolated FD-18 testnet using the exact release artifact and records the result; a production-safe configuration check follows without writing behavioral test data. Before beta, the team also rehearses the highest-cost failures: SIGTERM during delivery, Telegram timeout, OpenAI outage, Redis outage, database unavailability, duplicate Telegram update, and database restore with scheduler reconstruction.

Minimal fix: create a concise release checklist and executable smoke script. Verify at least: fresh start, existing active user, timezone resolution, scheduled delivery, complete/skip/expiry, Coach reply and deterministic outage fallback, runtime tool result, pause/resume/cancel, cycle completion, summary, automatic continuation, health endpoints, graceful shutdown, and restart. The company-specific invitation and attribution path remains additionally owned by [COMP-10](#).

OPS-11 — Delivery capacity has not been validated against a company-shaped burst

Severity: P1 before the first company rollout Status: unmeasured

Current behavior: APScheduler uses its default thread pool while each delivery can block waiting up to 30 seconds for an asyncio result. Telegram also applies provider-side throughput limits. No test establishes what happens when a real deployment schedules many employees for the same minute.

Expected behavior: the release candidate delivers a burst at least twice the largest expected launch cohort without duplicate messages, starvation, unbounded memory growth, lost terminal state, or misleading failure telemetry. Provider throttling is handled by bounded retry/backoff rather than by spawning more schedulers.

Minimal fix: run the capacity test before the first company rollout and record latency distribution, success/failure counts, retries, and backlog recovery. Do not add jitter for the 10–15-person beta. If the company-shaped test fails, consider a rate-aware bounded delivery dispatcher first; a founder-approved delivery window/jitter remains a documented fallback lever rather than a silent change to the selected-time promise.

OPS-12 — Scheduled exercise delivery is correctly independent of OpenAI

Severity: OK — preserve as an architectural invariant Status: confirmed

The scheduled exercise path sends a deterministic text snapshot and does not call the model at delivery time. An OpenAI outage can degrade Coach and new plan generation without stopping already-scheduled exercises. This reduces cost, latency, and provider blast radius at the product's primary touchpoint.

Preserve this boundary during the renderer and content overhaul. Canonical structured presentation and versioned media may be rendered by application code, but scheduled delivery must not become dependent on live LLM output.

Release Go/No-Go Checklist

Run after all accepted audit implementation work and before every production release. Keep the completed artifact with the release record.

1. Release commit, image ID, schema version, content version, and configuration manifest recorded; production worktree/build context is clean.
2. The same release artifact is selected for production and testnet; environment startup checks prove disjoint databases, Redis namespaces, Telegram bots, OpenAI projects, token keys, URLs, backups, alerts, and aggregate sinks.
3. Canonical test command collects and passes the complete suite in the same dependency environment used by CI.
4. Migration rehearsed against a restored production-like backup; startup performs schema verification only.
5. FD-12 backup is current, manual pre-change backup exists, and the restore procedure has a measured RPO/RT0.
6. Railway is configured for one replica and controlled non-overlapping rollout; `/live`, `/ready`, `PORT`, restart policy, and drain window verified.
7. SIGTERM test confirms scheduler threads drain while the event loop remains alive and all dependencies close within the termination window.
8. End-to-end smoke path passes on a real Telegram testnet account, including delivery, response, Coach degradation, lifecycle controls, summary, and continuation.
9. Restore drill confirms `apscheduler_jobs` reconstruction and no duplicate, stale, expired, canceled, or already-delivered exercise.
10. Monitoring and alerts identify the release, polling/scheduler health, delivery backlog/failure, dependency outage, backup age, and cost controls without logging sensitive payloads.
11. Global and deployment-scoped kill switches, key-revocation path, and named incident owner are reachable and tested under SEC-09 / COMP-09.
12. Capacity test passes at twice the expected peak simultaneous delivery cohort before the first company rollout.
13. Rollback choice is explicit: compatible code rollback, forward fix, or destructive restore with accepted RPO. Database restore is never treated as an ordinary code rollback.

Blanket references — do not duplicate

- Backup policy and purchase/configuration remain owned by FD-12; OPS-07 turns that decision into a verified restore procedure.
- One replica / one scheduler writer remains owned by DB-20; OPS-01 applies

it to Railway handover behavior.

- Retry-safe external side effects remain owned by DB-08, DEL-05, and UX-07; OPS-07 and OPS-10 add restore and release verification only.
 - Container/dependency hardening remains owned by SEC-08; incident response, kill switches, and sensitive logging by SEC-09 and PRIV-02.
 - Coach admission, queueing, and cost controls remain owned by FD-17 / SEC-06; Operations verifies rather than redesigns them.
 - Company-specific launch verification remains owned by COMP-10; the release gate proves the shared runtime before that per-deployment checklist begins.
 - Production/testnet enrollment and identity isolation remain owned by FD-18, COMP-08, and SEC-10; Operations verifies the release artifact and runtime configuration rather than redesigning that boundary.
 - Content snapshot/version semantics remain owned by FD-10 and the Content Library round; OPS-08 records release provenance and rollback behavior.
-

Exercise on Demand Findings

Exercise on Demand Area — Audit Round 2026-08-19

Status: product/audience/system audit complete; target accepted under FD-19; implementation not started in this round

Scope and evidence

The round inspected the end-user interview corpus and R&D synthesis separately, then traced the current application paths required to add the feature without creating another plan lifecycle:

- primary end-user interviews under

`/Users/Baracuda/Desktop/transcripts/love yourself/Інтерв'ю з end-користувачами;`

- R&D discovery and evidence-review documents under

`/Users/Baracuda/Desktop/Love Yourself/R&D;`

- `docs/audit/product_contract.md`, the Founder Decisions above, Product Maps,

and the accepted Content Library and Delivery UX rounds;

- `app/telegram.py`, `app/orchestrator.py`, `app/workers/coach_agent.py`, and

`app/plan_runtime/tools.py`;

- `app/content_library.py`, the content JSON, `app/ux/task_notification.py`, and

`app/scheduler.py`;

- `app/db.py`, `app/telemetry.py`, plan completion/metrics code, privacy,

lifecycle/FSM, PostgreSQL/Redis, Security, and Operations findings.

The interviews confirm a recurring workday state-switch problem: respondents walk, look outside, seek quiet, use coffee/media, or continue working while tired when stopping is difficult. They support a short, accessible action at a self-recognized moment. They do not establish demand for a Telegram command, random selection, the exact catalogue size, a 30-minute window, or long-term retention. Those are explicit beta hypotheses, not evidence claims.

The feature tests an action-retrieval mechanism: the user supplies the moment and the product removes recall and choice friction. It does not diagnose the user, infer why help is wanted, or promise stress reduction, restored focus, burnout prevention, or productivity.

Accepted end-to-end contract

```
Telegram command menu "Врпава запас"
or explicit Coach request -> request_on_demand_exercise tool
-> one shared authorized application service
-> create/reuse one open occurrence
-> release-gated uniform selection, excluding only immediate repeat
-> persist exercise/content version/presentation snapshot
-> deterministic Telegram delivery with text fallback
-> delivered + 30-minute database deadline
-> completed | skipped | expired
-> canonical source-stamped telemetry
-> conditional, separately labelled cycle-summary count
```

Module-by-module implementation contract

Module / boundary	Required interaction
Telegram command menu	Register internal <code>/exercise</code> with the visible description <code>Вправа запис</code> . Route it before generic <code>F.text</code> ; do not add a persistent reply keyboard.
Command handler	Resolve the existing authorized user, enforce onboarding/entitlement, derive an idempotent Telegram update key, and call the shared on-demand service. It must not write the command into <code>ChatHistory</code> or invoke the Coach.
Coach prompt and tool schema	Add <code>request_on_demand_exercise</code> in every post-onboarding authorized product mode. Use it only for an explicit exercise/state-switch request, with no redundant confirmation and no mood inference.
Coach tool executor / orchestrator	Re-authorize the user and call the same service as the command handler. Return a strict allowlisted result (<code>delivered</code> , <code>existing_open</code> , <code>temporarily_unavailable</code> , or <code>failed</code>). The deterministic exercise message is the product result; the model must not invent or restate instructions.
On-demand application service	Own request idempotency, one-open enforcement, selection, occurrence state, immutable presentation snapshot, delivery dispatch, and structured result. No Telegram adapter or Coach branch may reimplement selection.
Content Library	Read the six FD-10 <code>switch</code> records only. Enforce <code>is_active</code> , the review gate, and release asset readiness. Do not fall back to <code>unload</code> , a legacy parent/variation, or gated content.
Selector	Read on-demand shown history only; exclude the last successfully delivered on-demand exercise when it would repeat, then use equal probability across all remaining eligible records. Do not inspect completion, skip, expiry, feedback, plan state, inferred context, scheduled history, or <code>cooldown_days</code> .
Exercise presentation	Build one canonical <code>ExercisePresentation</code> containing occurrence ID, exercise ID, content version, title, duration, steps, media metadata, deadline, source, and available actions. Store an immutable snapshot before the external send.
Telegram delivery adapter	Render the same presentation contract used by scheduled delivery. Record confirmed message ID and delivery variant. A transient media failure retries/falls back to complete text; it never selects another exercise.
PostgreSQL	Add an independent on-demand occurrence table, foreign keys/indexes, source-operation uniqueness, and a partial unique constraint for one open occurrence per user. PostgreSQL is lifecycle truth.
Redis	No on-demand business state is stored in Redis. It may support a non-authoritative short operational debounce only if needed; correctness must survive Redis loss and process restart.
Callback router	Use an on-demand-specific callback namespace carrying the occurrence identity, not <code>AIPlanStep.id</code> . Verify Telegram ownership, entitlement boundary, current row state, and deadline, then perform one atomic terminal transition.
Expiry/reconciliation worker	Periodically claim due delivered occurrences, set <code>expired</code> once, emit the linked event, remove actions, and show <code>Час виконання минув</code> . Retry failed Telegram edits without reopening the row. Callback-time expiry is the race-safe fallback.
Scheduled plan lifecycle	Share the Content Library and renderer only. Never read on-demand history to reorder a prepared sequence, and never create or mutate <code>AIPlan</code> , <code>AIPlanDay</code> , <code>AIPlanStep</code> , plan jobs, current day, plan completion, automatic continuation, or scheduled metrics.
FSM / derived mode	Add no on-demand state. Availability is entitlement + onboarding, independent of active/paused/no-active-plan mode.
Telemetry	Emit validated on-demand events through the canonical event ingestion path with explicit source and occurrence linkage. Do not auto-create a fake plan instance/window for on-demand activity.
FD-07 feedback	Reuse the completed-only feedback interaction with <code>source=on_demand</code> and the occurrence ID. Feedback never changes selection.
Cycle summary	Query on-demand completions independently for the cycle observation window. Render <code>За власним запитом: N виконано</code> only when <code>N > 0</code> ; preserve the scheduled denominator and use a count-only combined total.
Privacy / company aggregate	Apply personal-data retention/deletion and independent aggregate contribution. No company surface gains channel or exercise detail. Only a completed/skipped response can satisfy the accepted aggregate <code>active</code> definition.
Operations	Monitor delivery/reconciliation failure, overdue open rows, duplicate attempts, callback conflicts, pool readiness, and media fallback without logging sensitive content or user text.

Target persistence contract

The minimal dedicated table is `on_demand_exercise_requests`:

Column	Contract
<code>id</code>	UUID primary key; the occurrence identity used by callbacks and telemetry.
<code>user_id</code>	Required personal-layer foreign key with retention/deletion behavior matching user telemetry.
<code>exercise_id</code>	Required foreign key to the stable Content Library record after selection.
<code>content_version</code>	Exact delivered instruction version; never derived later from the current library row.
<code>presentation_snapshot</code>	Controlled structured JSON needed to reproduce/edit the delivered occurrence; no Coach text or inferred state.
<code>status</code>	<code>pending_delivery</code> , <code>delivered</code> , <code>completed</code> , <code>skipped</code> , <code>expired</code> , <code>delivery_failed</code> , or <code>access/account canceled</code> .
<code>entry_surface</code>	<code>command_menu</code> or <code>coach</code> ; future values require an event-contract change.

Column	Contract
source_operation_id	Unique idempotency key derived from the Telegram update/tool execution, not random per retry.
requested_at	Server timestamp for accepted request creation.
delivered_at	Confirmed Telegram delivery timestamp; null on terminal delivery failure.
expires_at	Exactly 30 minutes after confirmed delivery.
responded_at	Accepted completed/skipped callback timestamp; null for expiry/failure.
telegram_chat_id/telegram_message_id	Restricted delivery references required to edit actions and reconcile external state.
delivery_variant	Exact media/text presentation actually delivered.
bounded operational fields	Retry count and allowlisted error class only; no raw provider payload or user content.

Required database rules:

- partial unique index on `user_id` for `pending_delivery` and `delivered`;
- unique `source_operation_id`;
- index on due `expires_at` for `delivered` rows;
- conditional/locked `delivered` -> `terminal` transition;
- no `plan_id`, `plan_day_id`, `plan_step_id`, or `plan_execution_id` on the

occurrence;

- a migration/backfill is additive; no legacy plan row is converted into an

on-demand occurrence.

Canonical on-demand event catalogue

The canonical envelope work already required by TEL-03/04/11 must support a nullable `on_demand_request_id` and explicit `exercise_source`. Required event facts include `event_id`, event/schema version, unique source-operation ID, user/deployment/environment references, occurrence ID, exercise ID, content version, entry surface or delivery variant where applicable, `occurred_at`, and `recorded_at`.

Event	Required meaning
on_demand_requested	One authorized occurrence was first accepted. Idempotent recovery does not emit another event.
on_demand_delivered	Telegram confirmed delivery and the 30-minute response window began.
on_demand_delivery_failed	No confirmed user opportunity existed; exclude from behavior denominators.
on_demand_completed	One valid callback registered completion before deadline. It does not prove execution or benefit.
on_demand_skipped	One valid callback registered explicit skip before deadline. No reason is inferred.
on_demand_expired	The authoritative occurrence closed without an accepted callback.
feedback_submitted	Optional completed-only FD-07 answer linked to the on-demand occurrence and source.

Duplicate requests and callbacks may produce privacy-safe operational deduplication/conflict signals, but they must not create duplicate behavioral events or independent aggregate contributions. Response latency is derived from `delivered_at` to the accepted terminal callback and may be reported in bounded buckets; it is never named reaction time, execution time, or effect.

Findings

EOD-01 — Neither accepted entry surface exists

Severity: P1 before feature beta Status: confirmed absent

Current behavior: `app/telegram.py` registers `/start`, admin `/spawn`, legacy callbacks, and a catch-all text handler. There is no `/exercise` menu command, command registration, or deterministic self-start handler. Generic text is stored in `ChatHistory`, logged as `user_message`, and sent to the Coach.

Expected behavior: the command-menu path bypasses the model and calls the shared on-demand service. An explicit natural-language request may reach the same service through the Coach tool, but ordinary text remains Coach text and is not counted as exercise use.

Minimal fix: add command-menu registration/handler before `F.text`, add the scoped Coach tool, and share one service. Do not add a reply keyboard or treat `user_message` as on-demand telemetry.

EOD-02 — Coach policy and runtime tools currently forbid the accepted action

Severity: P1 before Coach entry is enabled Status: confirmed

Current behavior: the Coach prompt says not to suggest exercises outside the current sequence and exposes only plan-management tools by current legacy state. There is no runtime executor for an independent exercise request. The existing bounded tool-result architecture is itself still implementation work under COACH-09/RT-09.

Expected behavior: `request_on_demand_exercise` is allowed in every authorized post-onboarding mode, requires explicit user intent but no redundant confirmation, and returns a strict runtime result. Runtime rechecks access and one-open invariants; model tool selection is never authorization.

Minimal fix: update the prompt boundary, tool schema, state/mode availability, executor dispatch, result schema, and Coach tests together. On successful deterministic delivery, do not add a model-authored second version of the exercise.

EOD-03 — The target pool is accepted, but runtime eligibility is not ready

Severity: P1 content/delivery dependency Status: confirmed; blocked on FD-10/FD-16 implementation prerequisites

Current behavior: runtime still loads the legacy eight-parent JSON/schema. The six target switch records, review fields, structured requirements, and media references are not one implemented source of truth. The cool-water medical gate and required GIF delivery are not runtime eligibility checks.

Expected behavior: on-demand reads only the migrated FD-10 records. The beta launch pool contains the five non-cool-water switch exercises after required GIF assets are configured; approved cool water becomes the sixth. Selection is equal-probability random after excluding only an immediate repeat.

Minimal fix: complete the existing Content Library migration and media work before enabling on-demand. Add selector tests proving distribution inputs, last on-demand-delivered exclusion, allowed `A -> B -> A -> B`, gate enforcement, and fail-closed behavior for fewer than two eligible records.

EOD-04 — No independent occurrence aggregate exists

Severity: data-integrity blocker before feature beta Status: confirmed absent

Current behavior: executable exercise state exists only on `AIPlanStep`, whose foreign keys, day relation, schedule, status, and callbacks all assume a plan. Reusing it would create fake days/steps and couple voluntary requests to plan completion. No separate request ID or one-open database constraint exists.

Expected behavior: the dedicated PostgreSQL table and lifecycle above own all on-demand state. The row stores the selected version/snapshot before send and survives restart. It is independently queryable without plan joins.

Minimal fix: add one additive migration, ORM model, repository/service access, constraints, indexes, and deletion/retention behavior. Do not generalize this into a universal workflow engine.

EOD-05 — Current renderer and delivery path are plan-step-specific

Severity: P1 before feature beta Status: confirmed; shared prerequisite with UX-03/15/16

Current behavior: `format_task_notification()` requires plan day, task index, task total, and rationale/decorative fields. `send_scheduled_message()` builds callbacks from `AIPlanStep.id` and records delivery back to that step. Media delivery and canonical `ExercisePresentation` are absent.

Expected behavior: the shared renderer consumes structured presentation facts without assuming a plan. The on-demand adapter displays title, duration, steps, media when required, and only its two actions. Delivery is deterministic and independent of OpenAI.

Minimal fix: implement UX-16's canonical presentation and a small on-demand delivery adapter over it. Preserve scheduled delivery behavior until its own accepted refactor is implemented; do not route on-demand through a fabricated scheduled job.

EOD-06 — Callback, expiry, and visible terminal-state behavior are absent

Severity: P1 lifecycle/concurrency before feature beta Status: confirmed absent

Current behavior: scheduled callbacks parse `AIPlanStep.id`, perform ownership checks, and then write terminal state without an atomic conditional transition. Expired/canceled scheduled taps may fail silently. Scheduled expiry and the legacy `task_ignored` event are already split incorrectly under TEL-05.

Expected behavior: on-demand callbacks identify the occurrence, enforce ownership and `delivered` state, compare authoritative time to `expires_at`, and atomically accept exactly one terminal action. A periodic reconciler and callback-time check close overdue rows, remove actions, and show `Час виконання минув`. Late/duplicate callbacks return a factual state and never reopen or duplicate the occurrence.

Minimal fix: add the on-demand callback namespace, conditional update/row locking, expiry reconciler, idempotent message edit, and race/restart tests. Reuse low-level Telegram edit helpers where useful, not scheduled plan state.

EOD-07 — Plan, FSM, scheduler, and Redis isolation must be enforced by tests

Severity: P1 system-integrity invariant Status: accepted target; no feature code exists yet

Current behavior: several current gates derive exercise availability from `User.current_state == ACTIVE`, an active plan, an active working day, and a scheduled timestamp. Plan/FSM truth is already duplicated, and Redis stores legacy session tunnels.

Expected behavior: on-demand authorization depends on completed onboarding and valid entitlement, not active plan or workday. Requests work during active, paused, and no-plan modes and outside schedule. No on-demand write touches plan tables, plan jobs, `current_state`, `current_day`, pause flags, or Redis business state.

Minimal fix: place availability in the on-demand service/access guard and add negative side-effect assertions across every plan/FSM field and job store. Keep PostgreSQL authoritative after process or Redis restart.

EOD-08 — Current telemetry cannot represent a truthful on-demand source

Severity: experiment-validity blocker before feature beta Status: confirmed; depends on TEL-03/04/11 and CONTENT-07

Current behavior: `log_user_event()` requires/reuses a legacy `PlanExecutionWindow`, event type and JSON properties are not one enforced catalogue, source-operation uniqueness is absent, and task statistics assume scheduled task semantics. `user_message` cannot identify reactive exercise use.

Expected behavior: on-demand events use the canonical envelope and occurrence foreign key without fake plan linkage. Source, content version, delivery variant, timestamps, and terminal outcome are explicit. Delivery failure is operational; completion/skip/expiry remain separate behavioral facts.

Minimal fix: extend the accepted canonical event migration, validation, idempotent aggregate write, and founder-only query set. Do not copy on-demand facts into legacy `TaskStats`, `FailureSignal`, compensation, streak, or engagement-inference tables.

EOD-09 — The accepted cycle summary has no conditional on-demand axis

Severity: P1 before the first cycle containing on-demand completion closes Status: confirmed absent; current report is separately obsolete under UX-14

Current behavior: completion metrics and report code query only plan steps and use plan/streak/persona language that FD-15 already replaces. There is no on-demand observation-window query or separate count.

Expected behavior: the deterministic FD-15 summary retains the canonical scheduled numerator/denominator under the neutral label `За розкладом`. On-demand contributes `За власним запитом: N виконано` only when $N > 0$ for the cycle window. `Усього виконано` may sum registered completions as a raw count; no shared percentage or adherence denominator exists. If the user has no active/completed scheduled cycle, on-demand telemetry does not create a standalone weekly report in MVP.

Minimal fix: add one independent completion-count query and structured summary field while implementing FD-15. Test zero omission, one/many completions, boundary timestamps, paused cycles, and denominator isolation.

EOD-10 — Privacy and company reporting must not expose channel behavior

Severity: privacy blocker before feature beta Status: accepted boundary under FD-09; implementation follows PRIV-06/07

Current behavior: independent privacy-preserving aggregate contributions are not implemented. User-linked telemetry and conversation records remain the only practical analytics layer.

Expected behavior: entry surface, exercise, response, latency, feedback, and channel mix remain personal/founder-restricted data. No company report gains on-demand counts or comparisons. A completed/skipped on-demand response may contribute idempotently to the sealed distinct-active aggregate, without a source dimension or user join; request/delivery/expiry does not.

Minimal fix: include on-demand events in personal retention/deletion and the already-accepted independent aggregate transaction. Add privacy tests proving that company queries cannot select the new occurrence/event fields.

EOD-11 — Failure and restart semantics need one reconcilable truth

Severity: P1 operational reliability before feature beta Status: target accepted; implementation absent

Current behavior: Telegram send and database telemetry can diverge in the scheduled path, and current scheduler jobs are not a durable on-demand request queue. Process memory cannot safely enforce double-tap or one-open behavior.

Expected behavior:

- a duplicate Telegram update or Coach tool execution resolves to one source

operation and one occurrence;

- selection and presentation snapshot are stable across retries;
- confirmed delivery begins the deadline; unconfirmed delivery is never a user

skip/expiry opportunity;

- transient GIF failure falls back to text; total send failure becomes

`delivery_failed` and permits a later new request;

- restart recovery scans durable pending/delivered rows and never reselects or

double-sends blindly;

- entitlement/account closure cancels open rows without classifying the user as

inactive;

- simultaneous scheduled/on-demand delivery may coexist without lifecycle

corruption.

Minimal fix: implement stable source-operation identity, transaction/outbox or equivalent narrow reconciliation, bounded delivery retry, overdue-row monitoring, and restart/failure tests. Do not solve this with process-local sets or Redis-only locks.

EOD-12 — Beta reporting must preserve the open channel hypothesis

Severity: P1 measurement contract before interpreting use Status: accepted under FD-19

Current behavior: FD-11's metric hierarchy is centered on scheduled rhythm and explicitly deferred reactive metrics until a distinct action existed. That action is now accepted, but no canonical founder view separates it.

Expected behavior: founder-only beta analytics shows raw cohort/window values for eligible users, distinct on-demand requesters, requests, confirmed deliveries, completed/skipped/expired outcomes, repeat requesters, response- latency buckets, content/version exposure, and optional completed-only effect feedback. It also permits descriptive scheduled-only, on-demand-only, both, and neither groupings without treating any group as success by definition.

This observational split reveals channel preference but does not by itself prove that one channel caused a better outcome. Pilot cohorts/feature flags are a possible later experiment mechanism, not implementation scope for this feature round.

Minimal fix: extend the restricted canonical founder query/report after event integrity is verified. Show numerator, denominator, raw `n`, observation window, and delivery failures; do not encode automatic keep/kill thresholds or claim causal superiority from self-selection.

Required implementation order

1. complete FD-10 target records, review eligibility, and FD-16 media

prerequisites for the five-record launch pool;

2. finish the canonical `ExercisePresentation` and deterministic media/text

delivery boundary required by UX-15/16;

3. add the on-demand table, constraints, migration, repository, and shared

application service;

4. add the command-menu route and exact authorization/idempotency path;

5. add the Coach tool, runtime re-authorization, strict result handling, and

prompt tests over the same service;

6. add the uniform immediate-no-repeat selector and pool-readiness checks;
7. implement delivery, callback, 30-minute expiry, visible terminal state, reconciliation, and restart/failure handling;
8. extend the canonical telemetry envelope/catalogue and independent aggregate contribution without legacy plan-window or inference writes;
9. extend the FD-15 deterministic summary with the conditional on-demand count;
10. reconcile seeded scheduled and on-demand trajectories against database state/events, then run real Telegram device QA before enabling the feature.

Required test matrix

- command and Coach entries call one service and differ only by `entry_surface/source` operation;
- explicit Coach request calls the tool; stress/fatigue mention alone does not;
- active, paused, no-active-plan, weekend, and off-hours access succeeds; onboarding-incomplete/revoked access fails;
- equal eligible pool, immediate duplicate exclusion, allowed `A -> B -> A -> B`, and no completion/skip/feedback weighting;
- cool-water/media/review gates and fewer-than-two pool fail closed;
- command/tool retries and double taps create one occurrence/delivery;
- success, existing-open, media fallback, delivery failure, and retry results;
- completed/skip race, expiry race, stale callback, duplicate callback, and exact `Час виконання минає`. presentation;
- restart between request/selection/send, between delivery/event write, and before expiry reconciliation;
- no mutation of plan, plan step/day, scheduler job, continuation, FSM, or Redis business state;
- event IDs/linkage/content version/source/variant and independent aggregate idempotency;
- summary omission at zero, conditional count above zero, count-only total, and unchanged scheduled denominator;
- retention/deletion, company-query exclusion, privacy-safe logs, and alert payloads.

Explicitly deferred

- additional exercises or user-browsable exercise catalogue;
- full non-repeating shuffle, anti-ping-pong rules, weights, recommendations, or learned preferences;
- user-facing daily limits and therapeutic cooldown claims;
- persistent keyboard, Web App, Mini App, or other channel adapters;
- separate weekly summary/pulse outside the accepted cycle artifact;

- causal scheduled-vs-reactive experiment assignment until a real pilot design and sample justify it.

Blanket references — do not duplicate

- FD-10 owns target exercise records and review eligibility; FD-16 owns the required GIF boundary.
- UX-15/16 own canonical presentation and media fallback; this round specifies how the new channel consumes them.
- COACH-09/RT-09 own the bounded tool-result architecture; EOD-02 adds one scoped tool and its authorization/result contract.
- FD-08 and the Lifecycle round own plan-derived modes; EOD-07 forbids a second lifecycle authority.
- CONTENT-07 and TEL-03/04/11 own canonical identity, linkage, event schema, and idempotency; EOD-08 adds the on-demand source/occurrence.
- FD-07 owns completed-only effect feedback; this round only adds source and occurrence linkage.
- FD-09/PRIV-06/07 own personal retention and independent aggregation; EOD-10 prevents channel leakage.
- FD-15/UX-14 own the deterministic cycle summary; EOD-09 adds a conditional count without changing scheduled adherence.
- SEC-07, DB-04/08, and OPS-09/10 own atomic callbacks, retry/reconciliation, monitoring, and release verification; this round applies them to the new occurrence.

Research-to-Product Traceability Findings

Research-to-Product Alignment — Audit Round 2026-08-20

Status: completed; evidence classification only, no new product decision

Purpose and authority boundary

This round tests whether the product model assembled by the audit is traceable to the available end-user interviews, buyer research, and intervention evidence. It is not a new discovery exercise and does not ask an AI system to invent a product from market material.

The traceability chain is:

```
primary participant signal
-> secondary research interpretation
-> accepted product mechanism
-> evidence class
-> beta question or system obligation
```

This section creates no Founder Decision and does not override one. A direct match means the underlying problem or behavior appeared in primary evidence; it does **not** prove that the exact feature, copy, cadence, interface, or business model will work. Conversely, no direct evidence does not automatically reject a choice. Security, privacy, consistency, and recoverability controls can be required by the selected architecture even when no interview participant asked for them.

Evidence classification

Class	Meaning in this audit
DIRECT	A participant described the relevant past behavior, constraint, or problem in a primary interview. This validates the signal, not the proposed feature.
SECONDARY	An R&D synthesis, buyer report, evidence review, or framework supports the mechanism or boundary. It remains interpretation unless linked back to a primary source.
HYPOTHESIS	A deliberate product bet that the reviewed evidence does not directly validate. It must be tested rather than presented as research truth.
NO EVIDENCE	The reviewed source set neither supports nor rejects the exact choice. Absence is recorded so later storytelling does not manufacture validation.
TENSION	Evidence points in more than one direction, or a source explicitly warns against an overly broad version of the mechanism.
SYSTEM	A technical, privacy, security, operational, or experiment-integrity requirement derived from implementing the accepted model. It is not a user-demand claim.

A row may carry more than one class. For example, the workday-fatigue problem can be `DIRECT`, while a Telegram command chosen to address it remains a `HYPOTHESIS`.

Blanket source register

Primary end-user interviews are stored under: `/Users/Baracuda/Desktop/transcripts/love yourself/Інтерв'ю з end-користувачами/`. The short references used below map to these authentic transcript documents:

- EU-01 — 01 — Інтерв'ю з Володимиром Красулею.docx;
- EU-02 — 02 — Інтерв'ю з Богданом Нескороженим.docx;
- EU-03 — 03 — Інтерв'ю з Яриком Єрміловим.docx;
- EU-04 — 04 — Інтерв'ю з Дмитром Ковальчуком.docx;
- EU-05 — 05 — Інтерв'ю з Іриною Падалкою.docx;
- EU-06 — 06 — Інтерв'ю із Сергієм Донецьким.docx;
- EU-07 — 07 — Інтерв'ю з Євгенієм Логвиненком.docx;
- EU-08 — 08 — Інтерв'ю з Олександром Вапняруком.docx;
- EU-09 — 09 — Інтерв'ю з Олегом Ліпняцьким.docx;
- EU-10 — 10 — Інтерв'ю з Михайлом Сахариленком.docx;

- EU-11 — 11 — Інтерв’ю з Олександром Очеретним.docx;
- EU-12 — 12 — Інтерв’ю з Василем Франківим.docx;
- EU-13 — 13 — Інтерв’ю з Артемом Колодієм.docx;
- EU-14 — 14 — Інтерв’ю з Іриною Лисач.docx;
- EU-15 — 15 — Інтерв’ю з Дмитром Селютіним.docx;
- EU-17 — 17 — Інтерв’ю з Дмитром Пархоменком.docx;
- EU-18 — 18 — Інтерв’ю з Максимом Будником.docx;
- EU-19 — 19 — Інтерв’ю з Романом Овчаренком.docx;
- EU-20 — 20 — Інтерв’ю з Олександром Потаповим.docx.

EU-ALL means the complete 19-document primary set above. There is no EU-16 file in the supplied corpus. Participant statements are direct evidence; interviewer prompts, embedded AI suggestions, editorial notes, and future willingness such as “show me the beta” are not counted as behavior.

Secondary and synthesis references are stored under /Users/Baracuda/Desktop/Love Yourself/R&D/:

- DISC — Дослідження ринку Love Yourself/discovery_working.docx;
- HR — Дослідження ринку Love Yourself/hr_discovery_final-2.pdf;
- POS — Дослідження ринку Love Yourself/love_yourself_market_positioning.docx;
- JTBD — Дослідження ринку Love Yourself/jtbd_love_yourself.docx;
- EVID-LY — Підготовлені матеріали/Love Yourself Evidence Review for Wartime Ukrainian IT Workplaces.docx;
- EVID-MICRO — Підготовлені матеріали/Evidence Review on Short Workplace Micro-Interventions for Cognitive Workers in Wartime Ukraine.docx;
- FOGG — Підготовлені матеріали/fogg_framework_loveyourself(1).md;
- ASSUMPTIONS — Дослідження ринку Love Yourself/t0_3_riskiest_assumptions.html.

DISC, POS, JTBD, EVID-*, FOGG, and ASSUMPTIONS are secondary interpretations or frameworks even where they summarize interviews. HR is a buyer-side synthesis based on nine full interviews and four LinkedIn exchanges; it does not substitute for employee evidence.

Accepted-decision coverage index

This compact index proves that every accepted Founder Decision is represented in the constructor below. It is a cross-reference only: it creates no new FD, does not re-open an accepted decision, and does not upgrade a hypothesis into research evidence.

Existing decision	Constructor coverage	Evidence classification and blanket source boundary
FD-01 automatic same-format continuation	Constructor C, cycle lifecycle	HYPOTHESIS + TENSION — friction reasoning and autonomy evidence, but no direct continuation test. (FOGG; EVID-LY; EVID-MICRO; EU-ALL; no exact flow)
FD-02 remove the two-hour completion trigger	Constructors C and H	SYSTEM — deadline and lifecycle correctness; no user-demand claim.
FD-03 MORNING/DAY/EVENING are internal tags	Constructors C and H	SYSTEM + HYPOTHESIS schedule model — internal normalization supports the selected schedule; exact slots are not research-validated. (EU-ALL; EVID-LY timing discussion)
FD-04 deterministic mechanism-sale onboarding	Constructor B	SECONDARY + HYPOTHESIS — supported by DISC/JTBD interpretation, exact funnel untested. (DISC §4; JTBD; EU-ALL)
FD-05 no behavioral contingency; randomness for variety; explicit effect feedback	Constructors B, D, and F	SECONDARY + SYSTEM for no hidden inference; HYPOTHESIS for random variety and exact feedback UI. (EVID-LY; EVID-MICRO; EU-ALL)
FD-06 canonical exercise delivery contract	Constructors C and F	SECONDARY + SYSTEM + HYPOTHESIS UI — concrete/shame-free interaction is supported; exact message composition is not. (DISC; EVID-LY; EVID-MICRO)
FD-07 three explicit feedback sources in one table	Constructors F and G	SECONDARY + SYSTEM for explicit proximal measurement and source separation; exact table/UI is not user demand. (EVID-LY Recommended MVP metrics; EVID-MICRO MVP metrics)
FD-08 plan-centric lifecycle	Constructor H	SYSTEM — consistency, concurrency, and migration authority.

Existing decision	Constructor coverage	Evidence classification and blanket source boundary
FD-09 privacy-gated company analytics and independent aggregation	Constructors B and G	DIRECT buyer-side + SECONDARY + SYSTEM for privacy; HYPOTHESIS policy for the exact 100/50 gate and three-line report. (HR; POS; EVID-LY; EVID-MICRO)
FD-10 versioned nine-record Content Library	Constructor E	SECONDARY for intervention families; HYPOTHESIS / NO DIRECT EVIDENCE for exact records, count, instructions, and effect. (EVID-LY; EVID-MICRO; EU-ALL)
FD-11 interaction-only telemetry	Constructor G	SECONDARY + SYSTEM — proximal metrics and claims ceiling are evidence-aligned. (EVID-LY; EVID-MICRO; ASSUMPTIONS)
FD-12 Railway backup and restore requirement	Constructor H	SYSTEM — durability and recoverability, not market evidence.
FD-13 Love Yourself / LY Workday naming architecture	Constructor A	HYPOTHESIS / NO EVIDENCE — brand decision not tested in the reviewed corpus. (DISC; HR; POS: no naming test)
FD-14 neutral preview, exercise in chat	Constructor F	SECONDARY + HYPOTHESIS — privacy/stigma support discretion; exact preview and reveal behavior require device/user testing. (HR; EVID-LY; EVID-MICRO)
FD-15 deterministic Telegram cycle image	Constructors C and F	HYPOTHESIS + SYSTEM — factual deterministic output avoids inference; the artifact itself is unvalidated. (EU-17 adjacent weekly-review behavior; EU-ALL: no image test)
FD-16 required GIFs for three exercises	Constructor E	HYPOTHESIS execution support + SYSTEM — no media-format comparison in the reviewed evidence. (EU-ALL; EVID-LY; EVID-MICRO)
FD-17 no visible Coach quota, invisible operational controls	Constructors F and H	HYPOTHESIS + SYSTEM — user quota expectations are untested; abuse, ordering, and spend bounds are operational necessities. (EU-ALL: no quota evidence)
FD-18 roster-gated production plus isolated testnet	Constructors B, G, and H	SECONDARY privacy/adoption boundary plus SYSTEM identity and environment design; exact OIDC/token flow untested. (HR; EVID-LY; EVID-MICRO)
FD-19 independent Exercise on demand channel	Constructors D, E, F, and G	DIRECT problem/mechanism match + HYPOTHESIS feature + SYSTEM separation. (EU-01, EU-02, EU-05–EU-10, EU-12, EU-14, EU-15, EU-17–EU-20; DISC; EVID-LY)

Constructor A — Problem, positioning, and market model

Product component or accepted choice	Evidence class	Evidence and mismatch boundary	Consequence for implementation or beta
Workday fatigue accumulates before the day ends, and a person may need a context switch while still working.	DIRECT + SECONDARY	Participants described mid-day fatigue, silence, looking out a window, walking, coffee, media, music, meditation, or continuing despite fatigue. The research synthesis names a concrete 2–5 minute reset job. (EU-01, EU-02, EU-06–EU-10, EU-14, EU-15, EU-17–EU-20; DISC §3.2–3.3)	This is the strongest end-user problem match. Beta should test whether LY Workday is hired at that moment, not merely whether the problem exists.
The product is bounded workday recovery support, not therapy, diagnosis, burnout prevention, or a productivity engine.	SECONDARY + SYSTEM	Evidence is strongest for momentary recovery, fatigue, vigor, and stress; effects on demanding cognitive performance are weak or null, and ultra-brief actions do not establish burnout prevention. (EVID-LY Executive summary, Safe claims, Product implications; EVID-MICRO Executive summary, Product implications)	Product, sales, Coach, summary, telemetry, and company reporting must stay inside the claims fence.
The core value is a trigger plus a small structure, not a browsable wellness catalogue.	DIRECT + SECONDARY + TENSION	People already switch through physical/contextual behaviors, while one participant explicitly rejected wellness apps. DISC says “trigger and structure, not a set of exercises” and separately records that willingness to perform exercises through an app was not confirmed. (EU-04; EU-01, EU-02, EU-05, EU-09, EU-10, EU-12, EU-15, EU-17–EU-20; DISC §3.3–3.5, §4)	Exercise delivery is a controlled mechanism hypothesis, not validated demand for an exercise app. Avoid building a catalogue/browser before use proves the mechanism.
Love Yourself is the company; LY Workday is the employee-facing product.	HYPOTHESIS / NO EVIDENCE	The reviewed research predates or does not test this naming architecture. (DISC, HR, POS: no direct naming test)	Treat as a brand-clarity decision. Validate comprehension in onboarding and buyer conversations; do not describe it as research-backed naming.
B2B2C: a company purchases access and employees use the product voluntarily.	DIRECT buyer-side + HYPOTHESIS employee-side	HR research supports a buyer, budget, privacy concern, leadership/champion dependency, and a detection gap. The same report labels employee use of a corporate tool as critical and beta-only validation. (HR §§2–7; POS §§4–6; ASSUMPTIONS C1/D1)	Preserve two distinct value propositions and two funnels. Buyer interest cannot stand in for employee adoption or continued use.
Initial ICP centers on remote/hybrid Ukrainian cognitive/IT work rather than all workplaces.	DIRECT + SECONDARY, bounded sample	Primary participants are mainly cognitive workers and often remote/hybrid; buyer research identifies remote/hybrid companies and explicitly notes sample limitations. (EU-01, EU-02, EU-07, EU-08, EU-15, EU-17–EU-19; HR §2 and §7)	Do not generalize beta conclusions to workforces without individual email, Telegram access, or cognitive-work context.

Constructor B — Entry, enrollment, onboarding, and user control

Product component or accepted choice	Evidence class	Evidence and mismatch boundary	Consequence for implementation or beta
Telegram is the first delivery and Coach channel.	HYPOTHESIS + SECONDARY	Telegram is argued as low-friction, but end-user interviews did not test channel preference. Buyer research marks whether Telegram is blocked as unverified until beta. (POS §3 and §5; HR C5 and §7; ASSUMPTIONS C5)	Instrument attributed start and delivery failure. A blocked or distrusted channel is a deployment result, not proof that the reset job is absent.
Production enrollment is roster-gated with verified identity, entitlement, and a single-use Telegram handoff; individual beta uses an isolated testnet.	SYSTEM + SECONDARY	Privacy and confidential adoption are direct buyer concerns. No interview specifies roster schemas, OIDC, email challenges, token hashing, or environment topology. (HR §4.2, C2; EVID-LY Privacy and trust; EVID-MICRO Privacy/Data Protection)	Exact access design is a privacy/security consequence of B2B2C, not a market-demand feature. It must be tested for friction and isolation without claiming interview validation.
Onboarding is short, deterministic, and helps the user understand the mechanism before scheduling.	SECONDARY + HYPOTHESIS	DISC interprets low Job Experience and low Solution Experience and recommends a dialogue rather than a form. Primary interviews did not test the accepted onboarding flow or copy. (DISC §4; JTBD switching/onboarding discussion)	Validate completion by stage and qualitative comprehension. Do not infer that research participants approved the funnel.
Onboarding and product copy avoid labeling the user as burned out.	DIRECT buyer-side + SECONDARY	HR participants reported "burnout" as overused or unsafe; evidence reviews support non-clinical framing. End users described fatigue and coping more often than asking for a diagnosis. (HR §4.10, B8, §6; DISC §4; EVID-LY Positioning; EVID-MICRO Positioning)	Keep symptom/state language bounded and verify actual copy in beta interviews.
Pause, resume, cancel, skip, time change, and format switch remain explicit user controls.	SECONDARY + SYSTEM	User control, opt-out, shame-free return, and privacy are consistent evidence-review and buyer requirements. Exact controls were not tested in primary interviews. (EVID-LY Delivery timing, notification frequency, privacy; EVID-MICRO Retention and shame-free return; HR §4.2)	These controls are integrity and autonomy requirements; measure their use without interpreting it as failure or pathology.
No hidden behavior-inference personalization or adaptation in MVP.	SECONDARY + SYSTEM	Evidence favors user control and warns about surveillance and context-sensitive engagement. The 10–15-user beta has no statistical basis for reliable personal inference. Primary interviews did not request learned adaptation. (EVID-LY Privacy, Open questions; EVID-MICRO Privacy and engagement; HR §4.2)	Remove adaptation code and derived user-trait logic completely. Global content learning may use aggregate beta evidence; per-user inference may not.

Constructor C — Scheduled channel and cycle lifecycle

Product component or accepted choice	Evidence class	Evidence and mismatch boundary	Consequence for implementation or beta
A scheduled prompt is one possible product trigger.	DIRECT problem match + SECONDARY + HYPOTHESIS mechanism	Some participants continued working because a task felt too important or defaulted to coffee/media; this supports a prompt problem. Evidence reviews support conservative prompts but do not prove pre-scheduled delivery is superior to JIT or self-start. (EU-01, EU-02, EU-07, EU-08, EU-18, EU-19; DISC §3.2–3.3; EVID-LY JIT delivery and Notification frequency)	Keep scheduled delivery observable as its own hypothesis. Do not treat response to a push as proof that schedule is the only correct model.
MVP default is one scheduled action per selected working day.	SECONDARY + HYPOTHESIS	One short action per day is described as a defensible conservative baseline; evidence does not establish the product's exact optimal dose. (EVID-LY Notification frequency; EVID-MICRO Executive summary and Notification frequency)	Launch conservatively, preserve user control, and test notification fatigue, expiry, and continued response.
User-selected exact delivery time and work days anchor the schedule.	SECONDARY + HYPOTHESIS	Evidence favors user control and rescheduling; no primary interview tested an exact-time picker, allowed company window, or weekday model. (EVID-LY JIT delivery, Notification frequency, Open questions; EVID-MICRO Notification frequency)	Timing policy remains an explicit MVP hypothesis. Record chosen time, successful delivery, and changes; do not claim the system detects an optimal moment.
The first cycle is seven working days; fourteen working days is a user-selected longer format.	HYPOTHESIS / NO DIRECT EVIDENCE	Evidence supports repeated practice over days but not these exact formats, progression rules, or two actions for the longer format. (EVID-LY Recommended MVP metrics and Open questions; EVID-MICRO dosage/open questions)	Treat 7/14 as beta containers for observation, not established effective doses.
A prepared deterministic sequence uses the same accepted library for all eligible users.	HYPOTHESIS + SYSTEM	Research supports low cognitive cost and concrete actions, but does not test one universal recipe. Avoiding hidden inference is evidence-aligned; exact sequence composition is not. (DISC §4; EVID-LY Exercise design and Privacy; EVID-MICRO Exercise design)	Version the recipe and exposure. Learn globally from beta; do not silently mutate delivered instructions or personalize from outcomes.
Completion automatically prepares the same 7- or 14-day format unless the user changes or stops it.	HYPOTHESIS + TENSION	No interview tested automatic continuation. Reduced friction is behavior-design reasoning, while user-control evidence creates a real autonomy tension. (FOGG; EVID-LY adherence/user control; EVID-MICRO shame-free return)	Beta must distinguish operational continuation from actual next-cycle response. Pause/cancel/switch must remain obvious and functional.
Scheduled completion, skip, and expiry close one action window without praise, streak pressure, or adaptation.	DIRECT + SECONDARY + SYSTEM	Motivational content and long to-do framing were described as exhausting; evidence recommends shame-free return and cautions against pressure mechanics. (EU-02; EVID-LY Engagement mechanics; EVID-MICRO Retention and shame-free return)	Use neutral terminal states and measure explicit response separately from silence.

Constructor D — On-demand channel

Product component or accepted choice	Evidence class	Evidence and mismatch boundary	Consequence for implementation or beta
An authorized user may request <i>Вправа зараз</i> when they already notice a need to switch state.	DIRECT problem/mechanism match + HYPOTHESIS feature	Many participants already self-initiated a walk, silence, window gaze, coffee ritual, meditation, music, food, or another context switch. No interview asked whether they wanted a one-button product action. (EU-01, EU-02, EU-05–EU-10, EU-12, EU-14, EU-15, EU-17–EU-20; DISC §3.3)	On-demand is well matched to existing behavior but still requires live usage validation. Requests, repeat requesters, and outcomes must be observed separately from scheduled activity.
Scheduled and on-demand channels coexist without declaring either one correct.	HYPOTHESIS + experiment-integrity SYSTEM	Evidence leaves timing superiority open; primary behavior includes both self-initiated pauses and pushing through without one. (EU-01, EU-02, EU-07, EU-17–EU-20; EVID-LY JIT delivery and Open questions; EVID-MICRO Open questions)	Preserve channel source and denominator. Movement toward reactive use is product learning, not adherence loss. Both and neither remain possible outcomes.
Entry is a Telegram command-menu item plus an explicit Coach tool call; no persistent reply button or Mini App.	HYPOTHESIS / NO EVIDENCE + SYSTEM	Research did not compare these entry surfaces. The choice minimizes new UI and routes both entries through one service. (EU-ALL, DISC, EVID-LY: no direct interface comparison)	Test discoverability and accidental invocation. Do not interpret low command use before verifying that users could find and understand the action.
On-demand remains available with an active, paused, or absent scheduled sequence, on weekends, and off-hours.	HYPOTHESIS + SYSTEM	A reactive job logically follows the user's recognized need, but the exact availability boundary was not researched. (EU-ALL: no direct availability test)	Access depends on onboarding and entitlement, not plan state. This prevents scheduled lifecycle from becoming a hidden prerequisite for the reactive hypothesis.
Selection is uniform random over eligible switch exercises, excluding only the last successfully delivered on-demand exercise.	HYPOTHESIS / NO EVIDENCE	No interview or evidence review tested random selection, immediate-repeat tolerance, shuffle bags, or recommendation quality. (EU-ALL, DISC, EVID-LY, EVID-MICRO: no direct selection-policy evidence)	Treat equal random plus no immediate repeat as the minimal novelty rule. Measure exposure by content version; do not claim random is therapeutically superior.
One open occurrence per user prevents double delivery; terminal response permits a new request immediately.	SYSTEM	This is a concurrency/idempotency consequence, not a user-research claim.	Enforce in PostgreSQL and reuse the occurrence across retries, commands, Coach calls, and double taps.
The response window is 30 minutes; afterward the occurrence becomes expired and shows <i>Час виконання минув</i> .	HYPOTHESIS + SYSTEM	Research supports bounded micro-actions but does not establish a 30-minute response window or this copy. (EVID-LY, EVID-MICRO: no direct window evidence)	Treat 30 minutes as an operational beta parameter. Record delivery-to-response latency and expiry; do not infer exercise duration or effect.
There is no user-visible quota, therapeutic cooldown, or daily on-demand allowance in MVP.	HYPOTHESIS + SYSTEM	No frequency demand was tested. Evidence cautions against burden and over-prompting, but user-initiated requests are not proactive notification load. (EVID-LY Notification frequency; EVID-MICRO Engagement; EU-ALL: no quota evidence)	Use only invisible abuse protection. Revisit frequency after real use and cost data, not before.

Constructor E — Content Library and exercise-level evidence

Product component or accepted choice	Evidence class	Evidence and mismatch boundary	Consequence for implementation or beta
Exercises are short, concrete, bounded, and require little interpretation.	DIRECT + SECONDARY	Participants used simple context switches; research supports 2–5 minute resets and concrete low-effort micro-interventions. (EU-01, EU-02, EU-05–EU-10, EU-12, EU-15, EU-17–EU-20; DISC §3.2–3.4; EVID-LY Exercise design; EVID-MICRO Exercise design)	Preserve exact steps and duration. Beta still must establish whether the product-delivered version is acceptable and useful.
The target catalogue contains six independent switch and three independent unload records.	HYPOTHESIS / NO DIRECT EVIDENCE	Research supports several intervention families, not this exact nine-record taxonomy, balance, or quantity. (EVID-LY Exercise design; EVID-MICRO Exercise design; EU-ALL: no library-size test)	Catalogue size and mix are versioned beta protocol choices. Do not market “nine evidence-proven exercises.”
breathing switch exercise.	SECONDARY + TENSION	Breathwork has adjacent evidence for stress recovery, but strongest protocols often exceed the shortest LY Workday duration and rely on repetition. Primary interviews did not report this exact technique. (EVID-LY Executive summary and Exercise design; EVID-MICRO Executive summary; EU-01 mentions meditation research, not personal breathing use)	Keep claims modest, instructions precise, and collect content-version feedback. Required GIF supports technique quality but does not strengthen efficacy evidence.
fist PMR switch exercise.	HYPOTHESIS / NO DIRECT EVIDENCE	The reviewed primary interviews and core evidence summaries do not directly validate this exact localized PMR protocol. (EU-ALL, DISC, EVID-LY, EVID-MICRO: no exact-protocol evidence recorded)	Treat as a beta content hypothesis with required instructional media and external review before broad claims.
surface touch switch exercise.	SECONDARY + HYPOTHESIS	Sensory grounding is supported as an intervention family, but this exact surface-touch instruction was not tested in interviews. (EVID-MICRO Executive summary and Exercise design; EU-ALL: no exact instruction)	Validate feasibility, accessibility, and reported usefulness by version.
distant gaze switch exercise.	DIRECT mechanism match + SECONDARY	Participants described looking out a window, sky, nature, or walking outside to reset attention. The exact duration and steps remain untested. (EU-01, EU-05, EU-10, EU-12; DISC §3.3; EVID-LY movement/attention-shift implications)	Strongest catalogue link to observed behavior; still measure the delivered instruction rather than assuming equivalence to a spontaneous pause.
one sound switch exercise.	DIRECT adjacent behavior + HYPOTHESIS instruction	Participants used silence, music, or reduced information load, but no one described the exact one-sound protocol. (EU-01, EU-05, EU-09, EU-14; DISC §3.3)	Treat the protocol as a content hypothesis derived from an observed attention/context need.
cool-water facial immersion switch exercise.	HYPOTHESIS + NO CORPUS EVIDENCE + safety SYSTEM	The feature originated outside the reviewed interview/R&D corpus. No supplied primary or secondary source validates this exact exercise. (EU-ALL, DISC, EVID-LY, EVID-MICRO: no recorded match)	Keep the explicit medical-review and media gate. Do not deliver before approval or generalize a founder anecdote into user evidence.
one-item brain dump unload exercise.	SECONDARY + TENSION	One participant used journaling with Claude, but occupational expressive-writing evidence is mixed and can increase stress. (EU-17; EVID-LY Executive summary, Exercise design, Open questions; EVID-MICRO expressive-writing summary)	Keep the action shallow, optional, and non-therapeutic. Review effect feedback and negative responses before wider claims.
one thing that went well unload exercise.	SECONDARY + HYPOTHESIS	Positive reflection has better adjacent support than expressive unloading, and one participant used weekly “what was good / what to improve” review. The exact micro-action was not tested. (EU-17; EVID-MICRO Exercise design; EVID-LY writing discussion)	Treat wording and immediate usefulness as beta questions; avoid gratitude pressure or emotional interpretation.
first step tomorrow unload exercise.	HYPOTHESIS / NO DIRECT EVIDENCE	The source set does not test this exact planning action. Motivation/to-do overload can be harmful when it expands demands. (EU-02; EU-ALL, EVID-LY: no exact-protocol support)	Keep it to one bounded item and verify that it reduces rather than creates cognitive load.
The on-demand launch pool is five switch exercises; approved cool water makes six.	HYPOTHESIS / NO EVIDENCE	No source establishes that five or six actions are sufficient for novelty, adherence, or effect. (EU-ALL, DISC, EVID-LY, EVID-MICRO: no pool-size evidence)	Enable only when all five are release-eligible, then observe exposure, repetition complaints, and request return before expanding the library.
Stable exercise ID, content version, structured requirements, review status, and fail-closed eligibility.	SYSTEM	These are reproducibility, safety, telemetry, and accessibility controls. They do not claim user demand.	Implement the library, builder, renderer, telemetry, and migrations as one versioned contract.
Instructional GIFs are required for breathing, fist PMR, and cool water; text remains authoritative.	HYPOTHESIS execution support + SYSTEM	The reviewed corpus does not compare GIF versus text. Technique sensitivity provides the rationale, not outcome evidence. (EU-ALL, EVID-LY, EVID-MICRO: no media-format comparison)	Produce and version assets before eligibility. Record <code>delivery_variant</code> ; media failure falls back to complete text.

Constructor F — Presentation, feedback, Coach, and summary

Product component or accepted choice	Evidence class	Evidence and mismatch boundary	Consequence for implementation or beta
The lock-screen preview is neutral; title, duration, steps, and actions appear inside Telegram.	SECONDARY + HYPOTHESIS	Workplace privacy and stigma concerns support discretion, but no source tested Пауза versus Перерва, curiosity, or Telegram truncation. (HR §4.2; EVID-LY Privacy and trust; EVID-MICRO Privacy; EU-ALL: no preview test)	Device-test the exact preview. Do not optimize notification opens as a value metric.
Scheduled and on-demand exercise messages share one canonical renderer and <code>ExercisePresentation</code> .	SYSTEM	One presentation authority prevents channel drift; this is not a user-research claim.	Route channel metadata and persistence outside the renderer. The renderer receives canonical content plus presentation state, not plan ownership.
The exercise message contains only title, duration, exact steps, Виконано, and Пропустити.	SECONDARY + HYPOTHESIS + SYSTEM	Concrete low-load actions and shame-free skip are evidence-aligned; the exact information architecture was not tested. (DISC §3.3–4; EVID-LY Exercise design/Engagement; EVID-MICRO Exercise design/Retention)	Verify comprehension and callback behavior. Do not add rationale, counters, streaks, or inferred state without new evidence and decision.
Optional <code>better / same / worse</code> feedback appears only after registered completion.	SECONDARY + HYPOTHESIS	Evidence reviews recommend immediate state change and perceived usefulness rather than inferred wellbeing. The exact three-label interaction is untested. (EVID-LY Recommended MVP metrics; EVID-MICRO MVP metrics and Pilot design)	Keep it optional and source-linked. Use it for content learning, not clinical or productivity claims.
The cycle summary is a deterministic Telegram image with factual completion and next-delivery data.	HYPOTHESIS + SYSTEM	One participant used weekly reflection, but no source validates a completion grid, image format, or automatic-continuation artifact. (EU-17; EU-ALL: no exact summary test)	Treat it as loop-closure UX. Measure successful delivery and next-cycle response rather than assuming the image creates retention.
On-demand completion appears in the cycle summary only when used, with no shared adherence denominator.	SYSTEM + HYPOTHESIS copy	This protects the experiment from denominator mixing; user-facing wording was not researched.	Omit zero usage, keep scheduled adherence intact, and use a count-only combined total.
Coach is a reactive, non-clinical support and control channel.	DIRECT weak signal + HYPOTHESIS	One participant reported using Claude for conversation, journaling, and weekly review; another used a wellness app. Most interviews did not test AI Coach demand. (EU-17; EU-04 rejects wellness apps; EU-ALL)	Coach value remains a separate beta hypothesis. Distinguish free-text support, product questions, controls, and explicit on-demand exercise requests.
Coach tool execution uses a bounded tool-result loop and deterministic backend authorization.	SYSTEM	This follows consistency and authorization requirements, not market research.	The model may select an allowed tool but never becomes the source of truth for state, access, delivery, or success.
Coach has no user-visible quota, while invisible concurrency, abuse, and cost controls apply.	HYPOTHESIS + SYSTEM	No primary evidence establishes expected Coach frequency or tolerance for quotas. (EU-ALL: no quota test)	Measure cost per complete turn and refusal events privately; revisit only with beta usage and an explicit loss budget.
Crisis handling routes to bounded safety language and professional/emergency support rather than therapy by the bot.	SECONDARY + SYSTEM	Evidence reviews require trauma-informed safeguards and non-clinical boundaries; interviews were not crisis-efficacy tests. (EVID-LY positioning/privacy; EVID-MICRO Trauma-informed safeguards)	Test deterministic detection/routing and do not claim treatment, monitoring, or human escalation unless it exists.

Constructor G — Telemetry, privacy, company reporting, and experiment truth

Product component or accepted choice	Evidence class	Evidence and mismatch boundary	Consequence for implementation or beta
Telemetry measures delivery and registered interaction, not verified exercise execution, wellbeing, burnout, or productivity.	SECONDARY + SYSTEM	Evidence supports proximal behavioral telemetry, short self-report, and bounded claims; hard outcome inference is not justified. (EVID-LY Executive summary, Recommended MVP metrics; EVID-MICRO MVP metrics, Safe/unsafe claims)	Event names, dashboards, summaries, and sales interpretation must retain this semantic ceiling.
Scheduled and on-demand sources, occurrences, denominators, and retention views stay separate.	SYSTEM	This is required to observe channel preference without constructing a misleading combined adherence rate.	Show scheduled-only, on-demand-only, both, and neither descriptively; do not infer causal superiority from self-selection.
Stable event identity, plan-step/exercise/content-version linkage, idempotency, and reconciliation are mandatory.	SYSTEM	These are experiment-integrity and failure-recovery requirements.	No beta metric is interpretable until seeded trajectories reconcile against database state.
User-linked data is personal data; deletion removes it while an independently aggregated contribution may remain.	SECONDARY + SYSTEM	Privacy, minimum collection, transparency, and no hidden profiling are strongly supported. Exact parallel-aggregate architecture is an engineering/legal design. (HR §4.2/C2; EVID-LY Privacy and trust; EVID-MICRO Privacy/Data Protection)	Separate identity/access, personal behavior, operational ledgers, and independent aggregates. Test account deletion across every store.
The company receives no individual behavior, Coach text, channel split, exercise history, or small-group view.	DIRECT buyer-side + SECONDARY + SYSTEM	Confidentiality is a repeated adoption condition. Buyer desire for aggregate visibility does not authorize individual monitoring. (HR §4.2, C2, D2; POS buyer discussion; EVID-LY Privacy; EVID-MICRO B2B reporting)	Preserve the employee trust boundary in schema, authorization, reports, contracts, and product language.
A post-pilot company summary is limited to eligible employees, deployment enrollments, and active users after a 100-eligible/50-contributor gate.	SECONDARY need + HYPOTHESIS policy + SYSTEM	Buyer research supports aggregate adoption visibility. It does not establish these exact three fields or the 100/50 threshold as market-standard anonymity. (HR B4/C2/D2 and §6; EVID-LY Privacy; EVID-MICRO B2B reporting)	Describe 100/50 as a conservative policy, not a mathematical anonymity guarantee. Re-review against real cohort and legal advice.
Founder-only beta analytics may inspect pseudonymous individual trajectories but no company dashboard is built.	SYSTEM	Small-cohort product learning requires restricted diagnosis of the mechanism; it is not employee- or buyer-requested functionality.	Restrict access, omit conversation/free text, print raw n and observation cutoffs, and delete personal history with the account.
D1/D3/D7, completion, feedback, and continuation thresholds are directional priors for a 10–15-user beta, not automatic product verdicts.	SECONDARY + SYSTEM	Evidence is heterogeneous and engagement clusters in subsets; the research documents themselves label adoption/retention as beta questions. (EVID-LY Adherence/Open questions; EVID-MICRO Engagement/Open questions; HR §7; ASSUMPTIONS)	Interpret quantitative results with delivery integrity and qualitative context. Do not manufacture statistical confidence.

Constructor H — Architecture and operations that do not require user-demand evidence

System module or accepted choice	Evidence class	Why it exists	Required boundary
Plan-centric lifecycle with derived mode; no duplicate stored FSM truth.	SYSTEM	Prevent contradictory plan/user state and make operations atomic.	Migrate once, constrain one current plan, and remove old state writers/readers rather than synchronizing them indefinitely.
PostgreSQL is durable business truth; Redis is transient coordination/session state only.	SYSTEM	Restart, expiry, entitlement, delivery, and callback outcomes cannot depend on process memory or an evictable cache.	Every durable transition and idempotency key has a PostgreSQL authority and reconciliation path.
Scheduler, delivery, callbacks, expiry, continuation, and on-demand occurrence use atomic state transitions.	SYSTEM	Telegram retries, double taps, process restarts, media failure, and partial writes are expected distributed-system conditions.	Conditional updates/constraints decide the winner; duplicate work returns the authoritative existing result.
Shared canonical renderer with channel-specific routing and persistence.	SYSTEM	Prevent scheduled/on-demand presentation drift without merging their lifecycles.	The renderer does not own selection, plan progression, occurrence status, or telemetry source.
Versioned forward migrations, startup schema checks, backup/restore, health/readiness, graceful shutdown, alerts, and one reproducible release artifact.	SYSTEM	These protect data and delivery and make findings implementable. No interview is expected to request them.	Audit closure does not equal production readiness; release proceeds only after the recorded DB/SEC/OPS gates pass.
Production/testnet isolation and provider-specific secrets, databases, Redis, bots, OpenAI projects, backups, and aggregate sinks.	SYSTEM	Test contamination would make privacy and metrics irreversible.	Build from the same release behavior but never share data or identity namespaces.
Removal of pulse, re-engagement, streak, persona, adaptation, legacy reports, old plan flows, and dead dev/test models.	SECONDARY + SYSTEM	Extra prompts risk fatigue; inferred personalization and evaluative messaging conflict with trust and the accepted minimal model. Dead code can still be accidentally reactivated. (EU-02; EVID-LY Adherence/Engagement/Privacy; EVID-MICRO Engagement/Privacy)	Delete rather than merely disable rejected mechanisms; replace tests that preserve retired architecture.

Evidence gaps that beta must keep visible

The reviewed research supports the existence of a workday recovery/context- switch problem and the need for privacy, autonomy, bounded claims, and low-friction action. It does **not** establish product-market fit or validate the assembled interface. The following remain explicit beta questions:

1. Will an employee use a company-provided Telegram product at all?
(HR C1/C5/D1; ASSUMPTIONS.)
2. Does scheduled delivery produce useful continued interaction, notification fatigue, or neither? (EVID-LY Open questions; EVID-MICRO Open questions.)
3. Do users independently request Вправа зараз, and do they return to it?
(EU-ALL: existing self-initiated coping, no feature test.)
4. Do users prefer scheduled, on-demand, both, or neither when both are available? No causal answer is available from self-selection alone.
5. Are the exact five/six switch exercises feasible, acceptable, sufficiently varied, and perceived as useful? (EVID-LY Exercise design/Open questions.)
6. Are seven/fourteen working-day cycles and automatic continuation helpful or an unwanted default? (EU-ALL: no direct validation.)
7. Does Coach solve a repeated user job distinct from exercise delivery and product controls? (EU-17 is a weak direct signal, not validation.)
8. Does the privacy/enrollment explanation create enough trust without making the funnel too heavy? (HR confidentiality signal; exact flow untested.)
9. Does the cycle summary close the loop or merely add another message?
(EU-ALL: no exact artifact evidence.)
10. Can the complete mechanism run reliably enough that behavioral results are about the product rather than delivery, schema, or concurrency defects? (DB/SEC/OPS findings.)

Alignment conclusion

The assembled LY Workday model is **aligned at the problem and boundary level**: short workday context switches, low cognitive load, autonomy, non-clinical positioning, privacy, and no employer-level individual surveillance all have credible direct or secondary support.

It is **not yet validated at the product-configuration level**. Telegram, scheduled cadence, the reactive entry UI, seven/fourteen-day cycles, automatic continuation, exact content catalogue, random selection, GIF requirements, Coach, and the cycle image are product or system hypotheses. That is acceptable for an MVP only if telemetry preserves the distinctions above and beta results are not retold as evidence that existed before the beta.

The scheduled-plus-on-demand model is therefore a defensible test portfolio, not a research conclusion. It lets the product observe scheduled preference, reactive preference, mixed use, and non-use without making one channel's denominator absorb the other. The honest outcome may be scheduled, reactive, both, or neither.

Miscellaneous Findings

Standalone findings that don't yet belong to an audited area above. File each one under its proper area (with a proper Audit Round) once that area gets its own audit pass. Do not leave them here permanently if a matching area already exists — check first.

MISC-01 — Company-level timezone model for B2B onboarding

Filed 2026-08-15 → see COMP-06 in the Company Deployment Findings. Retained here as a pointer only; the decision text and current code facts now live in that area, per this section's filing rule.

MISC-02 — Time Picker UX (Phase 3, not MVP)

Status: product decision confirmed (2026-07-15), not implemented Future area: Delivery UX / Choice Prompts Infrastructure (not yet audited as its own area)

Decision: buttons-first UX for time selection — preset time buttons + "и́нший час" opens a deterministic hour → 15-minute picker (2 taps, no free-text HH:MM, no LLM involved). Same reusable keyboard component across onboarding, first evening-time collection, and later time changes. Button selection counts as confirmation (no redundant "точно?" follow-up).

Requires (not built): a new `show_time_picker(target)` Coach tool (`target: DAY / EVENING / UNSPECIFIED`), a callback schema like `time:day:15:00`, callbacks that call runtime tools directly without an LLM round-trip.

Explicitly deferred — not MVP. Coach currently uses a natural-language fallback (Section 7 Time Arguments) instead.

Move to a proper "Delivery UX" audit round when that area is inspected.

MISC-03 — Remove stale product and architecture documents before production

Status: required pre-production documentation cleanup

After the accepted core refactors are implemented, inventory repository documentation, prompts, context files, and other artifacts accessible to AI agents. Remove or clearly archive superseded concepts, abandoned architecture, and stale product behavior so they cannot be treated as current implementation instructions. Preserve history in Git rather than leaving contradictory files inside the active context surface.

MISC-04 — Finalize the Product Contract after core implementation stabilizes

Status: required pre-production contract synchronization

After all accepted core refactors and feature work are implemented, update `docs/audit/product_contract.md` to match the final shipped product, data, privacy, enrollment, telemetry, lifecycle, and operational contracts. This is a final consistency gate before production, not a reason to keep rewriting the contract while implementation is still moving.

MISC-05 — Synchronize the public website before production

Status: required pre-production product-language synchronization

After the product and final Product Contract stabilize, review and update the public website so its name, capabilities, claims, privacy boundary, enrollment model, and user journey describe the product that actually ships. Do not use the current website as an implementation source of truth during refactoring.

MISC-06 — Retire or rebuild development tools against the target model

Status: required implementation/release cleanup

The repository contains `devtools/common.py`, `devtools/fsm.py`, `devtools/plan.py`, `devtools/smoke.py`, `devtools/spawn_tasks.py`, and `scripts/run_coach_stress_tests.py`. They are manual development entrypoints, not imported production application routes, but several encode the retired 21-day/generated-plan/adaptation/FSM model, call internal implementation helpers, or can create and reschedule database rows directly. Their environment guard also reads `ENV`, while the application configuration authority uses `ENVIRONMENT`.

Do not carry these tools forward as if green execution proved the target LY Workday model. During implementation either:

1. delete the obsolete tools and their stale fixtures; or
2. rebuild a minimal explicit toolset for the accepted lifecycle, scheduled

delivery, on-demand occurrence, telemetry reconciliation, and deterministic end-to-end smoke path.

Exclude development-only tooling from the canonical production artifact unless a specific operational tool is deliberately retained, documented, tested, and guarded by the same fail-closed environment contract as the application. This finding does not require speculative internal tooling before implementation; it prevents old tools from silently becoming the test oracle for the new model.